
ROADASSIST BHARAT

RoadAssist Bharat

Project Overview

| *One Platform. Every Vehicle. Every Phone. Every Road.*

Document README.md

Version 1.0 — planning baseline

Team 4 engineers · 22 weeks · 11 sprints · 18 phases

Status Baseline (frozen at Sprint 0 exit)

RoadAssist Bharat

One Platform. Every Vehicle. Every Phone. Every Road.

An offline-first, AI-powered roadside assistance and vehicle-safety platform for India — spanning cars, two-wheelers, autos, trucks, buses, tractors, and EVs, across Android, iOS, web, Android Auto, and feature phones (SMS/IVR/USSD).

Planning documents

Document	Contents
Team Charter	Team composition, RACI, rituals, sprint calendar, git/PR/commit rules, code review checklist, Definition of Done, documentation standards, quality gates, risk register
Master Roadmap	All 18 phases with objectives/tasks/dependencies/deliverables/acceptance criteria, target architecture, why each service exists, business model
D1 — Backend & Data Lead	Per-phase plan: schema (60 tables), 140 APIs, booking state machine, dispatch, auth/RBAC, sync conflict resolution, emergency engine
D2 — Frontend & Mobile Lead	Per-phase plan: design system, 48 screens, 42 components, offline client, maps, gov portal, accessibility, 8 languages
D3 — AI & Data Science Lead	Per-phase plan: 9 AI systems with inputs/outputs/algorithms/training data/metrics/baselines, on-device bundle, analytics
D4 — DevOps, QA & Security Lead	Per-phase plan: K8s/Terraform/CI-CD, observability, telecom gateway, load/chaos/security testing, deployment, DR

Team

ID	Role	Owns
D1	Backend & Data Lead	Services, PostgreSQL/PostGIS, event bus, API contracts, auth, booking/dispatch, emergency, payments
D2	Frontend & Mobile Lead	React Native app, Next.js portals, design system, offline client, maps UI, Android Auto, a11y, i18n
D3	AI & Data Science Lead	9 AI subsystems, model training/serving, on-device inference, feature store, voice/NLU, analytics
D4	DevOps, QA & Security Lead	Kubernetes, Terraform, CI/CD, observability, all testing, security, telecom gateway, compliance

Timeline

22 weeks · 11 sprints of 2 weeks · 18 phases · 10 milestones

Sprint	Weeks	Phases	Milestone
S0	1–2	Research, Planning	M0 — Baseline frozen
S1	3–4	Architecture, Database	M1 — Skeleton alive
S2	5–6	Auth, Backend APIs (I)	M2 — Login works
S3	7–8	Backend APIs (II), Frontend (I)	M3 — First booking
S4	9–10	Frontend (II), AI (I)	M4 — Vertical slice
S5	11–12	AI (II), Offline Engine	M5 — Works with no internet
S6	13–14	Maps, Vehicle Diagnostics	M6 — Diagnose & route
S7	15–16	Emergency, Gov Dashboard	M7 — Crash to responder
S8	17–18	Analytics, Testing	M8 — Quality bar
S9	19–20	Deployment, Optimization	M9 — Production
S10	21–22	Future Roadmap, GA hardening	M10 — GA + pitch

Non-negotiables

1. **Emergency paths never regress.** Two approvals, dedicated test run, staged rollout.
2. **Offline is a first-class mode, not a fallback.**
3. **Feature phones are users.** Every core journey completable over SMS/IVR.
4. **No PII leaves India.** Including logs, backups, and crash reports.
5. **We never fake it in a demo.** If it's mocked, we say so.

Status: planning baseline. No implementation code yet — S0 begins with Phase 0 research.

ROADASSIST BHARAT

Team Charter

Engineering Constitution

| *One Platform. Every Vehicle. Every Phone. Every Road.*

Document docs/00-team-charter.md

Version 1.0 — planning baseline

Team 4 engineers · 22 weeks · 11 sprints · 18 phases

Status Baseline (frozen at Sprint 0 exit)

RoadAssist Bharat — Team Charter & Engineering Constitution

"One Platform. Every Vehicle. Every Phone. Every Road." Document owner: CTO · Version 1.0 · Status: Baseline (frozen at Sprint 0 exit)

1. Team Composition

Four engineers. Every person is a **Lead** — meaning they own an architectural vertical end-to-end, not a ticket queue. No one is a "helper."

ID	Role	Owns	Secondary (backup)
D1	Backend & Data Lead	All Node/NestJS services, PostgreSQL/PostGIS schema, Kafka event bus, API contracts, payments	Backs up D3 on AI-gateway plumbing
D2	Frontend & Mobile Lead	React Native app, Next.js portals (Gov / Mechanic / Admin), design system, offline UI, Android Auto	Backs up D4 on E2E test authoring
D3	AI & Data Science Lead	All 9 AI subsystems, model training/serving, on-device TFLite, telemetry feature store, IVR/NLU	Backs up D1 on analytics service
D4	DevOps, QA & Security Lead	Kubernetes, Terraform, CI/CD, observability, load/security testing, SMS/IVR gateway ops, compliance	Backs up D2 on portal builds

Rotating hats (1 sprint each, round-robin): - **Release Captain** — owns the sprint's release train, cuts the tag, writes release notes. - **Incident Commander** — on-call for staging/prod, runs postmortems. - **Scribe** — keeps ADRs and the decision log current.

Named slots (fill in):

D1 = _____ D2 = _____

D3 = _____ D4 = _____

2. Ownership Matrix (RACI)

Area	D1	D2	D3	D4
System architecture / ADRs	A	C	C	C
Database schema & migrations	A/R	I	C	C
API contract (OpenAPI/proto)	A/R	C	C	I
Auth, RBAC, token lifecycle	A/R	C	I	C
Booking / Dispatch engine	A/R	C	C	I

Area	D1	D2	D3	D4
Mobile app (RN)	C	A/R	I	C
Web portals (Next.js)	C	A/R	I	C
Design system & a11y	I	A/R	I	I
Offline DB + sync client	C	A/R	C	I
Sync conflict resolution (server)	A/R	C	I	I
AI models & training	I	I	A/R	C
On-device inference (TFLite/ONNX)	I	C	A/R	I
Voice assistant / IVR NLU	I	C	A/R	C
Maps, routing, geofencing	C	C	C	A/R shared with D1
Emergency (112 / SOS) engine	A/R	C	C	C
Government dashboard	C	A/R	C	I
Analytics / ClickHouse	C	I	A/R	C
Kubernetes, Terraform, CI/CD	I	I	I	A/R
Observability & SLOs	C	C	C	A/R
Security testing & pentest	C	C	C	A/R
Load & chaos testing	C	I	C	A/R
DPDP / compliance evidence	C	I	C	A/R
Pitch deck / business model	C	C	C	C — A = CTO

A = Accountable · R = Responsible · C = Consulted · I = Informed

3. Cadence

Daily

- **09:45 IST — Standup (12 min, hard stop).** Format below. Async in Slack [#ra-standup](#) if anyone is travelling.
- **Focus block 10:00–13:00.** No meetings, no PR review, notifications off.
- **16:30 — PR sweep.** Every open PR must be reviewed or explicitly deferred with a reason.
- **18:00 — Green build gate.** [main](#) must be green at EOD. Whoever broke it fixes it or reverts it.

Standup format (each person, 3 lines max):

```

Yesterday : <shipped thing + PR link>
Today      : <one primary outcome>
Blocked    : <blocker + who unblocks it, or "none">

```

Discussion is deferred to a "parking lot" after the 12 minutes with only the affected people.

Weekly

Day	Ritual	Duration	Who
Mon	Sprint planning (odd weeks) / Mid-sprint checkpoint (even weeks)	60 min	All
Tue	Architecture review — ADRs raised last week	45 min	All
Wed	Design review — Figma → build parity	30 min	D2 + 1
Thu	Risk & security review — new threats, dependency CVEs	30 min	D4 + all
Fri	Demo (recorded, 20 min) + Retro (25 min)	45 min	All

Sprint

2 weeks. 10 working days. Capacity per person per sprint: **8 ideal days** (2 days reserved for review, support, and unplanned work). Team velocity target: **32 ideal days/sprint**.

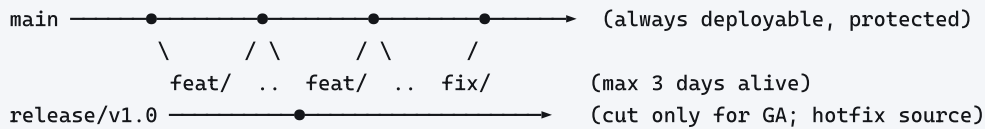
Sprint ends with: 1. A recorded demo of working software (never slides). 2. A tagged release `v0.<sprint>.0` deployed to staging. 3. A retro with at most **2 action items**, each with an owner and a due date.

4. Sprint Calendar & Milestones

Sprint	Weeks	Phases covered	Milestone
S0	1–2	P0 Research, P1 Planning	M0 — Baseline frozen: PRD, ADR-000..010, repo scaffolded, CI green
S1	3–4	P2 Architecture, P3 Database	M1 — Skeleton alive: all services boot, migrations run, ER diagram signed off
S2	5–6	P4 Auth, P5 Backend APIs (I)	M2 — Login works: OTP → JWT → RBAC across 3 client types
S3	7–8	P5 Backend APIs (II), P6 Frontend (I)	M3 — First booking: end-to-end request → mechanic accept → complete
S4	9–10	P6 Frontend (II), P7 AI (I)	M4 — Vertical slice: mobile app + live dispatch on a map
S5	11–12	P7 AI (II), P8 Offline Engine	M5 — Works with no internet: airplane-mode booking that syncs
S6	13–14	P9 Maps, P10 Vehicle Diagnostics	M6 — Diagnose & route: OBD/photo diagnosis + offline routing
S7	15–16	P11 Emergency, P12 Gov Dashboard	M7 — Crash to responder: auto-detect → SOS → 112 handoff → gov map
S8	17–18	P13 Analytics, P14 Testing	M8 — Quality bar: ≥80% coverage, load test 10k RPS passed
S9	19–20	P15 Deployment, P16 Optimization	M9 — Production: multi-AZ prod, DR drill passed, p95 < 300 ms
S10	21–22	P17 Future Roadmap + GA hardening	M10 — GA + pitch: public beta, pitch deck, pilot MoU draft

5. Git Workflow

Branching model — Trunk-based with short-lived branches



- `main` is protected. No direct pushes. Squash-merge only.
- Branch lifetime $\leq$ **3 days**. If it lives longer, it was scoped wrong — split it.
- Feature flags (`unleash`) over long-lived branches. Always.
- `release/vX.Y` cut at GA. Hotfixes branch from it and cherry-pick back to `main`.

Branch naming

`<type>/<phase>-<owner>-<short-kebab-summary>`

```
feat/p05-d1-booking-state-machine
fix/p08-d2-sync-conflict-toast
chore/p15-d4-argocd-bootstrap
model/p07-d3-crash-detect-v2
docs/p02-d1-adr-012-event-bus
spike/p10-d3-obd-ble-feasibility
```

Types: `feat` `fix` `chore` `refactor` `perf` `test` `docs` `model` `spike` `hotfix`

Commit naming — Conventional Commits, enforced by commitlint

`<type>(<scope>): <imperative summary, ≤72 chars>`

`<body: why, not what – wrap at 100>`

Refs: RA-142

BREAKING CHANGE: booking.status enum renamed PENDING→REQUESTED

Scopes: `auth` `booking` `dispatch` `vehicle` `maps` `emergency` `notify` `sync` `analytics` `gov` `ai` `mobile` `web`
`infra` `db`

Good:

```
feat(dispatch): rank mechanics by ETA-adjusted acceptance probability
fix(sync): drop tombstoned rows before applying remote patch
```

Bad: `update code`, `fixes`, `WIP`, `final final`

6. Pull Request Rules

Hard rules — CI blocks the merge: 1. PR diff $\leq$ **400 changed lines** (excluding lockfiles, generated code, snapshots). Larger needs an explicit `size/xl` label plus CTO approval. 2. **1 approving review minimum, 2 for:** DB migrations, auth/crypto, payment flows, emergency service, anything touching PII. 3. The reviewer must **not** be the author's backup for that area on trivial PRs — rotate reviewers so knowledge spreads. 4. All CI gates green: lint, typecheck, unit, integration, coverage delta ≥ 0 , SAST, dependency audit, license check, container scan. 5. PR description uses the template. An empty template body = automatic request-changes. 6. Migrations must include a tested **down** path or an explicit "forward-only, approved by D1" note. 7. No `TODO` without a ticket ID. No commented-out code. No `console.log` in shipped paths. 8. Review SLA: **first response within 4 working hours**. If you can't, reassign — don't sit on it.

PR template:

```
## What
One paragraph. What changed, in user terms.

## Why
Link the ticket / ADR. What problem does this solve?

## How
Notable design decisions and anything a reviewer would otherwise have to reverse-engineer.

## Risk
- Blast radius: [ ] single service [ ] cross-service [ ] data migration [ ] auth/PII
- Rollback plan:
- Feature flag: `flag.name` (default OFF)

## Verification
- [ ] Unit tests added/updated
- [ ] Integration test covers the happy path + 1 failure path
- [ ] Manually verified on: [ ] Android [ ] iOS [ ] Web [ ] low-end device [ ] airplane mode
- [ ] Screenshots / recording attached (UI changes)
- [ ] Load impact considered (new N+1? new hot query? index present?)

## Docs
- [ ] OpenAPI updated [ ] ADR written [ ] Runbook updated [ ] README updated [ ] N/A + reason
```

7. Code Review Checklist

The reviewer works this list. It's the contract — no vibes-based reviews.

Correctness - [] Does it actually do what the PR says? Read the test, not just the code. - [] Edge cases: empty list, null, zero, negative, duplicate, concurrent, timezone, `NULL` vs missing. - [] Error paths: what happens on network failure, DB timeout, partial write? - [] Idempotency: can this be safely retried? (All write APIs must be — `Idempotency-Key` header.)

Data - [] Migration is backwards-compatible with the currently-deployed code (expand → migrate → contract). - [] New query has a supporting index; `EXPLAIN ANALYZE` attached for anything on a hot path. - [] Soft delete respected (`deleted_at IS NULL` in every read path). - [] Audit log entry written for anything state-changing on a regulated entity.

Security - [] AuthZ check present at the resource level, not just the route level (no IDOR). - [] Input validated with a schema (Zod / class-validator) at the boundary. - [] No raw string SQL concatenation. Parameterized or query-builder only. - [] No secrets, keys, tokens, or PII in code, logs, or error messages. - [] Rate limit applied to any unauthenticated or

expensive endpoint.

Performance - ☐ No N+1. No unbounded query (every list endpoint is paginated, max page size enforced). - ☐ Payload size sane for a 2G connection — is this shipping fields the client never reads? - ☐ Caching considered; cache invalidation path is explicit.

Frontend-specific - ☐ Works offline or degrades honestly (never a silent failure — always a visible state). - ☐ Loading / empty / error / offline states all implemented. - ☐ Touch targets $\geq 44 \times 44$ dp; contrast $\geq 4.5:1$; screen-reader labels present. - ☐ Tested on a 2 GB RAM Android device profile, not just the simulator. - ☐ i18n: no hardcoded user-facing strings.

AI-specific - ☐ Model version pinned; model card updated; eval metrics reported vs. previous version. - ☐ Fallback path when the model is unavailable or confidence is below threshold. - ☐ No PII in training data or feature store without a documented lawful basis. - ☐ Latency and model size budget respected (on-device: ≤ 15 MB, ≤ 120 ms p95).

Craft - ☐ Naming reads like the domain, not like the implementation. - ☐ The code matches the conventions of the file it lives in. - ☐ Comments explain *why*, and only where the why isn't obvious. - ☐ No speculative generality — no abstraction with exactly one caller.

8. Definition of Done

A story is Done only when **all** of these are true. Not "done except tests."

1. Acceptance criteria in the ticket are all demonstrably met.
2. Code merged to `main` via a reviewed PR.
3. Unit tests written; branch coverage on new code $\geq 80\%$.
4. At least one integration test covers the happy path and one failure path.
5. OpenAPI / proto contract updated and the generated client regenerated.
6. Feature flag defined, default OFF, and the ON path tested.
7. Observability: at least one metric, one structured log event, and a trace span exist for the new path.
8. Alert defined if the feature has an SLO.
9. Docs updated: API reference, and the runbook if it can page someone.
10. Deployed to staging and smoke-tested there.
11. Works on the **low-end reference device** (Android 10, 2 GB RAM, 3G throttled) if user-facing.
12. Accessibility pass (WCAG 2.1 AA) if user-facing.
13. Security checklist signed off by D4 if it touches auth, PII, payments, or emergency.
14. Demoed in the Friday demo.

Definition of Ready (before a story enters a sprint): has acceptance criteria, has a design/API contract, has no unresolved external dependency, is estimated, and is ≤ 3 ideal days.

9. Documentation Standards

```
docs/
├── 00-team-charter.md           ← this file
├── 01-master-roadmap.md        ← all 18 phases, cross-team
├── 02-backend-lead-roadmap.md  ← D1
├── 03-frontend-lead-roadmap.md ← D2
├── 04-ai-lead-roadmap.md       ← D3
├── 05-devops-qa-lead-roadmap.md ← D4
├── adr/                        ← Architecture Decision Records, numbered, immutable
│   └── 0012-event-bus-redpanda-over-kafka.md
├── api/                       ← generated OpenAPI + proto, never hand-edited
├── runbooks/                  ← one per alert; every alert links to its runbook
├── models/                    ← one model card per deployed model
└── decisions.md               ← running decision log (date, decision, who, why)
```

Rules - Every non-obvious technical choice gets an **ADR**. Format: Context → Options considered → Decision → Consequences → Status. ADRs are never edited after acceptance; they're superseded by a new one. - Every alert has a **runbook** with: what it means, how to confirm, how to mitigate, how to escalate. An alert without a runbook gets deleted. - Every deployed model has a **model card**: purpose, training data + provenance, metrics, known failure modes, bias evaluation, fallback behaviour, owner. - Diagrams are **Mermaid in markdown**, not images. Diagrams that can't be diffed rot. - READMEs answer exactly four questions: what is this, how do I run it, how do I test it, who owns it. - Docs are updated **in the same PR** as the code. A follow-up docs PR never happens.

10. Communication Protocol

Channel	Use for	Response expectation
#ra-standup	Async standups	Post by 09:45
#ra-dev	Technical discussion	Best effort
#ra-prs	PR notifications	4 working hours
#ra-alerts	Automated alerts only, no chat	On-call: 15 min ack
#ra-incidents	Live incident coordination	Immediate
ADR + PR	Any decision anyone will ask "why?" about in 3 months	2 working days

Escalation: blocked > 2 hours → say so in #ra-dev . Blocked > 4 hours → escalate to CTO. Sitting quietly on a blocker is the only genuinely unacceptable behaviour on this team.

11. Quality Gates (enforced in CI, not by goodwill)

Gate	Threshold	Blocks
ESLint / Ruff	0 errors	PR

Gate	Threshold	Blocks
TypeScript / mypy	strict, 0 errors	PR
Unit coverage (new code)	≥ 80% branch	PR
Integration suite	100% pass	PR
SAST (Semgrep, CodeQL)	0 high/critical	PR
Dependency audit (osv-scanner)	0 critical, high needs waiver	PR
Container scan (Trivy)	0 high/critical in final image	Release
License check	no GPL/AGPL in distributed code	PR
API contract diff	no breaking change without version bump	PR
Bundle size (web)	≤ 180 KB gz initial route	PR
APK size	≤ 28 MB base	Release
Load test	p95 < 300 ms @ 10k RPS	Release
Lighthouse (portals)	Perf ≥ 90, A11y ≥ 95	Release
DAST (ZAP)	0 high	Release

12. Risk Register (top 10, reviewed every Thursday)

#	Risk	Impact	Prob	Owner	Mitigation
R1	Offline sync conflicts corrupt booking state	Critical	Med	D1+D2	CRDT-style LWW per-field + server-authoritative state machine; conflict test suite from S5
R2	SMS/IVR gateway DLT approval delays (TRAI)	High	High	D4	Start DLT template registration in S0; keep 2 gateway vendors integrated
R3	Crash-detection false positives dispatch real 112 units	Critical	Med	D3	Two-stage confirm (sensor + 30s user cancel window + call-back); never auto-dispatch on model alone
R4	Map/routing licensing cost at national scale	High	Med	D4	OSM + self-hosted Valhalla/MapLibre as the default; commercial tiles only as premium fallback
R5	DPDP Act non-compliance (consent, localization)	Critical	Med	D4	Consent ledger from P4; all data in-India regions; DPIA before beta
R6	On-device AI too heavy for 2 GB devices	High	Med	D3	Hard budget: ≤15 MB, INT8 quantized, ONNX/TFLite; server fallback
R7	Mechanic supply density too low for ETA promises	High	High	PM/D1	Dispatch honestly surfaces "no mechanic within X"; pre-seed partner network per pilot city
R8	4-person team under-capacity for 18 phases	High	High	CTO	Phases 12/13/17 are deliberately thin-sliced; feature flags let scope drop without branch surgery

#	Risk	Impact	Prob	Owner	Mitigation
R9	Emergency service outage during a real emergency	Critical	Low	D4	Multi-AZ + degraded SMS-only path that works with the whole app stack down
R10	Key-person dependency (single lead per vertical)	High	Med	All	Mandatory backup owner (\$1), rotating reviewers, ADRs, no undocumented tribal knowledge

13. Non-Negotiables

1. **Emergency paths never regress.** Any change touching SOS/112 needs 2 approvals, a dedicated test run, and a staged rollout.
2. **Offline is a first-class mode, not a fallback.** If a feature can't degrade, it needs an explicit, visible offline state.
3. **Feature phones are users.** Every core journey (request help, track, cancel, confirm) must be completable over SMS/IVR.
4. **No PII leaves India.** Ever. Including logs, backups, error reports, and third-party analytics.
5. **We never fake it in a demo.** If it's mocked, we say it's mocked, on the slide.

ROADASSIST BHARAT

Master Roadmap

Phases 0-17

| *One Platform. Every Vehicle. Every Phone. Every Road.*

Document docs/01-master-roadmap.md

Version 1.0 — planning baseline

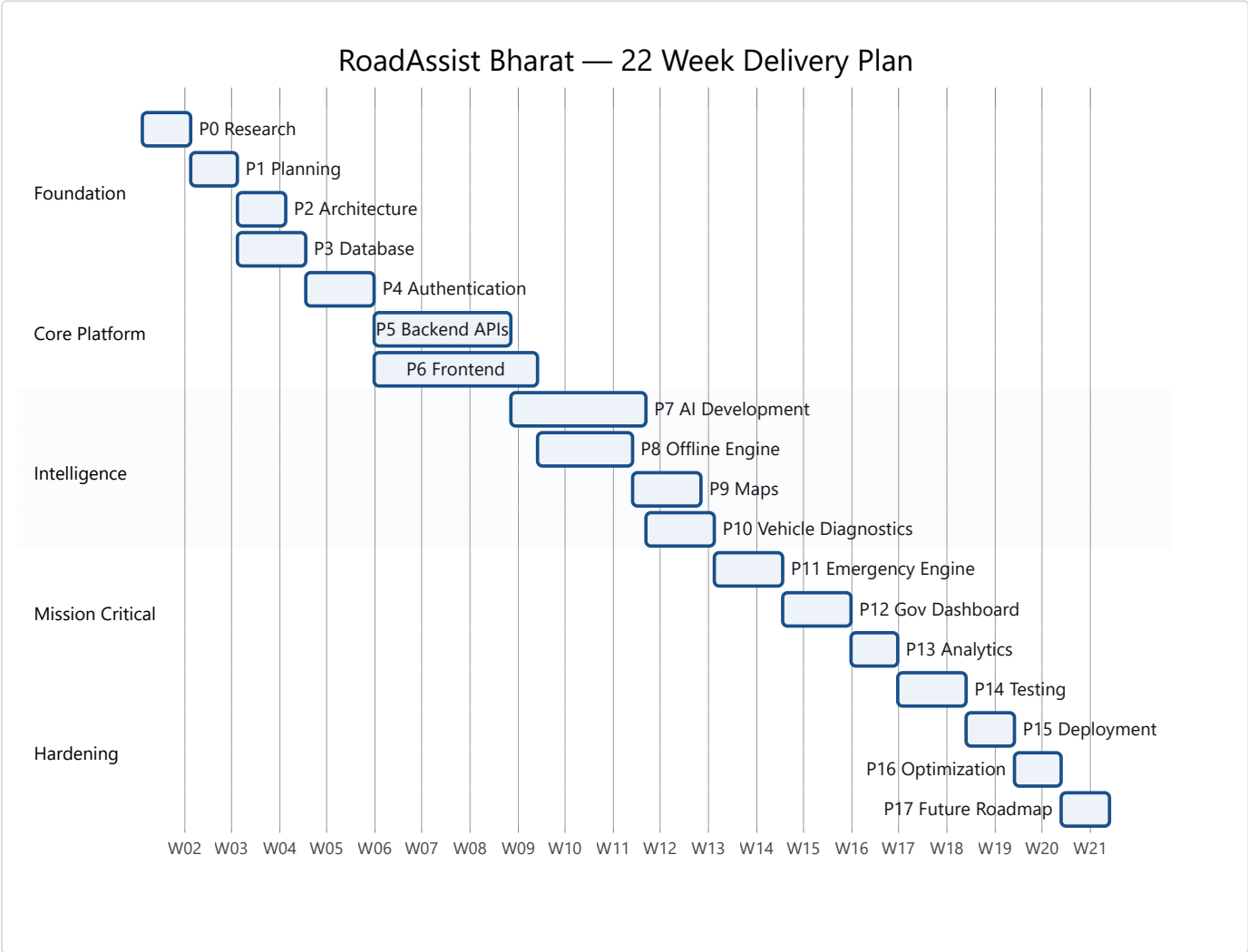
Team 4 engineers · 22 weeks · 11 sprints · 18 phases

Status Baseline (frozen at Sprint 0 exit)

RoadAssist Bharat — Master Roadmap (Phases 0–17)

Cross-team view. Each person's day-by-day work lives in their own roadmap file: [D1 Backend](#) · [D2 Frontend](#) · [D3 AI](#) · [D4 DevOps/QA](#)

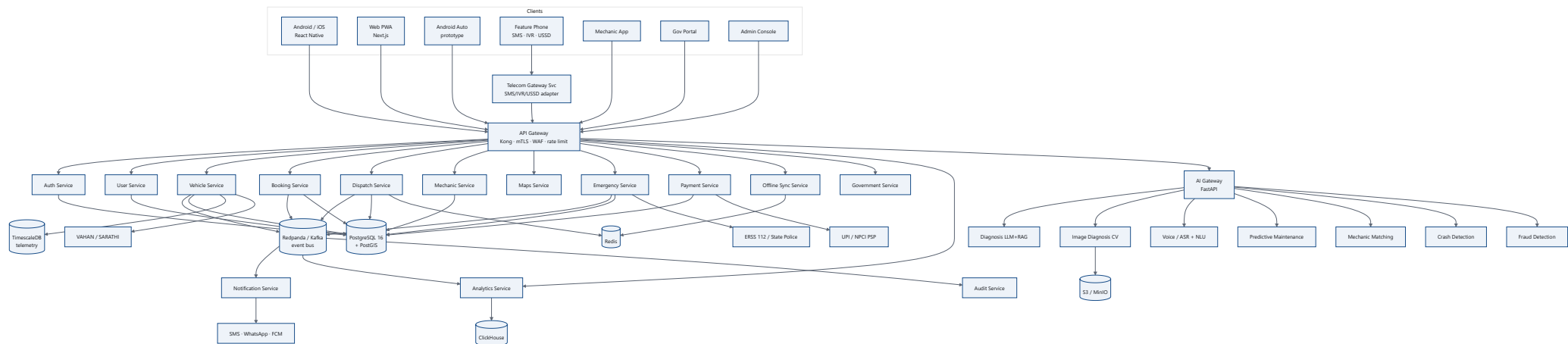
Phase → Sprint → Owner Map



Phase	Sprint	Primary	Support	Exit gate
P0 Research	S0	All	—	Research dossier signed off by CTO
P1 Planning	S0	CTO + all	—	PRD + backlog + estimates frozen
P2 System Architecture	S1	D1	D3, D4	ADR-001..015 accepted; C4 diagrams merged
P3 Database Design	S1	D1	D3	Migrations run clean up + down on empty DB
P4 Authentication	S2	D1	D2, D4	OTP→JWT→RBAC works on mobile + web + SMS
P5 Backend APIs	S2–S3	D1	D3	100% of v1 OpenAPI implemented + contract-tested

Phase	Sprint	Primary	Support	Exit gate
P6 Frontend	S3–S4	D2	D1	Full booking journey on Android/iOS/Web
P7 AI Development	S4–S5	D3	D1	5 models served, each beating its baseline
P8 Offline Engine	S5	D2	D1, D4	Airplane-mode journey completes and syncs
P9 Maps	S6	D2	D1	Offline tiles + routing + live tracking
P10 Vehicle Diagnostics	S6	D3	D2	OBD-II + photo + symptom triage all live
P11 Emergency Engine	S7	D1	D3, D4	Crash → SOS → responder handoff, <10 s
P12 Government Dashboard	S7	D2	D3	Live incident map + 6 statutory reports
P13 Analytics	S8	D3	D1	ClickHouse pipeline + 4 dashboards
P14 Testing	S8	D4	All	All quality gates green
P15 Deployment	S9	D4	D1	Prod multi-AZ, blue/green, DR drill passed
P16 Optimization	S9	D4	All	p95 <300 ms, APK <28 MB, cost/user modelled
P17 Future Roadmap	S10	CTO + all	—	18-month roadmap + pitch deck + pilot MoU

Target Architecture



Why each service exists

Service	Exists because	Would break without it
API Gateway	Single ingress for auth, rate limiting, WAF, request signing, protocol translation. Clients on 2G need one TLS handshake, not twelve.	Every service re-implements auth; mobile battery and latency suffer
Auth Service	Identity is the one thing that must never be duplicated. Owns OTP, JWT issuance, refresh rotation, device binding, RBAC claims.	Inconsistent session semantics; token replay attacks
User Service	Profiles, consent ledger, KYC state, preferences, language. Split from Auth so PII has a separate blast radius and its own encryption keys.	PII sprawl across every service; DPDP non-compliance
Vehicle Service	The domain is vehicle-centric, not user-centric: one user, many vehicles; one vehicle, many owners over time. Owns VAHAN sync, OBD telemetry ingest, service history.	Can't model fleets, resale, or per-vehicle prediction
Booking Service	Owns the booking state machine and money-adjacent invariants. Deliberately boring, heavily tested, no AI in the critical path.	Double-booking, lost requests, payment disputes
Dispatch Service	Real-time matching is a fundamentally different workload (geospatial, low-latency, in-memory, high churn) than transactional booking. Separated so it can scale and fail independently.	Dispatch load takes down bookings
Mechanic Service	Supply-side lifecycle: onboarding, verification, skills, inventory, payouts, ratings, availability. Different actors, different SLAs.	Supply and demand logic tangle; can't onboard partners independently
Maps Service	Centralizes tiles, geocoding, routing, ETA, geofencing, and offline map-pack generation so vendor choice is one swap, not thirty.	Vendor lock-in; per-client licence costs; inconsistent ETAs
Emergency Service	Life-safety has a different reliability class: separate deploy, separate quota, degraded-mode path that survives a full platform outage.	An unrelated bug in bookings can kill an SOS
Notification Service	Fan-out across FCM/APNs/SMS/WhatsApp/IVR with per-channel retry, DLT template compliance, quiet hours, and dedup.	Duplicate/ spam notifications; TRAI violations
Offline Sync Service	Server-side half of the offline engine: change feed, vector clocks, conflict resolution, tombstones, batch replay.	Data corruption the moment a user goes offline
AI Gateway	One boundary for model routing, versioning, A/B, confidence thresholds, fallback, cost control, and PII scrubbing. Python stays isolated from the Node core.	Models leak into business logic; no way to roll back a model
Analytics Service	OLAP separated from OLTP. Also the compliance boundary for aggregate/anonymized data shared with government.	Analytics queries take down production
Government Service	Different tenancy, different auth (mTLS + IP allowlist), different data contract (aggregated, anonymized), different audit rules.	Accidental PII exposure to a third party
Payment Service	PCI/RBI scope containment. Isolating it keeps the rest of the platform out of audit scope.	Entire platform enters PCI scope
Audit Service	Append-only, tamper-evident log required for DPDP, insurance disputes, and government audits.	No defensible record when it matters

Phase Detail

Each phase below gives **Objectives · Tasks · Dependencies · Deliverables · Acceptance Criteria**. Per-person breakdowns (folder structure, tables, APIs, screens, models, libraries, risks, security, testing, docs, branches, time, demo) are in the four individual roadmaps.

Phase 0 — Research · S0 W1 · All hands

Objectives — Replace assumptions with evidence before a line of product code is written.

Tasks - Field research: 20 vehicle owners (5 car, 5 two-wheeler, 4 auto, 3 truck, 3 tractor), 10 mechanics, 2 fleet managers, 1 RTO/traffic official. Structured interview guide, recorded, transcribed. - Competitive teardown: Bosch RoadSide, ReadyAssist, Allianz/Europ Assistance India, Tata Capital RSA, OEM apps (MyTVS, Maruti Suzuki), plus Uber/Ola dispatch mechanics as a technical reference. - Regulatory scan: DPDP Act 2023, TRAI DLT/UCC, MoRTH AIS-140, ERSS 112 integration norms, VAHAN/SARATHI API access, RBI PA/PG guidelines, insurance IRDAI motor-claim workflows. - Technical feasibility spikes (timeboxed 1 day each): OBD-II BLE dongle read on a low-end Android; offline vector tiles size for one district; USSD session flow with a gateway sandbox; crash detection from raw accelerometer traces. - Connectivity reality study: measure real 2G/3G latency and packet loss on 3 highway corridors; catalogue device mix from India Android distribution data.

Dependencies — None. This phase gates everything.

Deliverables — Research dossier (findings + verbatim quotes), 6 personas, journey maps for 4 scenarios (highway breakdown, city no-start, night-time accident, fleet truck failure), competitor matrix, regulatory obligation register, 4 spike reports with go/no-go.

Acceptance criteria - Every product assumption in the pitch is either evidenced or explicitly flagged as an open bet. - Each spike ends with a written go/no-go and a named fallback. - Regulatory register lists every obligation with owner and phase where it's satisfied.

Phase 1 — Planning · S0 W2 · CTO + all

Objectives — Convert research into a scoped, estimated, sequenced plan with a frozen v1 boundary.

Tasks — Write the PRD (problem, users, jobs-to-be-done, scope, explicit non-goals). Define success metrics and SLOs. Cut the v1/v2/v3 line. Build the epic → story backlog with acceptance criteria. Estimate in ideal days, per person. Assemble the risk register. Choose the stack and record it as ADRs. Set up the repo, project board, CI skeleton, and coding standards. Draft the business model canvas.

Dependencies — P0.

Deliverables — PRD v1, SLO document, prioritized backlog (~180 stories), estimate sheet, risk register, ADR-000 (stack selection), monorepo scaffold, project board, this Team Charter.

Acceptance criteria - Every S1–S4 story is Ready (has AC, ≤3 ideal days, no unresolved dependency). - Total estimate fits 22 weeks × 32 ideal days/sprint with ≥20% buffer, or scope is cut until it does. - Non-goals are written down explicitly. "Not in v1" is a documented decision, not a silence.

SLO targets (frozen here)

Journey	SLO
App cold start (low-end device)	p95 < 2.5 s

Journey	SLO
Booking create API	p95 < 300 ms
Mechanic match returned	p95 < 4 s
SOS → responder notified	p95 < 10 s
Sync of 100 queued ops	p95 < 8 s on 3G
Core API availability	99.9% (99.99% for Emergency)
Crash-free session rate	≥ 99.5%

Phase 2 — System Architecture · S1 · Owner D1

Objectives — Define service boundaries, contracts, event model, and non-functional strategy so four people can build in parallel without collisions.

Tasks — C4 diagrams (context/container/component). Service decomposition with explicit boundaries and ownership. Event catalogue and schema registry (Avro). Sync/async decision per interaction. API style guide (REST + JSON:API-ish envelope; gRPC internal; WebSocket for live tracking). Idempotency, pagination, versioning, and error-envelope standards. Multi-tenancy model for fleets and government. Caching strategy per layer. Rate-limit tiers. Failure-mode analysis per service with degraded-mode definition. Threat model (STRIDE) with D4.

Dependencies — P1.

Deliverables — 15 ADRs, C4 diagram set, event catalogue, API style guide, OpenAPI v0 skeleton, threat model, degraded-mode matrix.

Acceptance criteria - Every service has one owner, one database schema, and no shared tables. - Every cross-service call is classified sync or async with a documented reason. - Every service has a written degraded-mode behaviour ("what still works if this is down"). - Two engineers can independently implement against the contract without talking.

Phase 3 — Database Design · S1 · Owner D1

Objectives — An enterprise PostgreSQL schema that survives 10M users, regulatory audit, and five years of change.

Tasks — Full ER model. Normalize to 3NF, then document each deliberate denormalization with a reason. PostGIS geometry for locations/geofences. TimescaleDB hypertables for telemetry. Index plan derived from the actual query catalogue. Constraints (FK, CHECK, EXCLUDE for overlapping availability, partial unique). Audit log via triggers → append-only table. Soft delete convention + enforced read views. Optimistic-concurrency `version` columns. Partitioning strategy (booking by month, telemetry by time+vehicle). Migration tooling and expand/contract discipline. Seed and synthetic data generators.

Dependencies — P2.

Deliverables — ER diagram (Mermaid), ~60-table schema, migration set, index plan with EXPLAIN evidence, audit/soft-delete/versioning conventions doc, seed generator producing 100k realistic rows.

Acceptance criteria - Migrations apply and roll back cleanly on an empty DB, twice in a row. - Every table has: PK, `created_at`, `updated_at`, `deleted_at`, `version`, and an owner service. - Every FK has a supporting index. Zero sequential scans on the top-20 query catalogue at 100k rows. - Audit trigger proven to capture INSERT/UPDATE/DELETE with before/after payloads.

Phase 4 — Authentication · S2 · Owner D1 (+D2 clients, +D4 secrets)

Objectives — One identity system spanning smartphones, browsers, feature phones, mechanics, fleets, and government — with real RBAC.

Tasks — Phone-OTP primary flow with rate limits and lockout. JWT access (10 min) + rotating refresh (30 days) with reuse detection. Device binding and multi-device session list. RBAC: roles (citizen, mechanic, fleet_admin, fleet_driver, gov_officer, support, admin) × permissions, resource-scoped. Feature-phone auth via SIM-bound MSISDN + PIN. Government mTLS + IP allowlist + short-lived tokens. Optional DigiLocker/eKYC linkage for mechanic verification. Consent ledger (DPDP): purpose-scoped, versioned, withdrawable. Session revocation, step-up auth for high-risk actions.

Dependencies — P3.

Deliverables — Auth service, RBAC policy engine + policy tests, consent ledger, client SDK (TS) with silent refresh, IVR/SMS auth path, admin session management screen.

Acceptance criteria - OTP brute force blocked (5 attempts / 15 min / MSISDN + IP). - Refresh token reuse detected → entire family revoked → user notified. - 20 RBAC policy tests pass, including 6 negative IDOR cases. - Auth works end-to-end on mobile, web, mechanic app, gov portal (mTLS), and SMS. - Consent withdrawal propagates to all services within 60 s.

Phase 5 — Backend APIs · S2–S3 · Owner D1 (+D3 AI endpoints)

Objectives — Implement the full v1 surface: the platform's actual product logic.

Tasks — Booking state machine (REQUESTED → MATCHING → ASSIGNED → EN_ROUTE → ON_SITE → IN_PROGRESS → COMPLETED, with CANCELLED/FAILED/ESCALATED branches) with guarded transitions and idempotent commands. Dispatch: geospatial candidate query → scoring → offer → accept/timeout/reoffer ladder. Vehicle CRUD + VAHAN lookup + service history. Mechanic onboarding, verification, availability, inventory. Pricing engine (base + distance + parts + surge, with a transparency breakdown). Payments: UPI intent, card, cash, corporate credit, wallet, refunds. Notification fan-out. Fleet APIs (bulk vehicles, drivers, policies, cost centres). Reviews/ratings with fraud hooks. Support/ticketing. Full webhook system for partners.

Dependencies — P4.

Deliverables — 12 services, ~140 endpoints, OpenAPI 3.1 spec, generated TS client, Postman collection, contract tests (Pact), seeded demo scenarios.

Acceptance criteria - 100% of the v1 OpenAPI implemented; spec is generated from code, never hand-written. - All write endpoints idempotent under **Idempotency-Key** ; proven by a replay test. - State machine rejects every illegal transition (exhaustive test over the transition matrix). - Contract tests pass against the frontend's consumer expectations. - p95 < 300 ms for the top 10 endpoints at 1k RPS locally.

Phase 6 — Frontend · S3–S4 · Owner D2

Objectives — Ship the real product surface across 7 clients, designed for low-end devices and bad networks first.

Tasks — Design system (tokens, 40+ components, dark/light, 8 languages incl. Hindi/Tamil/Telugu/Bengali/Marathi, RTL-ready). React Native app: onboarding, garage, request-help flow, live tracking, chat/call, payment, history, profile. Mechanic app: job feed, accept, navigation, checklist, parts, invoice, earnings. Web PWA parity for core flows. Admin console. Fleet dashboard. Android Auto prototype (media/navigation templates). Motion system, skeletons, optimistic UI, error/empty/offline states everywhere. Accessibility pass.

Dependencies — P4 (auth), P5 partial (contracts can be mocked via MSW from day one).

Deliverables — Design system package, RN app (Android+iOS), 4 web apps, Android Auto prototype, Storybook, screenshot/visual regression suite.

Acceptance criteria - Full booking journey works on Android 10 / 2 GB RAM / throttled 3G. - Every screen has designed loading, empty, error, and offline states — verified in review. - WCAG 2.1 AA: contrast, focus order, screen-reader labels, 44 dp targets. - Cold start p95 < 2.5 s on the reference device; initial web route ≤ 180 KB gz. - 8 languages render without truncation or layout break.

Phase 7 — AI Development · S4–S5 · Owner D3

Objectives — Nine AI subsystems that are genuinely useful, measurably better than a baseline, and safe to fail.

Tasks — See [D3's roadmap](#) for full model specs. Summary: symptom→diagnosis triage (retrieval + classifier + LLM explanation), image damage/part diagnosis (CV), voice assistant (ASR for Indian languages + intent NLU + TTS), predictive maintenance (survival + gradient boosting on telemetry), mechanic matching (learning-to-rank), crash detection (on-device sensor model), fraud detection (graph + anomaly), government analytics (spatiotemporal forecasting), offline AI (quantized on-device bundle).

Dependencies — P3 (feature store schema), P5 (data flowing), P10 shares the diagnostics work.

Deliverables — 9 model pipelines, AI Gateway, model registry (MLflow), feature store, 9 model cards, eval harness with held-out sets, on-device bundle ≤ 15 MB.

Acceptance criteria - Every model beats its documented baseline on a held-out set (baselines defined *before* training). - Every model has a confidence threshold and a defined fallback when it isn't met. - Crash detection: recall ≥ 0.95 on labelled crash traces, false-positive rate ≤ 1 per 1,000 driving hours, and it never auto-dispatches without human confirmation. - On-device inference ≤ 120 ms p95 on the reference device. - Model card exists and is reviewed for bias before any model reaches production.

Phase 8 — Offline Engine · S5 · Owner D2 (client) + D1 (server)

Objectives — Make the product work where it's needed most: no signal, on a highway, at night.

Tasks — Local DB (WatermelonDB/SQLite) mirroring the domain. Write-ahead operation queue with priority (SOS > booking > telemetry > analytics). Sync protocol: cursor-based change feed, per-field last-write-wins with server-authoritative state machine override, tombstones, batched compressed deltas. Conflict resolution rules per entity, with a user-visible resolution UI where the machine can't decide. Offline map packs. SMS fallback protocol: compact encoded messages for request/status/cancel. IVR menu tree and USSD session flow. Bluetooth relay (phone-to-phone hop when one device has signal). Background sync scheduling that respects battery and metered data.

Dependencies — P6, P5, P9 (map packs).

Deliverables — Offline client library, sync service, conflict matrix doc, SMS/IVR/USSD flows live on a sandbox gateway, Bluetooth relay prototype, offline test harness (network chaos).

Acceptance criteria - Complete a help request in airplane mode; it syncs correctly on reconnect with no data loss. - Conflict test suite (30 scenarios) passes; no scenario produces a corrupt or ambiguous state. - Feature-phone user can request help, receive mechanic details, and cancel — entirely over SMS. - Sync of 100 queued operations completes p95 < 8 s on simulated 3G. - Queue survives app kill, device reboot, and OS storage pressure.

Phase 9 — Maps · S6 · Owner D2 + D1

Objectives — Maps that work offline, cost nothing per-tile at national scale, and are accurate enough to dispatch on.

Tasks — MapLibre GL rendering (native + web). Self-hosted OSM vector tiles + Valhalla routing/ETA. District-level offline map pack generation, download manager, and delta updates. Geocoding + reverse geocoding with Indian address quirks (landmarks, no house numbers). Geofencing (service zones, toll plazas, accident blackspots, gov jurisdictions). Live location tracking over WebSocket with adaptive sampling and battery-aware throttling. Highway kilometre-marker resolution ("NH-48, KM 212") — critical for rural dispatch. Snap-to-road, trip replay.

Dependencies — P6, P5.

Deliverables — Maps service, tile infrastructure, offline pack builder, map component library, geofence admin UI, tracking channel.

Acceptance criteria - Navigation and search work with the device fully offline for a downloaded district. - ETA within $\pm 20\%$ of ground truth on 20 sampled routes. - Location tracking costs $\leq 4\%$ battery/hour with the screen off. - A user can be located by highway KM marker with no street address available. - Map pack for a typical district ≤ 60 MB.

Phase 10 — Vehicle Diagnostics · S6 · Owner D3 + D2

Objectives — Tell the user what's wrong before the mechanic arrives — from a dongle, a photo, a sound, or a sentence.

Tasks — OBD-II BLE dongle integration (ELM327 profile): DTC read, live PIDs, freeze frames; DTC → plain-language mapping in 8 languages. Photo diagnosis (damage type, severity, part identification). Sound-based diagnosis (engine/brake noise classification) as a v1.5 spike. Symptom triage conversation. Two-wheeler and tractor-specific rule packs (most Indian two-wheelers have no OBD — rule + symptom based). EV-specific diagnostics (SoC, cell balance, thermal, charge faults). Truck telematics (AIS-140/FMS where available). Severity → "can I drive it?" recommendation with an explicit safety-conservative bias. Parts prediction feeding mechanic inventory.

Dependencies — P7, P6.

Deliverables — Diagnostics service, BLE module, DTC knowledge base (~5k codes), diagnosis UI flow, per-vehicle-class rule packs, mechanic pre-brief (mechanic sees the diagnosis before arriving).

Acceptance criteria - OBD read works on ≥ 3 dongle models and ≥ 5 vehicle makes. - Photo diagnosis top-3 accuracy $\geq 85\%$ on the held-out set. - Non-OBD vehicles (bike, auto, tractor) get a useful symptom-based triage — verified by 5 mechanics rating outputs. - Safety recommendation errs conservative: measured false "safe to drive" rate = 0 on the eval set. - Mechanic receives the diagnosis and predicted parts list before departure.

Phase 11 — Emergency Engine · S7 · Owner D1 + D3 + D4

Objectives — The feature that justifies the platform. Must work when everything else is broken.

Tasks — Crash detection pipeline: on-device sensor model → 30 s user-cancellable countdown → confirmed incident. Manual SOS (in-app, hardware button pattern, voice, SMS keyword). Automatic escalation ladder: emergency contacts → nearest responder → ambulance → ERSS 112 handoff (with the state integration protocol). Medical profile (blood group, allergies, conditions) available to responders under break-glass access with full audit. Live incident room: location stream, audio capture consent, responder ETA. Nearest hospital/ambulance/police routing. Insurance FNOL auto-trigger. Degraded mode: SOS over raw SMS works even with the entire application platform down. Drill mode for testing without touching real services.

Dependencies — P7 (crash model), P9 (routing), P5.

Deliverables — Emergency service (independently deployed, own quota, own alerting), SOS UI across all clients, 112 handoff adapter, break-glass audit, degraded SMS path, drill runbook.

Acceptance criteria - Crash → contacts + responders notified, p95 < 10 s. - 30 s cancel window works and is impossible to miss (audio + haptic + full-screen). - Emergency path passes a chaos test with every other service down. - Break-glass access to medical data logs actor, reason, and time; alerts the user afterwards. - No auto-dispatch to public emergency services without either user confirmation or a two-signal confirmation policy documented and approved. - Runs as a monthly drill with a written report.

Phase 12 — Government Dashboard · S7 · Owner D2 + D3

Objectives — Give NHAI/state transport/traffic police a live operational picture, without handing them citizens' personal data.

Tasks — Multi-tenant gov portal (national → state → district → jurisdiction hierarchy). Live incident map with clustering and severity layers. Accident blackspot analysis and heatmaps. Breakdown-density analysis for infrastructure planning. Response-time analytics per corridor. Statutory report generation (6 formats, exportable). Anonymization/aggregation layer with k-anonymity enforcement (no cell below k=10 is ever shown). Alerting for corridor incidents. Read-only API for state integration. Full audit trail of every officer's query.

Dependencies — P11, P13 partial.

Deliverables — Gov portal, anonymization service, 6 report templates, gov API, officer audit log, data-sharing agreement template.

Acceptance criteria - No individual citizen is identifiable in any view or export — verified by a re-identification test. - k-anonymity ≥ 10 enforced at query time, not just in the UI. - Live map handles 10k concurrent incidents without client jank. - Every officer query is audited with purpose code. - Reports match the statutory format samples collected in P0.

Phase 13 — Analytics · S8 · Owner D3 + D1

Objectives — Instrument the business so decisions are made on data, and give fleets/mechanics their own numbers.

Tasks — Event taxonomy and tracking plan (one canonical event schema, no ad-hoc events). CDC pipeline Postgres → Kafka → ClickHouse. Core dashboards: operations (demand, supply, match rate, cancellations), reliability (SLOs), business (revenue, CAC/LTV, cohort retention), safety (incidents, response times). Fleet analytics (cost per km, downtime, driver behaviour). Mechanic analytics (earnings, utilization, ratings, acceptance). Funnel and cohort analysis. Anomaly alerting on business metrics. Self-serve query layer for the team.

Dependencies — P5, P12.

Deliverables — Event taxonomy doc, CDC pipeline, ClickHouse schema, 4 dashboard suites, fleet + mechanic analytics screens, metric definitions dictionary.

Acceptance criteria - Every metric has exactly one definition in the dictionary; no two dashboards disagree. - Pipeline lag p95 < 60 s from event to dashboard. - Dashboards load < 2 s at 100M events. - Anomaly alerts fire on injected synthetic anomalies during testing.

Phase 14 — Testing · S8 · Owner D4, all contribute

Objectives — Prove the thing works, including on the paths nobody wants to think about.

Tasks — Unit tests to 80% branch coverage on new code. Integration tests with testcontainers (real Postgres/Redis/Kafka). API contract tests (Pact) both directions. E2E: Detox (mobile), Playwright (web) across the 12 critical journeys. Security testing: OWASP ASVS L2 checklist, ZAP DAST, Semgrep/CodeQL SAST, dependency + container scanning, secrets scanning, plus an authenticated pentest pass. Load testing (k6): 10k RPS steady, 50k spike, 100k concurrent WebSocket tracking sessions. Soak test 24 h. Chaos: kill pods, partition network, saturate DB connections, expire certs. Offline test matrix (airplane mode, flaky 2G, mid-sync kill, clock skew, storage full). Device farm across 12 real Android devices + 4 iOS. Accessibility audit. Localization QA across 8 languages. Emergency-path drill.

Dependencies — P5–P13.

Deliverables — Full test suites in CI, test strategy doc, load test report, security test report, chaos experiment log, device compatibility matrix, bug triage and burn-down to zero P0/P1.

Acceptance criteria - All quality gates in Charter §11 green. - Zero open P0/P1 bugs at sprint exit. - Load: p95 < 300 ms at 10k RPS; graceful degradation (not collapse) at 50k spike. - Chaos: every experiment ends with the documented degraded behaviour, not an outage. - Emergency drill passes with the platform artificially degraded.

Phase 15 — Deployment · S9 · Owner D4

Objectives — Production infrastructure that a 4-person team can actually operate.

Tasks — Terraform for all infra (India regions only). Kubernetes with per-service HPA, PDBs, resource limits, and node pools sized by workload class. GitOps with ArgoCD. Blue/green for stateless, expand-contract for schema. Progressive delivery: canary 5% → 25% → 100% with automated rollback on SLO breach. Secrets in Vault/KMS with rotation. Multi-AZ; documented multi-region DR. Backups: PITR for Postgres, tested restores, cross-region encrypted copies. Observability: Prometheus + Grafana + Loki + Tempo + Sentry, RED/USE dashboards per service, SLO burn-rate alerts. On-call rotation with runbooks. Cost monitoring and budget alerts. Store submission (Play/App Store) with staged rollout.

Dependencies — P14.

Deliverables — Terraform modules, K8s manifests/Helm charts, ArgoCD apps, CI/CD pipelines, observability stack, 25+ runbooks, DR plan, on-call schedule, published apps.

Acceptance criteria - A commit to `main` reaches production through the full pipeline with zero manual steps beyond one approval. - Rollback of any service completes < 5 min, proven by drill. - DR drill: restore from backup into a clean region, RTO < 4 h, RPO < 5 min — proven, not asserted. - Every alert has a runbook; a new on-call can resolve a simulated incident using only the runbook. - All data at rest and in transit encrypted; no plaintext secret anywhere in the pipeline.

Phase 16 — Optimization · S9 · Owner D4, all contribute

Objectives — Make it fast, small, and cheap enough to serve millions on cheap phones and cheap data.

Tasks — Backend: query tuning from `pg_stat_statements`, index refinement, connection pooling (PgBouncer), targeted caching, payload trimming, response compression. Mobile: APK size (Proguard/R8, resource shrinking, dynamic feature modules), startup profiling, memory leak hunt, jank elimination at 60 fps, battery profiling, data-usage reduction (protobuf on hot paths, delta sync, image transcoding to AVIF/WebP). Web: route-level code splitting, edge caching, image optimization, critical CSS. AI: quantization, distillation, batching, model caching. Infra: right-sizing, spot instances for batch, autoscaling tuning, cost-per-active-user model. Database: partition pruning, vacuum tuning, read replicas.

Dependencies — P15.

Deliverables — Performance report (before/after), optimization ADRs, cost model per 100k MAU, performance budgets wired into CI.

Acceptance criteria - p95 API < 300 ms; p99 < 800 ms at production load. - APK ≤ 28 MB; cold start p95 ≤ 2.0 s on the reference device (improved from 2.5 s). - Monthly data usage for a typical user ≤ 30 MB. - Infra cost per 100k MAU modelled and within target; regressions alert in CI. - Performance budgets fail the build when exceeded — no silent regression.

Phase 17 — Future Roadmap · S10 · CTO + all

Objectives — Define what's next and package the story for pilots, partners, and investors.

Tasks — 18-month product roadmap (Now/Next/Later). Technical debt register with a repayment plan. Scaling plan to 10M users with the specific bottlenecks named. Expansion: EV charging assistance network, insurance-embedded RSA, OEM white-label, ADAS/dashcam integration, V2X readiness, drone-delivered parts (research), predictive road-condition mapping. Business: revenue model, subscription tiers, government licensing, insurance and fleet partnerships, mechanic network economics, unit economics, BMC, market sizing, competitor analysis, SWOT. Pitch deck. Pilot MoU draft for one city + one highway corridor.

Dependencies — P16.

Deliverables — 18-month roadmap, tech debt register, scaling plan, business model canvas, pitch deck (12 slides), market/competitor/SWOT analysis, pilot proposal.

Acceptance criteria - Roadmap items are sequenced with dependencies and rough sizing, not a wish list. - Unit economics show a path to contribution-margin-positive with stated assumptions. - Pitch deck survives a hostile Q&A dry run with the team playing judges. - Pilot proposal is concrete enough to hand to a government or fleet counterpart.

Business Model Summary (detail in P17 deliverables)

Stream	Who pays	Model	Notes
B2C subscription	Vehicle owners	₹299/₹699/₹1,499 per year (Basic/Plus/Pro)	Unlimited assistance calls at Pro; per-use pricing below
Pay-per-use	Non-subscribers	₹399–₹1,999 per incident + parts	Entry point that converts to subscription
B2B fleet	Fleet operators	₹99–₹249/vehicle/month	Volume tiers, SLA-backed, analytics included
Government licensing	State transport / NHAI	Annual platform licence + per-district	Dashboard, analytics, corridor monitoring
Insurance partnership	Insurers	Per-policy bundling + FNOL data	RSA as a policy rider; crash data reduces claim fraud
OEM white-label	Vehicle manufacturers	Licence + revenue share	Branded RSA inside the OEM's own app
Mechanic network	Mechanics	12–18% take rate on jobs	Plus paid tooling: inventory, invoicing, credit
Data products	Insurers, planners	Aggregated, anonymized road-risk data	Strictly k-anonymized; never personal data

D1 - Backend & Data Lead

Individual Roadmap

| *One Platform. Every Vehicle. Every Phone. Every Road.*

Document docs/02-backend-lead-roadmap.md

Version 1.0 — planning baseline

Team 4 engineers · 22 weeks · 11 sprints · 18 phases

Status Baseline (frozen at Sprint 0 exit)

D1 — Backend & Data Lead · Individual Roadmap

Owens: all backend services, PostgreSQL/PostGIS schema, event bus, API contracts, auth, booking/dispatch, emergency core, payments. **Backs up:** D3 on the AI gateway and analytics service. **Total allocation:** 22 weeks · ~176 ideal days.

Standing responsibilities (every sprint, not phase-specific): - The OpenAPI spec is the single source of truth. It is generated from code and published on every merge to `main`. - No other engineer is ever blocked waiting on a contract from you — mock-first, spec before implementation. - Every migration you write is reviewed by a second person and has a tested rollback. - You are the final word on data modelling. Push back on schema shortcuts, including your own.

Phase 0 — Research · S0 W1 · 4 days

Goal	Establish whether the intended backend architecture survives contact with Indian mobility reality — connectivity, regulation, and integration partners.
Deliverables	Integration feasibility report (VAHAN/SARATHI, FASTag, ERSS 112, UPI/PSP); data-model research notes from 10 mechanic + 2 fleet interviews; event-model draft; ADR-000 stack proposal.
Expected output	A written go/no-go per external integration, with fallback if the API is unavailable or the approval timeline exceeds the project.
Folder structure	<code>docs/research/backend/</code> <code>docs/adr/</code>
Database tables	None yet — but produce a <i>candidate entity list</i> (~55 entities) with cardinalities from interview evidence.
APIs	None. Draft the API style guide instead.
UI screens	None.
Components	None.
AI models	None — but define with D3 which backend events the models will need as features.
Libraries	Evaluation only: NestJS vs Fastify-standalone, Prisma vs Drizzle vs TypeORM, Kafka vs Redpanda vs NATS, BullMQ.
Risks	VAHAN API access requires a government MoU that may take months → design a manual-entry + OCR fallback for RC books from day one.
Security	Map every external integration to its data-sharing obligation; flag anything that would push us into PCI or RBI PA/PG scope.
Testing	N/A — but write the <i>testability</i> requirements into the style guide (every service must run fully offline in tests via testcontainers).
Documentation	Integration feasibility report; candidate ER sketch; API style guide v0.
Git branches	<code>docs/p00-d1-integration-research</code> , <code>docs/p00-d1-adr-000-stack</code>
Time	4 ideal days
Demo	10-min walkthrough: "here is what we can integrate with, here is what we must fake, and here is the fallback for each."

Phase 1 — Planning · S0 W2 · 4 days

Goal	Convert the backend surface into an estimated, sequenced backlog with a frozen v1 contract boundary.
Deliverables	Backend epic breakdown (~70 stories with AC), estimates, service inventory with owners, repo scaffold + CI skeleton for backend packages.
Expected output	Every backend story for S1–S4 is Ready; D2 and D3 can see exactly which contracts land in which sprint.
Folder structure	<code>apps/</code> <code>libs/</code> <code>tools/</code> Nx monorepo scaffold; <code>libs/shared-contracts/</code> created and wired into CI.

Database tables	Entity list frozen; naming conventions decided (snake_case, singular table names, <code>_id</code> FKs, <code>uuid v7</code> PKs).
APIs	Publish the <i>contract delivery calendar</i> — which endpoints exist as mocks in which sprint.
UI screens	None.
Components	Shared libs: <code>logger</code> , <code>errors</code> , <code>validation</code> , <code>idempotency</code> , <code>pagination</code> , <code>auth-guard</code> , <code>event-bus</code> .
AI models	Agree the AI Gateway contract shape with D3 (request/response envelope, confidence, model version, fallback flag).
Libraries	Locked: NestJS 11, Fastify adapter, Drizzle ORM, Zod, Redpanda + <code>kafkajs</code> , BullMQ, <code>ioredis</code> , <code>@nestjs/terminus</code> , Pino, OpenTelemetry SDK.
Risks	Over-decomposition into microservices with a 4-person team → decision: modular monolith deployed as 5 process groups , with clean module boundaries that can be split later. Record as ADR-002.
Security	Threat model kickoff with D4; decide the trust boundaries that shape the service split.
Testing	Test pyramid targets per layer; testcontainers harness scaffolded.
Documentation	ADR-001 (modular monolith), ADR-002 (Drizzle), ADR-003 (Redpanda), ADR-004 (error envelope), contract calendar.
Git branches	<code>chore/p01-d1-monorepo-scaffold</code> , <code>docs/p01-d1-adrs</code>
Time	4 ideal days
Demo	<code>npm nx run-many -t test</code> passes on an empty scaffold in CI; contract calendar walked through with D2/D3.

Phase 2 — System Architecture · S1 W3 · 5 days · You own this phase

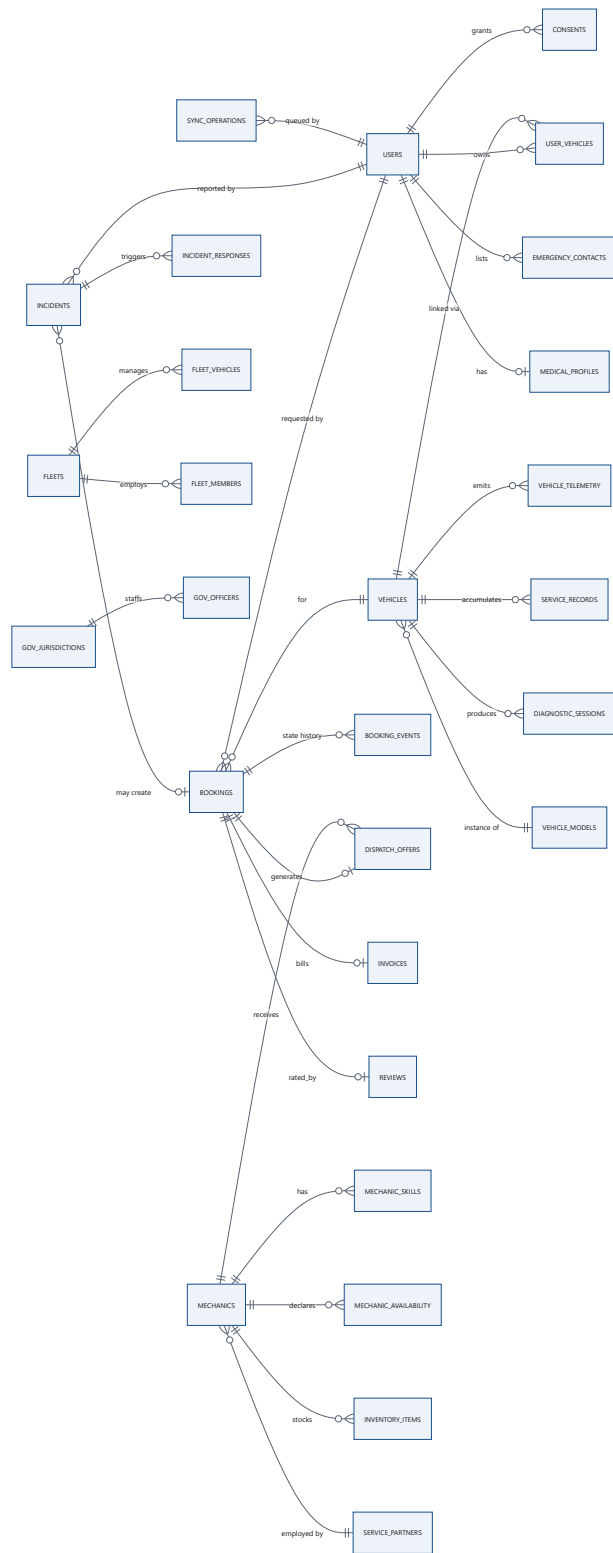
Goal	Define boundaries, contracts, and failure semantics precisely enough that four people build in parallel for 20 weeks without collisions.
Deliverables	C4 diagrams (context/container/component), 15 ADRs, event catalogue with Avro schemas, API style guide v1, degraded-mode matrix, OpenAPI v0 skeleton generating a TS client.
Expected output	D2 codes against a generated client on day 1 of S2. D3 codes against a fixed AI Gateway envelope. Nobody waits.
Folder structure	<code>apps/{gateway,auth,core-api,dispatch,emergency,worker}/</code> <code>libs/{contracts,domain,db,events,auth,observability,testing}/</code> <code>infra/</code> <code>docs/{adr,api,runbooks}/</code>
Database tables	Schema-per-module decided (<code>auth.*</code> , <code>user.*</code> , <code>vehicle.*</code> , <code>booking.*</code> , <code>dispatch.*</code> , <code>mech.*</code> , <code>emg.*</code> , <code>pay.*</code> , <code>audit.*</code>). No cross-schema FKs except to <code>user.users</code> .
APIs	Style guide: REST + envelope <code>{data, meta, error}</code> ; cursor pagination; <code>Idempotency-Key</code> on all writes; <code>If-Match</code> for optimistic concurrency; RFC 7807 problem details; <code>/v1</code> URI versioning; gRPC internal; WebSocket for tracking.
UI screens	None.
Components	<code>libs/contracts</code> (Zod schemas → OpenAPI → TS client), <code>libs/events</code> (typed publisher/consumer, schema registry client), <code>libs/observability</code> (trace/log/metric conventions).
AI models	AI Gateway contract finalized: <code>POST /ai/v1/{capability}</code> with <code>{input, context, requestId}</code> → <code>{result, confidence, modelVersion, fallbackUsed, latencyMs}</code> .
Libraries	<code>@asynccapi/generator</code> , <code>avsc</code> , <code>zod-to-openapi</code> , <code>nestjs-otel</code> , <code>@grpc/grpc-js</code> .
Risks	Contract churn later breaks the frontend → mitigation: contracts are versioned and additive-only after S3; breaking changes require a new version path and a deprecation window.
Security	STRIDE threat model with D4 across all trust boundaries; output is a control list mapped to phases. Decide: mTLS between process groups, gateway-only ingress, no service reachable from the internet directly.
Testing	Contract testing strategy (Pact) agreed with D2. Architecture fitness functions defined (dependency-cruiser rules that fail CI on illegal module imports).
Documentation	15 ADRs, C4 set, event catalogue, style guide, degraded-mode matrix.

Git branches	<code>docs/p02-d1-c4-architecture</code> , <code>feat/p02-d1-contracts-lib</code> , <code>feat/p02-d1-event-bus-lib</code>
Time	5 ideal days
Demo	Walk the C4 diagrams; show a Zod schema producing OpenAPI producing a typed TS client that D2 imports; show CI failing on an illegal cross-module import.

Phase 3 — Database Design · S1 W4 · 6 days · You own this phase

Goal	A schema that survives 10M users, five years of change, and a regulatory audit.
Deliverables	Full ER model, ~60 tables, migration set, index plan with EXPLAIN evidence, audit/soft-delete/versioning conventions, seed generator (100k rows).
Expected output	<code>pnpm db:migrate && pnpm db:seed</code> produces a realistic database in under 2 minutes.
Folder structure	<code>libs/db/{schema,migrations,seeds,queries}/</code> , <code>docs/db/er.md</code>
Database tables	See the table catalogue below.
APIs	None — but publish the query catalogue : the top 40 queries the API layer will run, used to derive indexes.
UI screens	None.
Components	<code>libs/db</code> repository layer, transaction manager, soft-delete read views, audit trigger generator, migration lint.
AI models	Feature-store table design with D3: <code>ml.feature_vehicle_daily</code> , <code>ml.feature_user_behaviour</code> , <code>ml.training_snapshot</code> .
Libraries	<code>drizzle-orm</code> , <code>drizzle-kit</code> , <code>postgis</code> , <code>timescaledb</code> , <code>pgcrypto</code> , <code>pg_stat_statements</code> , <code>@faker-js/faker</code> , <code>pgbouncer</code> .
Risks	(a) PostGIS query performance on national-scale geography → GiST + KNN operators, benchmarked at 1M mechanic rows. (b) Telemetry volume → TimescaleDB hypertable + compression + retention policy, decided now not later.
Security	Column-level encryption (<code>pgcrypto</code>) for Aadhaar-adjacent IDs, medical profile, and payment tokens. Row-level security for fleet and government tenancy. Separate DB roles per module with least privilege — the app never connects as owner.
Testing	Migration up/down idempotency test in CI. Constraint tests (attempt every illegal insert, assert rejection). Index effectiveness test asserting zero seq-scans on the query catalogue at 100k rows.
Documentation	<code>docs/db/er.md</code> (Mermaid), normalization notes with justified denormalizations, index plan, partitioning strategy, migration playbook (expand→migrate→contract).
Git branches	<code>feat/p03-d1-schema-core</code> , <code>feat/p03-d1-audit-softdelete</code> , <code>feat/p03-d1-seed-generator</code>
Time	6 ideal days
Demo	Live: run migrations from empty → seed 100k rows → run 5 geospatial dispatch queries showing sub-50 ms with index, sub-second without, and show the audit table capturing an update.

Table catalogue (abridged)



Module	Tables
auth	otp_challenges , sessions , refresh_tokens , devices , roles , permissions , role_permissions , user_roles
user	users , user_profiles , consents , consent_purposes , emergency_contacts , medical_profiles , addresses , preferences , notification_channels
vehicle	vehicle_models , vehicles , user_vehicles , vehicle_documents , service_records , vehicle_telemetry (hypertable), dtc_codes , diagnostic_sessions , diagnostic_findings
booking	bookings (partitioned monthly), booking_events , booking_items , service_types , pricing_rules , quotes , cancellations
dispatch	dispatch_requests , dispatch_offers , dispatch_scores , service_zones (PostGIS), mechanic_locations (PostGIS, Redis-backed hot path)
mech	service_partners , mechanics , mechanic_skills , mechanic_verification , mechanic_availability , inventory_items , payouts , reviews
emg	incidents , incident_signals , incident_responses , responder_units , escalation_policies , break_glass_access
pay	invoices , payments , refunds , payment_methods (tokenized), subscriptions , subscription_plans , wallet_ledger
fleet	fleets , fleet_vehicles , fleet_members , cost_centres , fleet_policies
gov	gov_jurisdictions , gov_officers , gov_queries (audit), gov_reports
sync	sync_operations , sync Cursors , tombstones , conflict_log
audit	audit_log (append-only, partitioned), data_access_log
ml	feature_vehicle_daily , feature_user_behaviour , model_predictions , training_snapshots

Universal columns on every table: `id` uuid PK default `uuidv7()` , `created_at` timestamptz , `updated_at` timestamptz , `deleted_at` timestamptz NULL , `version` int (optimistic lock), `created_by` uuid NULL .

Phase 4 — Authentication · S2 W5 · 6 days · You own this phase

Goal	One identity system across smartphones, browsers, feature phones, mechanics, fleets, and government — with resource-level RBAC.
Deliverables	Auth service, RBAC policy engine, consent ledger, TS auth SDK with silent refresh, SMS/IVR auth path, session management API.
Expected output	A user logs in by OTP on a smartphone, a mechanic on the mechanic app, an officer via mTLS, and a feature-phone user by MSISDN+PIN — all producing correctly-scoped tokens.
Folder structure	apps/auth/src/{otp,token,rbac,consent,device,session}/ , libs/auth/ (guards, decorators, policy DSL)
Database tables	otp_challenges , sessions , refresh_tokens , devices , roles , permissions , role_permissions , user_roles , consents , consent_purposes
APIs	POST /v1/auth/otp/request · POST /v1/auth/otp/verify · POST /v1/auth/refresh · POST /v1/auth/logout · GET /v1/auth/sessions · DELETE /v1/auth/sessions/:id · POST /v1/auth/step-up · GET/POST/DELETE /v1/consents · POST /v1/auth/gov/token (mTLS) · POST /v1/auth/sms/verify (internal, from telecom gateway)
UI screens	Contracts for D2's: OTP entry, session list, consent centre. You build none.
Components	@Roles() / @Permissions() decorators, ResourceGuard (resolves ownership, prevents IDOR), policy DSL, token rotation service, reuse-detection family revocation.
AI models	None. Emit auth events for D3's fraud model: auth.otp_failed , auth.device_new , auth.refresh_reuse_detected .
Libraries	jose (JWT, EdDSA), argon2 , otplib , @casl/ability or custom policy DSL, ioredis (OTP + rate limit), zod .

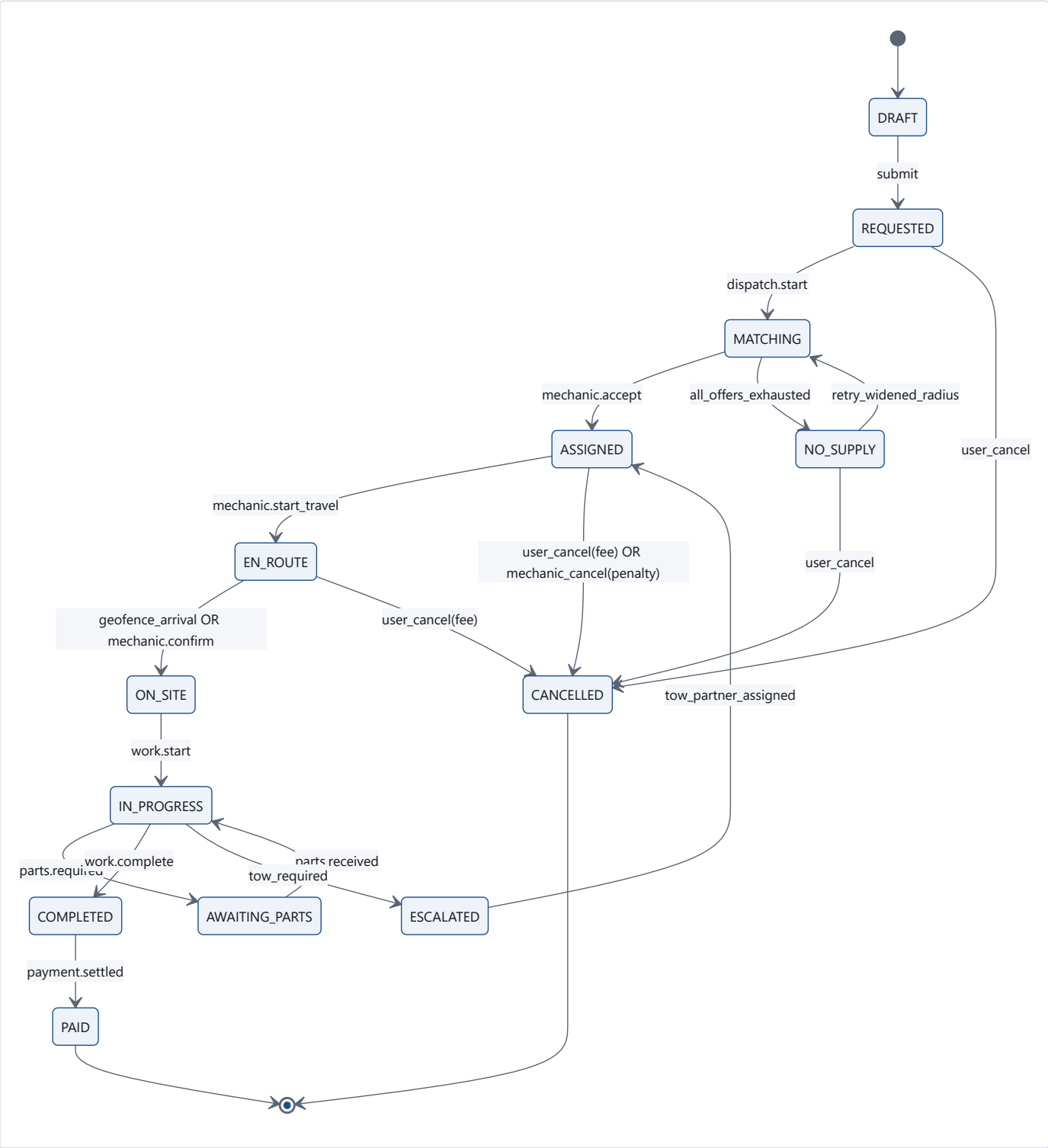
Risks	(a) OTP delivery failure/cost → fallback ladder SMS → WhatsApp → voice call, and never block on the SMS vendor. (b) SIM swap fraud → device binding + step-up auth for payment/emergency-contact changes + cooling period on MSISDN change.
Security	Access token 10 min, EdDSA-signed, no PII in claims. Refresh token 30 d, rotated on every use, hashed at rest, reuse ⇒ revoke the whole family + notify. Rate limits: 5 OTP/15 min/MSISDN, 20/hr/IP, exponential backoff. Constant-time OTP comparison. Consent is purpose-scoped, versioned, and withdrawable (DPDP §6). Government tokens are 15 min, mTLS-bound, IP-allowlisted.
Testing	20 RBAC policy tests including 6 negative IDOR cases; OTP brute-force test; refresh reuse-detection test; token expiry/clock-skew test; consent withdrawal propagation test (<60 s).
Documentation	Auth flow sequence diagrams (5 flows), RBAC matrix (7 roles × ~45 permissions), token lifecycle doc, ADR-016 (EdDSA + rotating refresh).
Git branches	<code>feat/p04-d1-otp-flow</code> , <code>feat/p04-d1-token-rotation</code> , <code>feat/p04-d1-rbac-engine</code> , <code>feat/p04-d1-consent-ledger</code>
Time	6 ideal days
Demo	Live: OTP login on the app; show refresh rotation in the network tab; steal a refresh token and replay it — watch the family get revoked and the user get a notification; attempt an IDOR against another user's vehicle and get a 403; withdraw a consent and watch a downstream service stop processing.

Phase 5 — Backend APIs · S2 W6 – S3 W8 · 15 days · You own this phase

Goal	Implement the entire v1 product surface. This is the largest single block of work in the project.
Deliverables	12 service modules, ~140 endpoints, generated OpenAPI 3.1, typed TS client, Postman collection, Pact contract tests, seeded demo scenarios.
Expected output	A complete booking journey runs end-to-end via API calls alone, with money, notifications, and events flowing.
Folder structure	<code>apps/core-api/src/modules/{user,vehicle,booking,mechanic,pricing,payment,notification,fleet,review,support,webhook}/</code> · <code>apps/dispatch/src/</code> · <code>apps/worker/src/jobs/</code>
Database tables	All from P3 now exercised; add <code>webhooks</code> , <code>webhook_deliveries</code> , <code>support_tickets</code> , <code>idempotency_keys</code> .
APIs	Grouped below.
UI screens	None — but you maintain the MSW mock handlers D2 develops against, generated from the same contracts.
Components	Booking state machine (XState-modelled, guarded transitions), dispatch scoring pipeline, pricing engine with a transparent breakdown, outbox pattern for reliable event publishing, saga for booking→payment consistency, webhook delivery with exponential backoff + DLQ.
AI models	Integrate D3's endpoints: mechanic ranking (<code>/ai/v1/match</code>), diagnosis triage (<code>/ai/v1/diagnose</code>), fraud scoring (<code>/ai/v1/fraud</code>). Every one has a deterministic fallback — dispatch falls back to distance+rating sort, diagnosis to a rules engine, fraud to threshold rules. AI is never on the critical path for correctness.
Libraries	<code>@nestjs/*</code> , <code>xstate</code> , <code>drizzle-orm</code> , <code>bullmq</code> , <code>kafkajs</code> , <code>nestjs-zod</code> , <code>@golevelup/nestjs-rabbitmq</code> (n/a), <code>decimal.js</code> (money — never floats), <code>date-fns-tz</code> , <code>p-retry</code> .
Risks	(a) Scope: 140 endpoints in 15 days is tight → strict prioritization, feature-flag the non-critical 30%, and cut ruthlessly rather than shipping half-tested. (b) Distributed consistency between booking and payment → outbox + saga, with an explicit reconciliation job, not distributed transactions.
Security	Every endpoint has an explicit authorization policy — no route defaults to permitted. Input validated by Zod at the boundary. Output DTOs are allowlists, never entity spread. Money handled in integer paise with <code>decimal.js</code> , never floats. Payment card data never touches our servers (PSP tokenization only). Rate-limit tiers per endpoint class. Idempotency enforced on all writes.
Testing	Unit ≥80% branch. Integration with testcontainers (real Postgres/Redis/Redpanda). Exhaustive state-machine transition matrix test. Idempotency replay test on every write endpoint. Pact contract tests against D2's consumer expectations. Load smoke: 1k RPS on the top 10 endpoints.
Documentation	Generated OpenAPI + rendered reference site, booking state machine diagram, pricing rules doc, event catalogue updated, webhook partner guide.

Git branches	feat/p05-d1-booking-state-machine , feat/p05-d1-dispatch-engine , feat/p05-d1-vehicle-service , feat/p05-d1-mechanic-lifecycle , feat/p05-d1-pricing-engine , feat/p05-d1-payments-upi , feat/p05-d1-notification-fanout , feat/p05-d1-fleet-apis , feat/p05-d1-webhooks
Time	15 ideal days
Demo	Scripted end-to-end via API: user requests help → dispatch offers to 3 mechanics → one accepts → status transitions to completion → invoice generated → UPI payment → review submitted → webhook fires to a partner endpoint. Then break it: cancel mid-flight, timeout a mechanic, fail a payment — show every path handled.

Booking state machine



Endpoint groups

Group	Count	Key endpoints
Users & profiles	12	<code>GET/PATCH /v1/me</code> , <code>/v1/me/emergency-contacts</code> , <code>/v1/me/medical-profile</code> , <code>/v1/me/preferences</code>
Vehicles	16	<code>POST /v1/vehicles</code> , <code>GET /v1/vehicles/:id/health</code> , <code>POST /v1/vehicles/lookup-rc</code> , <code>/v1/vehicles/:id/service-records</code> , <code>/v1/vehicles/:id/telemetry</code>
Bookings	22	<code>POST /v1/bookings</code> , <code>POST /v1/bookings/:id/cancel</code> , <code>GET /v1/bookings/:id/track</code> , <code>POST /v1/bookings/:id/rate</code> , <code>/v1/bookings/:id/events</code>
Dispatch	14	<code>POST /internal/dispatch/start</code> , <code>POST /v1/offers/:id/accept</code> , <code>POST /v1/offers/:id/decline</code> , <code>GET /v1/mechanic/feed</code> (WS)
Mechanics	20	onboarding, verification, availability, skills, inventory, earnings, payouts
Pricing & quotes	8	<code>POST /v1/quotes</code> , <code>GET /v1/pricing/breakdown/:bookingId</code>
Payments	16	<code>POST /v1/payments/intent</code> , UPI collect, webhook receiver, <code>/v1/refunds</code> , <code>/v1/subscriptions</code>
Notifications	6	preferences, device registration, template send (internal)
Fleet	14	bulk vehicle import, drivers, policies, cost centres, fleet bookings, fleet reports
Reviews & support	8	reviews CRUD, ticket create/reply/close
Webhooks	6	partner registration, secret rotation, delivery log, replay
Sync (P8)	4	<code>GET /v1/sync/changes</code> , <code>POST /v1/sync/operations</code> , <code>GET /v1/sync/cursor</code> , <code>POST /v1/sync/resolve</code>

Phase 6 — Frontend · S3 W7 – S4 W10 · 3 days (support role)

Goal	Unblock D2 completely. Your job this phase is to be a zero-latency dependency.
Deliverables	Generated TS client kept current, MSW mock handlers, realistic demo seed data, BFF aggregation endpoints where D2 would otherwise make 5 round-trips on 2G.
Expected output	D2 never waits more than 4 hours for a contract fix.
Folder structure	<code>libs/api-client/</code> (generated, committed), <code>libs/mocks/</code> (MSW handlers generated from contracts)
Database tables	None new; tune indexes for the queries the real UI actually generates (often different from the ones you predicted).
APIs	Add BFF composites: <code>GET /v1/home</code> (one call for the entire home screen), <code>GET /v1/bookings/:id/full</code> (booking + mechanic + vehicle + invoice + tracking in one payload). This matters enormously on 2G.
UI screens	None.
Components	Response shaping for low bandwidth: sparse fieldsets (<code>?fields=</code>), ETags, <code>304</code> support, gzip/brotli.
AI models	None.
Libraries	<code>openapi-typescript</code> , <code>msw</code> , <code>@faker-js/faker</code> .
Risks	Contract churn breaking D2 mid-sprint → all contract changes after S3 must be additive; breaking changes need a deprecation window and a migration note in the PR.
Security	Verify no BFF composite over-fetches: the aggregate must respect the same field-level authorization as the individual endpoints. Easy place to leak data — test it explicitly.
Testing	Pact provider verification runs in your CI on every merge.
Documentation	Client SDK usage guide, mock data catalogue.

Git branches	feat/p06-d1-bff-composites , chore/p06-d1-client-regen
Time	3 ideal days
Demo	Show the home screen loading in one request instead of six, and the payload size difference on a throttled connection.

Phase 7 — AI Development · S4 W9 – S5 W12 · 3 days (support role)

Goal	Give D3 clean data and a safe integration surface.
Deliverables	Feature pipeline from OLTP → feature store, AI Gateway integration with circuit breakers, model prediction logging.
Expected output	Every AI call is observable, timeout-bounded, circuit-broken, and has a working fallback that is exercised in tests.
Folder structure	apps/worker/src/jobs/feature-*, libs/ai-client/
Database tables	ml.feature_vehicle_daily , ml.feature_user_behaviour , ml.model_predictions (every prediction logged with model version, input hash, confidence, and whether fallback was used).
APIs	libs/ai-client wrapping the AI Gateway with: 800 ms timeout, circuit breaker (5 failures / 30 s → open), fallback invocation, and prediction logging.
UI screens	None.
Components	Circuit breaker, fallback registry (each capability registers its deterministic fallback), prediction audit logger.
AI models	None owned — you own the plumbing and the fallbacks.
Libraries	opossum (circuit breaker), p-timeout .
Risks	AI latency degrading the core UX → hard timeout budget: 800 ms for dispatch ranking, 3 s for diagnosis, 200 ms for on-device paths. Exceed it and the fallback runs, silently and correctly.
Security	PII scrubbing before anything leaves for a model. Prediction logs store an input hash, not the input. Model outputs are validated against a schema before use — never trust a model's output shape.
Testing	Chaos test: AI Gateway returns 500 / times out / returns garbage — assert the fallback produces a correct, complete result every time.
Documentation	Fallback registry doc: for each AI capability, exactly what happens when it is unavailable.
Git branches	feat/p07-d1-ai-client-circuit-breaker , feat/p07-d1-feature-pipeline
Time	3 ideal days
Demo	Kill the AI service entirely, then run a full booking. Everything still works, using fallbacks. Show the logs proving fallbacks fired.

Phase 8 — Offline Engine (server side) · S5 W11–12 · 6 days

Goal	The server half of offline-first: a correct, resumable, conflict-resolving sync protocol.
Deliverables	Sync service, change feed, conflict resolution engine, tombstone lifecycle, batch operation replay, SMS command protocol handler.
Expected output	100 queued operations from an offline device replay correctly, in order, idempotently, with conflicts resolved by documented rules.
Folder structure	apps/core-api/src/modules/sync/{feed,replay,conflict,tombstone}/ , apps/telecom-gateway/
Database tables	sync_operations , sync_cursors , tombstones , conflict_log ; add updated_by_device , client_updated_at to synced tables.

APIs	<code>GET /v1/sync/changes?cursor=&limit=</code> (cursor-based change feed) · <code>POST /v1/sync/operations</code> (batch, idempotent, ordered) · <code>POST /v1/sync/resolve</code> (user's conflict decision) · <code>GET /v1/sync/manifest</code> (what the client should hold locally)
UI screens	None — contract for D2's sync status and conflict resolution UI.
Components	Change feed built on the transactional outbox (not polling). Per-field last-write-wins using <code>client_updated_at</code> with server clock-skew correction. Server-authoritative override : booking state transitions are always decided server-side regardless of client claims — a client can never force an illegal state. Tombstones with a 90-day retention.
AI models	None.
Libraries	<code>@msgpack/msgpack</code> (compact wire format), <code>zlib</code> brotli, <code>hlc</code> (hybrid logical clocks).
Risks	This is R1, the project's top technical risk. Mitigations: (a) the conflict matrix is written and reviewed <i>before</i> implementation; (b) 30 named conflict scenarios become an executable test suite; (c) any unresolvable conflict is surfaced to the user rather than silently guessed; (d) <code>conflict_log</code> records every resolution for post-hoc audit.
Security	Replayed operations are re-authorized server-side — an offline device's claim of identity or permission is never trusted. Operation batches are signed with the device key. Replay protection via operation IDs with a dedup window.
Testing	30-scenario conflict suite. Clock-skew tests (± 24 h). Out-of-order delivery test. Duplicate batch test. Partial-failure test (op 47 of 100 fails — the other 99 must still apply, and 47 must be reported). Kill-mid-sync test.
Documentation	Conflict resolution matrix (per entity, per field: LWW / server-wins / client-wins / user-decides), sync protocol spec, SMS command grammar.
Git branches	<code>feat/p08-d1-sync-change-feed</code> , <code>feat/p08-d1-conflict-resolution</code> , <code>feat/p08-d1-operation-replay</code> , <code>feat/p08-d1-sms-command-handler</code>
Time	6 ideal days
Demo	Two devices, both offline, both edit the same booking and the same vehicle profile. Reconnect both. Show the conflict matrix applied: field-level merge where safe, server-authoritative on booking state, user prompt where genuinely ambiguous. Then show a feature phone doing the whole thing over SMS.

Conflict resolution matrix (extract)

Entity	Field class	Rule	Rationale
<code>booking</code>	<code>status</code>	Server wins, always	State machine integrity is non-negotiable
<code>booking</code>	<code>notes</code> , <code>photos</code>	Merge (union)	Additive, no reason to lose either
<code>booking</code>	<code>scheduled_at</code>	Last-write-wins by HLC	Simple, and the user sees the result
<code>vehicle</code>	profile fields	Per-field LWW	Independent fields shouldn't clobber each other
<code>vehicle</code>	<code>odometer</code>	Max wins	Odometers only go up; max is always correct
<code>user</code>	<code>emergency_contacts</code>	User decides	Too important to guess
<code>incident</code>	anything	Server wins	Life-safety; the server has the fullest picture
<code>sync_operation</code>	—	Idempotent by op ID	Replay safety

Phase 9 — Maps · S6 W13 · 3 days (support role)

Goal	Server-side geospatial: zones, geofences, tracking ingestion, KM-marker resolution.
Deliverables	Geofence engine, live tracking ingestion channel, highway KM-marker resolver, service zone admin API.
Folder structure	<code>apps/core-api/src/modules/geo/</code> , <code>apps/dispatch/src/tracking/</code>
Database tables	<code>service_zones</code> (PostGIS polygon), <code>geofences</code> , <code>highway_markers</code> , <code>mechanic_locations</code> (Redis GEO hot path, Postgres cold)

APIs	<code>POST /v1/geo/resolve</code> (lat/lng → address + nearest KM marker + district + jurisdiction) · <code>WS /v1/track/:bookingId</code> · <code>POST /internal/geo/geofence-check</code> · zone CRUD
Components	Redis GEO for the sub-50 ms nearest-mechanic query; PostGIS for polygons and analytics. Adaptive location sampling — the server tells the client how often to report based on booking state (1/30 s idle, 1/5 s en-route).
Libraries	<code>postgis</code> , <code>ioredis</code> GEO commands, <code>@turf/turf</code> , <code>ws</code> .
Risks	WebSocket scale (100k concurrent) → sticky sessions via Redis pub/sub fan-out, tested in P14 load testing.
Security	Location is the most sensitive data we hold. Retention: precise traces 30 days, then aggregated. Access to another user's live location requires an active booking relationship — enforced and tested.
Testing	Geofence accuracy tests against known polygons; KM-marker resolution against 50 surveyed points; WebSocket reconnection/backpressure tests.
Documentation	Geo data model, location retention policy, tracking protocol.
Git branches	<code>feat/p09-d1-geofence-engine</code> , <code>feat/p09-d1-tracking-channel</code> , <code>feat/p09-d1-km-marker-resolver</code>
Time	3 ideal days
Demo	Drop a pin on a highway with no address; the API returns "NH-48, KM 212, near Manesar, Gurugram district, Haryana Highway Patrol jurisdiction." Then show live tracking with adaptive sampling reducing update rate when the mechanic is stationary.

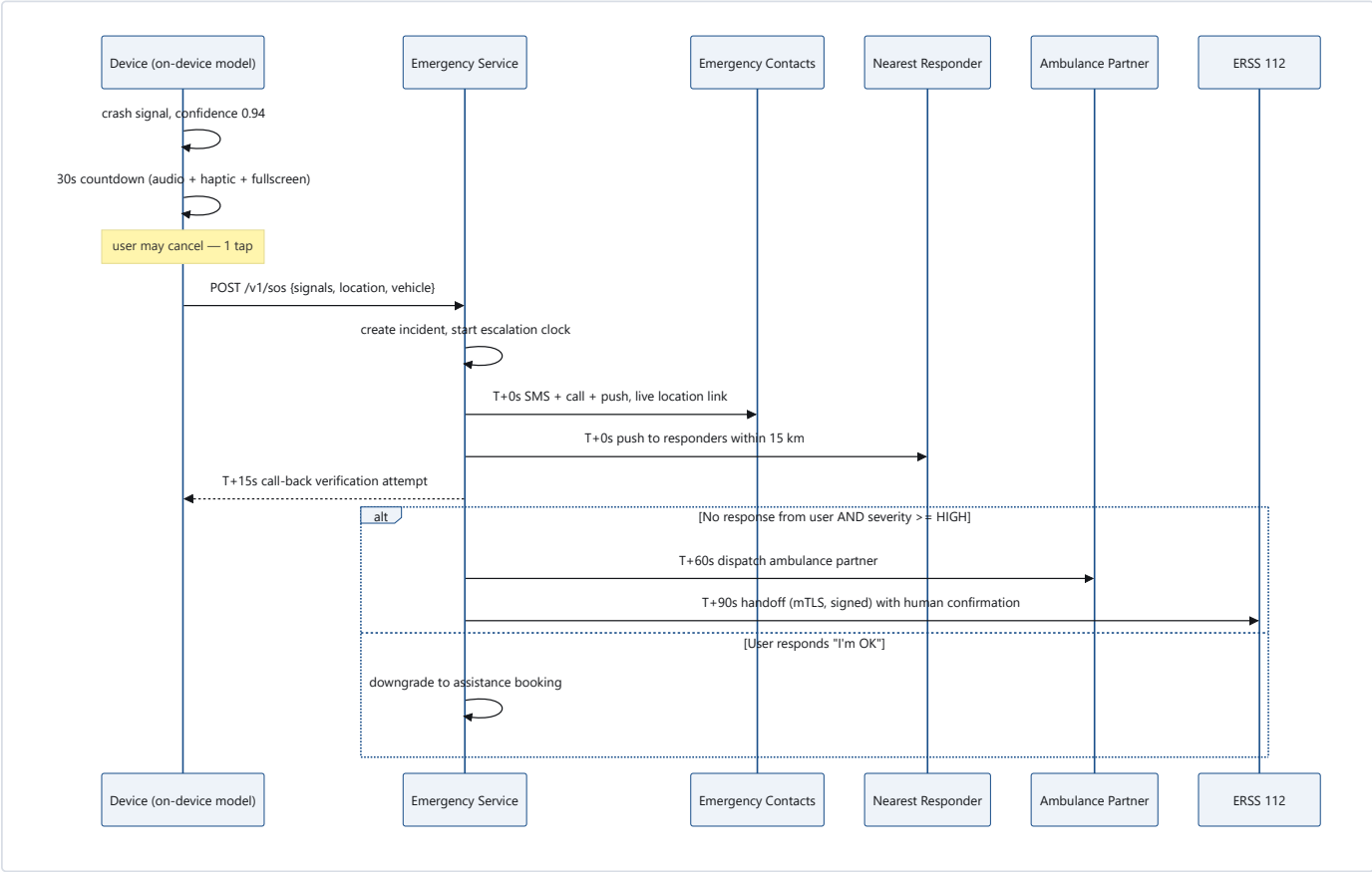
Phase 10 — Vehicle Diagnostics · S6 W14 · 3 days (support role)

Goal	Persist, serve, and act on diagnostics; make the mechanic arrive already knowing the problem.
Deliverables	Diagnostics service, DTC knowledge base (~5k codes × 8 languages), mechanic pre-brief, parts prediction → inventory check.
Folder structure	<code>apps/core-api/src/modules/diagnostics/</code>
Database tables	<code>dtc_codes</code> , <code>dtc_translations</code> , <code>diagnostic_sessions</code> , <code>diagnostic_findings</code> , <code>parts_catalogue</code> , <code>predicted_parts</code>
APIs	<code>POST /v1/diagnostics/sessions</code> · <code>POST /v1/diagnostics/sessions/:id/obd</code> · <code>POST /v1/diagnostics/sessions/:id/photo</code> · <code>POST /v1/diagnostics/sessions/:id/symptoms</code> · <code>GET /v1/diagnostics/sessions/:id/report</code> · <code>GET /v1/mechanic/jobs/:id/prebrief</code>
Components	Rules engine (deterministic fallback and the primary path for non-OBD vehicles), severity → drivability recommendation, parts prediction feeding a live inventory check against nearby mechanics.
AI models	Consume D3's diagnosis endpoints; own the rules-engine fallback.
Risks	A wrong "safe to drive" verdict is a safety failure → the rules engine biases conservative and overrides the model whenever the model says "safe" but any rule says "unsafe." Model can only downgrade safety, never upgrade it.
Security	Diagnostic photos may contain number plates and faces → stored encrypted, access limited to the assigned mechanic and the owner, auto-deleted after 180 days.
Testing	Rules-engine test over all severity combinations; assert the false-"safe" rate is exactly zero on the eval set.
Documentation	DTC knowledge base sourcing/licensing note, drivability rules table, pre-brief contract.
Git branches	<code>feat/p10-d1-diagnostics-service</code> , <code>feat/p10-d1-dtc-knowledge-base</code> , <code>feat/p10-d1-mechanic-prebrief</code>
Time	3 ideal days
Demo	Plug a dongle into a car with a seeded fault; the app shows a plain-Hindi explanation; the mechanic's phone shows the diagnosis, the two likely parts, and whether they have them in stock — before they've left.

Phase 11 — Emergency Engine · S7 W15 · 6 days · You own this phase

Goal	Build the one feature that must work when everything else has failed.
Deliverables	Emergency service (independently deployed, own quota, own alerting), escalation ladder, 112 handoff adapter, break-glass medical access with audit, degraded SMS-only path, drill mode.
Expected output	Crash detected → contacts and responders notified in under 10 seconds — and the same works with the rest of the platform switched off.
Folder structure	<code>apps/emergency/src/{intake,escalation,responder,handoff,degraded,drill}/</code> — a separate deployable , separate DB connection pool, separate alert channel.
Database tables	<code>incidents</code> , <code>incident_signals</code> , <code>incident_responses</code> , <code>responder_units</code> , <code>escalation_policies</code> , <code>break_glass_access</code> , <code>drill_runs</code>
APIs	<code>POST /v1/sos</code> (highest priority, minimal deps) · <code>POST /v1/sos/:id/cancel</code> · <code>GET /v1/incidents/:id</code> · <code>WS /v1/incidents/:id/room</code> · <code>POST /internal/emergency/crash-signal</code> · <code>POST /v1/emergency/break-glass</code> · <code>POST /internal/emergency/112-handoff</code> · <code>POST /v1/emergency/drill</code>
UI screens	Contract for D2's SOS button, countdown, incident room.
Components	Two-stage confirmation (sensor signal + 30 s cancellable countdown). Escalation ladder with per-step timeouts. Responder routing using P9. Break-glass access: any responder can read the medical profile, but it logs actor, reason, and time, and notifies the user afterwards. Degraded mode: a standalone SMS path that shares no runtime dependency with the main platform.
AI models	Consume D3's crash detection. The model never auto-dispatches. It can only <i>raise</i> an incident that a human confirms or that a second independent signal corroborates. This is written into the code, not just the policy.
Libraries	Minimal by design — fewer dependencies means fewer failure modes. Direct pg driver, direct SMS vendor SDK, no ORM in the degraded path.
Risks	(a) False positives dispatching real emergency units (R3) → two-stage confirm, 30 s window, call-back verification before any 112 handoff. (b) The emergency service itself failing (R9) → separate deploy, separate AZ spread, dedicated quota, degraded path with zero shared dependencies, monthly drill.
Security	Break-glass access is fully audited and irrevocably logged. Medical data encrypted at column level with a separate key. 112 handoff over mTLS with a signed payload. Location during an incident is shared only with confirmed responders, and the sharing ends when the incident closes.
Testing	Chaos test with every other service down — SOS must still work. Latency test asserting p95 < 10 s. Cancel-window test. False-positive suppression test. Break-glass audit completeness test. Monthly drill with a written report.
Documentation	Emergency runbook (highest-priority runbook in the repo), escalation policy doc, 112 integration spec, break-glass policy, drill procedure.
Git branches	<code>feat/p11-d1-emergency-service</code> , <code>feat/p11-d1-escalation-ladder</code> , <code>feat/p11-d1-112-handoff</code> , <code>feat/p11-d1-degraded-sms-path</code> , <code>feat/p11-d1-break-glass-audit</code>
Time	6 ideal days
Demo	Simulate a crash on a real device. Watch the countdown. Let it complete. Contacts get an SMS with a live location link, the nearest responder is notified, the incident room opens. Then scale the entire rest of the platform to zero replicas and do it again — it still works over the degraded SMS path.

Escalation ladder



Phase 12 — Government Dashboard · S7 W16 · 3 days (support role)

Goal	Serve government data that is genuinely useful and genuinely impossible to de-anonymize.
Deliverables	Gov service, anonymization/aggregation layer with query-time k-anonymity, gov read-only API, officer query audit.
Folder structure	<code>apps/core-api/src/modules/gov/{tenancy, anonymize, reports, audit}/</code>
Database tables	<code>gov_jurisdictions</code> , <code>gov_officers</code> , <code>gov_queries</code> , <code>gov_reports</code> , materialized views per aggregation grain
APIs	<code>GET /v1/gov/incidents</code> (aggregated) · <code>GET /v1/gov/blackspots</code> · <code>GET /v1/gov/response-times</code> · <code>POST /v1/gov/reports/:template</code> · <code>GET /v1/gov/corridors/:id/health</code>
Components	k-anonymity enforced at the query layer , not the UI: any result cell with fewer than 10 underlying records is suppressed, and the suppression is visible ("suppressed: below reporting threshold") rather than silently zeroed. Jurisdiction-scoped row-level security. Differential-privacy noise on the most sensitive aggregates.
Risks	Re-identification via query differencing (repeatedly querying overlapping slices to isolate an individual) → query budget per officer per period, plus logging that makes the attack detectable.
Security	Gov officers authenticate by mTLS + IP allowlist + short-lived tokens. Every query is logged with a purpose code. No endpoint returns a personal identifier under any parameter combination — proven by a dedicated re-identification test.
Testing	Re-identification test suite: attempt to isolate a single known individual through 20 query strategies; all must fail. k-anonymity boundary tests.
Documentation	Data-sharing agreement template, anonymization methodology, officer audit policy.
Git branches	<code>feat/p12-d1-gov-tenancy</code> , <code>feat/p12-d1-anonymization-layer</code> , <code>feat/p12-d1-gov-audit</code>
Time	3 ideal days

Demo	Play attacker: try to identify a specific person through the government API. Show every attempt failing, and show the audit log that would have flagged you.
------	--

Phase 13 — Analytics · S8 W17 · 3 days (support role)

Goal	Get data out of Postgres and into ClickHouse without touching production performance.
Deliverables	CDC pipeline (Debezium → Redpanda → ClickHouse), event taxonomy implementation, metric definition dictionary.
Folder structure	<code>apps/worker/src/analytics/</code> , <code>infra/cdc/</code>
Database tables	ClickHouse: <code>events</code> , <code>bookings_fact</code> , <code>incidents_fact</code> , <code>mechanic_daily</code> , <code>vehicle_daily</code>
APIs	<code>POST /v1/events/batch</code> (client telemetry ingest, sampled and rate-limited)
Components	Debezium CDC connectors, event schema validation at ingest (reject unknown events rather than accumulating garbage), PII stripping before anything lands in ClickHouse.
Risks	CDC lag or replication slot growth taking down Postgres → monitor slot lag as a first-class alert with a documented kill switch.
Security	No PII in ClickHouse. Ever. Enforced by a schema-level allowlist and a test that scans for identifier-shaped columns.
Testing	Pipeline lag test (<60 s p95), schema-rejection test, PII-leak scan in CI.
Documentation	Event taxonomy, metric dictionary (one definition per metric, no exceptions).
Git branches	<code>feat/p13-d1-cdc-pipeline</code> , <code>feat/p13-d1-event-taxonomy</code>
Time	3 ideal days
Demo	Create a booking; watch it appear in ClickHouse in under a minute, with all PII columns absent.

Phase 14 — Testing · S8 W18 · 5 days

Goal	Prove the backend is correct, including under failure.
Deliverables	Backend coverage ≥80% branch, full integration suite, Pact provider verification, load-test fixes, chaos-test fixes, zero P0/P1 bugs.
Components	Fix what D4's testing finds. This phase is mostly <i>your</i> debugging time, budgeted honestly rather than pretended away.
Risks	Load testing revealing an architectural problem too late → mitigate by running a load smoke test from S3 onward, not only in P14.
Security	Remediate every finding from D4's pentest and SAST/DAST runs. Nothing high or critical survives this phase.
Testing	Property-based tests for the state machine and pricing engine (<code>fast-check</code>). Mutation testing (Stryker) on the booking and pricing modules — coverage percentage means little if the tests don't actually assert anything.
Documentation	Test strategy contribution, known-limitations register.
Git branches	<code>test/p14-d1-property-tests</code> , <code>fix/p14-d1-load-findings</code> , <code>fix/p14-d1-security-findings</code>
Time	5 ideal days
Demo	Mutation testing score on the pricing engine; a property-based test finding a real edge case live.

Phase 15 — Deployment · S9 W19 · 3 days (support role)

Goal	Make the backend deployable, observable, and operable by someone who isn't you.
------	---

Deliverables	Health/readiness/liveness probes, graceful shutdown, migration-on-deploy strategy, service runbooks, SLO instrumentation.
Components	Expand→migrate→contract deploy discipline; connection draining; PgBouncer sizing; per-service RED metrics; SLO burn-rate alerts.
Risks	Migrations blocking a deploy at scale → all migrations must be online (<code>CREATE INDEX CONCURRENTLY</code> , no blocking <code>ALTER</code> s on large tables); migration lint enforces this in CI.
Security	No secret in any image or manifest — everything from Vault at runtime. Verified by D4's scan.
Testing	Rollback drill for every service. Migration rollback drill on a production-sized dataset copy.
Documentation	One runbook per backend service; migration playbook.
Git branches	<code>feat/p15-d1-health-probes</code> , <code>docs/p15-d1-service-runbooks</code>
Time	3 ideal days
Demo	Deploy a schema change to a table with 10M rows with zero downtime and zero locking, live.

Phase 16 — Optimization · S9 W20 · 4 days

Goal	p95 < 300 ms at production load, and a payload small enough for 2G.
Deliverables	Query optimization pass, caching layer, payload reduction, connection pool tuning, before/after performance report.
Components	<code>pg_stat_statements</code> -driven top-50 query tuning; index refinement (and index <i>removal</i> — unused indexes cost writes); Redis caching with explicit invalidation; protobuf on the tracking hot path; sparse fieldsets; read replicas for analytics-adjacent reads.
Risks	Cache invalidation bugs producing stale data in a safety context → nothing on the emergency or diagnostics path is cached. Caching is for catalogues and profiles, not decisions.
Testing	Performance budgets enforced in CI — a PR that regresses p95 on a tracked endpoint fails.
Documentation	Performance report with before/after numbers and the reasoning for each change.
Git branches	<code>perf/p16-d1-query-optimization</code> , <code>perf/p16-d1-caching-layer</code> , <code>perf/p16-d1-payload-reduction</code>
Time	4 ideal days
Demo	Side-by-side load test: before and after. Numbers, not adjectives.

Phase 17 — Future Roadmap · S10 W21–22 · 3 days

Goal	Say honestly what we built, what we owe, and what it takes to reach 10M users.
Deliverables	Technical debt register with repayment sequencing, scaling plan to 10M users with named bottlenecks, backend section of the 18-month roadmap.
Components	Scaling analysis: where the modular monolith must split first (dispatch, then emergency), when Postgres needs sharding (and by what key — <code>user_id</code> hash, decided now), when Redis needs clustering, what breaks at 100k concurrent WebSockets.
Risks	Debt register becoming a graveyard → each item gets a size, an impact, and a "pay by" phase. Anything unpaid after two phases is either done or formally accepted.
Documentation	Tech debt register, scaling plan, ADR-supersession review (which of our early decisions are now wrong?).
Git branches	<code>docs/p17-d1-scaling-plan</code> , <code>docs/p17-d1-tech-debt-register</code>
Time	3 ideal days
Demo	Present the scaling plan with the specific number at which each component breaks, and what it costs to fix.

D1 Effort Summary

Phase	Days	Phase	Days
P0 Research	4	P9 Maps	3
P1 Planning	4	P10 Diagnostics	3
P2 Architecture	5	P11 Emergency	6
P3 Database	6	P12 Gov	3
P4 Auth	6	P13 Analytics	3
P5 Backend APIs	15	P14 Testing	5
P6 Frontend support	3	P15 Deployment	3
P7 AI support	3	P16 Optimization	4
P8 Offline (server)	6	P17 Roadmap	3
		Total	85 ideal days

Remaining ~91 days across 22 weeks cover code review, incident response, unplanned work, meetings, and the rotating hats. This is deliberate: a plan with 100% allocation is a plan that fails in week 3.

ROADASSIST BHARAT

D2 - Frontend & Mobile Lead

Individual Roadmap

| *One Platform. Every Vehicle. Every Phone. Every Road.*

Document docs/03-frontend-lead-roadmap.md

Version 1.0 — planning baseline

Team 4 engineers · 22 weeks · 11 sprints · 18 phases

Status Baseline (frozen at Sprint 0 exit)

D2 — Frontend & Mobile Lead · Individual Roadmap

Owens: React Native app, Next.js portals (Gov / Mechanic / Admin / Fleet), design system, offline client, maps UI, Android Auto prototype, accessibility, localization. **Backs up:** D4 on E2E test authoring. **Total allocation:** 22 weeks · ~176 ideal days.

Standing responsibilities: - The **reference device** is the source of truth: Android 10, 2 GB RAM, throttled 3G. If it doesn't work there, it doesn't work. - Every screen ships with four states designed and built: loading, empty, error, offline. A screen with only a happy path is not done. - No hardcoded user-facing strings. Ever. Eight languages from day one. - You own perceived performance. A 300 ms API is irrelevant if the UI feels slow.

Phase 0 — Research · S0 W1 · 4 days

Goal	Understand who actually uses this — their phones, their thumbs, their languages, their signal.
Deliverables	Device & connectivity report; 20 user interview contributions with UX focus; competitor UX teardown; accessibility & literacy findings; Android Auto + feature-phone UX feasibility spikes.
Expected output	A design constraint list that is evidence-based, not aesthetic preference.
Folder structure	<code>docs/research/ux/</code> , <code>design/research/</code>
Database tables	None.
APIs	None.
UI screens	None — produce journey maps for 4 scenarios instead (highway breakdown, city no-start, night accident, fleet truck failure).
Components	None.
AI models	With D3: decide where AI is <i>visible</i> to the user and where it must be invisible. Users don't want "AI"; they want to know if the car will start.
Libraries	Evaluate: React Native (bare) vs Expo, Next.js 15, Tamagui vs Nativewind vs Restyle, Reanimated 3, WatermelonDB vs op-sqlite.
Risks	Designing for a flagship phone and discovering in P14 that it's unusable on a real device → mitigation: procure 3 low-end devices in week 1 and use them for <i>all</i> development testing, not just QA.
Security	Research secure input patterns for OTP and payment on shared/borrowed phones — a very common Indian usage pattern that most apps ignore.
Testing	None yet.
Documentation	Device matrix, connectivity report, 6 personas (with D1/D3), 4 journey maps, accessibility findings (literacy levels, vision, one-handed use, gloves, rain).
Git branches	<code>docs/p00-d2-ux-research</code> , <code>spike/p00-d2-android-auto</code>
Time	4 ideal days
Demo	Present the reference device physically. Show a competitor's app failing on it. That's the brief.

Design constraints (output of P0 — these are non-negotiable)

Constraint	Why
Works one-handed, in the rain, at night, on a highway shoulder	The actual usage context
Primary action reachable by thumb on a 6.5" screen	Users are standing next to a broken vehicle
Readable at arm's length in direct sunlight	Contrast $\geq$ 7:1 on primary actions
Comprehensible at low literacy	Icons + colour + voice, never text alone for critical actions
Every critical action has a voice alternative	Hands may be occupied or dirty
Never a blank screen while loading	Skeletons, always
Never a silent failure	Every failure has a visible, actionable state
$\leq$ 28 MB APK	Users on 32 GB phones with 40 apps
$\leq$ 30 MB/month data	Data cost is a real adoption barrier

Phase 1 — Planning · S0 W2 · 4 days

Goal	Turn the research into a frontend backlog, an information architecture, and a design direction.
Deliverables	Frontend epic breakdown (~65 stories with AC), estimates, information architecture, navigation map, low-fi wireframes for the 12 critical screens, design direction (3 options → 1 chosen).
Folder structure	<code>apps/mobile/</code> <code>apps/web/</code> <code>apps/mechanic/</code> <code>apps/gov/</code> <code>apps/admin/</code> <code>packages/ui/</code> <code>packages/design-tokens/</code> scaffolded.
APIs	Review D1's contract calendar; flag anything that would force a bad UX (e.g. an endpoint that needs 3 round-trips before the user sees anything).
UI screens	Wireframes: splash, onboarding, home/garage, request-help, diagnosis, matching, tracking, mechanic-detail, payment, history, SOS, profile.
Components	Component inventory derived from wireframes (~42 components).
Libraries	Locked: React Native 0.79 (bare), Expo modules, Next.js 15, TypeScript strict, Zustand + TanStack Query, Reanimated 3, MapLibre GL Native/JS, WatermelonDB, i18next, Tamagui.
Risks	Building 5 apps with 1 person → mitigation: a shared <code>packages/ui</code> with platform adapters means one component library serves all 5; the gov/admin/fleet portals share a single Next.js app with role-based routing rather than being three codebases.
Security	Decide the secure-storage strategy (Keychain/Keystore via <code>react-native-keychain</code>), and that no token ever touches AsyncStorage.
Testing	Test strategy: Jest + RNTL unit, Storybook + Chromatic visual, Detox E2E mobile, Playwright E2E web.
Documentation	IA diagram, navigation map, wireframes, design direction rationale, component inventory.
Git branches	<code>chore/p01-d2-app-scaffolds</code> , <code>docs/p01-d2-ia-wireframes</code>
Time	4 ideal days

Demo	Click-through wireframe prototype of the full breakdown journey.
------	--

Phase 2 — System Architecture · S1 W3 · 3 days (support role)

Goal	Ensure the API contract serves the UI, not the database.
Deliverables	Frontend architecture ADRs, state management strategy, consumer contract expectations (Pact) handed to D1.
Folder structure	<code>packages/{ui,design-tokens,api-client,offline,i18n,analytics}/</code>
APIs	Consumer-driven contracts: you specify what the UI needs; D1 implements it. Push hard for composite/BFF endpoints — on 2G, six round-trips is a broken product.
Components	Architecture: feature-sliced folders, container/presentational split, a <code>useOfflineQuery</code> hook wrapping TanStack Query + local DB as the single data access pattern.
AI models	Define the UI contract for AI results: every AI-derived value in the UI must carry a confidence indication and a "this is an estimate" affordance. Never present a model output as certain fact.
Libraries	<code>@pact-foundation/pact</code> , <code>openapi-typescript</code> .
Risks	State management sprawl → one rule: server state in TanStack Query, local UI state in component state, cross-screen client state in Zustand. Nothing else. Enforced in review.
Security	ADR: token storage, certificate pinning, screenshot prevention on payment/medical screens, deep-link validation.
Testing	Pact consumer contracts written for the 20 most important interactions.
Documentation	Frontend ADRs 1–6, state management guide, offline data access pattern.
Git branches	<code>docs/p02-d2-frontend-adrs</code> , <code>feat/p02-d2-pact-consumer-contracts</code>
Time	3 ideal days
Demo	Show a Pact contract failing D1's build when a field is removed. That's the safety net for the next 18 weeks.

Phase 3 — Database Design · S1 W4 · 2 days (support role)

Goal	Design the local database — the client's schema is not the server's schema.
Deliverables	Local schema design (WatermelonDB), the sync manifest (what a client holds locally), local storage budget.
Database tables (local)	<code>vehicles</code> , <code>bookings</code> , <code>booking_events</code> , <code>mechanics_cache</code> , <code>diagnostic_sessions</code> , <code>sync_queue</code> , <code>map_packs</code> , <code>dtc_cache</code> , <code>user_profile</code> , <code>emergency_contacts</code>
Components	Storage budget: ≤ 120 MB total (excluding map packs, which the user opts into per district). LRU eviction on caches.
Risks	Unbounded local growth filling a cheap phone's storage → hard caps per table, eviction policy, and a visible storage screen the user controls.

Security	Local DB encrypted (SQLCipher). Medical and payment data are never persisted locally at all.
Testing	Storage-cap test; eviction correctness test.
Documentation	Local schema, sync manifest, storage budget.
Git branches	<code>docs/p03-d2-local-schema</code>
Time	2 ideal days
Demo	Fill the local DB past the cap; show eviction keeping the app healthy instead of crashing.

Phase 4 — Authentication (clients) · S2 W5 · 4 days

Goal	A login that works for a 22-year-old with a flagship and a 55-year-old truck driver with a cracked screen.
Deliverables	Auth flow on mobile + web + mechanic app, secure token storage, silent refresh, session management UI, consent centre.
Folder structure	<code>apps/mobile/src/features/auth/</code> , <code>packages/api-client/src/auth/</code>
APIs	Consumes D1's <code>/v1/auth/*</code> and <code>/v1/consents</code> .
UI screens	Splash · Language select (before login — critical) · Phone entry · OTP entry (with auto-read on Android via SMS Retriever) · Profile setup · Consent centre · Session list · Biometric unlock setup
Components	<code><OTPInput></code> (auto-advance, paste, auto-read), <code><PhoneInput></code> (country code, validation), <code><ConsentToggle></code> (plain-language purpose description — no legalese), <code><SessionCard></code> , <code><BiometricGate></code> , auth interceptor with silent refresh + request queue during refresh.
AI models	None.
Libraries	<code>react-native-keychain</code> , <code>@react-native-firebase/auth</code> (SMS Retriever only), <code>react-native-biometrics</code> , <code>libphonenumber-js</code> .
Risks	(a) OTP auto-read failing → always keep manual entry equally prominent, never hide it. (b) Users on borrowed phones → explicit "this is a shared phone" mode that skips biometric setup and shortens the session.
Security	Tokens in Keychain/Keystore only, never AsyncStorage. Certificate pinning on auth and payment endpoints. Screenshot blocked on OTP and payment screens. No token in logs, ever — enforced by a lint rule. Silent refresh queues concurrent requests instead of firing N refreshes. Biometric gate for re-entry.
Testing	OTP flow E2E (Detox); token expiry mid-request test; refresh race-condition test (10 concurrent 401s → exactly 1 refresh); logout clears everything test.
Documentation	Auth flow UX doc, consent copy (reviewed for plain language in 8 languages).
Git branches	<code>feat/p04-d2-otp-flow</code> , <code>feat/p04-d2-secure-token-storage</code> , <code>feat/p04-d2-consent-centre</code>
Time	4 ideal days
Demo	Log in on the reference device on throttled 3G, in Hindi, with SMS auto-read. Then kill the app mid-refresh and show it recovering cleanly.

Phase 5 — Backend APIs · S2–S3 · 2 days (support role)

Goal	Keep the contract honest from the consumer side.
Deliverables	Pact consumer contracts kept current, API feedback to D1, mock-based development harness.
Components	MSW-based dev harness so the entire app is developable with zero backend running — this is what lets you and D1 work fully in parallel.
Risks	Developing against mocks that diverge from reality → Pact verification in D1's CI catches divergence within one merge cycle.
Testing	Pact consumer tests for 20 interactions.
Git branches	<code>test/p05-d2-pact-contracts</code>
Time	2 ideal days
Demo	Run the entire app offline from the backend, on mocks.

Phase 6 — Frontend · S3 W7 – S4 W10 · 18 days · You own this phase

Goal	Build the actual product. This is your largest block.
Deliverables	Design system package, React Native app (Android + iOS), web PWA, mechanic app, admin console, fleet dashboard, Android Auto prototype, Storybook, visual regression suite.
Expected output	A user can complete the full breakdown journey on a real low-end phone, in Hindi, on 3G.
Folder structure	See below.
Database tables	Local schema from P3 implemented.
APIs	Consumes essentially all of D1's v1 surface.
UI screens	See screen inventory below.
Components	~42 in <code>packages/ui</code> . See component inventory below.
AI models	Consumes D3's endpoints via D1's gateway. UI rule: every AI output shows confidence and offers "not right? tell us" feedback that flows back to D3's training loop.
Libraries	<code>react-native</code> , <code>@react-navigation/native</code> , <code>@tanstack/react-query</code> , <code>zustand</code> , <code>react-native-reanimated</code> , <code>react-native-gesture-handler</code> , <code>@shopify/flash-list</code> (never FlatList for long lists on low-end devices), <code>react-native-svg</code> , <code>i18next</code> , <code>react-native-mmkv</code> , <code>nativewind</code> , <code>next</code> , <code>@radix-ui/*</code> , <code>framer-motion</code> , <code>recharts</code> .
Risks	(a) 5 apps, 1 person, 18 days → the shared <code>packages/ui</code> and a single Next.js app for all three portals is what makes this possible; the Android Auto build is explicitly a <i>prototype</i> and is the first thing cut if the sprint slips. (b) Jank on low-end devices → Reanimated on the UI thread for all animation, FlashList everywhere, memoization discipline, and a profiling pass at the end of each sprint rather than at the end of the phase.

Security	Screenshot prevention on payment/medical screens. Deep links validated and never used to bypass auth. No PII in analytics events. Clipboard cleared after copying sensitive values. WebView usage minimized and sandboxed.
Testing	Unit (Jest + RNTL) ≥80% on logic; Storybook stories for every component; visual regression (Chromatic) on the design system; Detox E2E for 8 critical journeys; manual pass on the reference device every sprint.
Documentation	Storybook as living documentation, design token reference, component API docs, contribution guide, localization guide.
Git branches	<code>feat/p06-d2-design-system</code> , <code>feat/p06-d2-onboarding</code> , <code>feat/p06-d2-garage</code> , <code>feat/p06-d2-request-help-flow</code> , <code>feat/p06-d2-live-tracking</code> , <code>feat/p06-d2-payment-flow</code> , <code>feat/p06-d2-mechanic-app</code> , <code>feat/p06-d2-web-pwa</code> , <code>feat/p06-d2-admin-console</code> , <code>feat/p06-d2-android-auto</code>
Time	18 ideal days
Demo	The full journey on a real ₹8,000 Android phone, throttled to 3G, in Hindi, one-handed, with the screen brightness at maximum to simulate sunlight. No simulator.

Folder structure

```

apps/
├─ mobile/                                # React Native (bare) - citizen + mechanic in one binary, role-switched
│  └─ src/
│     ├─ features/
│     │  ├─ auth/ garage/ request/ diagnose/ tracking/
│     │  └─ payment/ history/ profile/ sos/ mechanic/
│     └─ navigation/
│        ├─ offline/                    # sync client, queue, conflict UI
│        └─ services/                  # api, location, ble, notifications
│           └─ app.tsx
├─ web/                                # Next.js 15 - citizen PWA
├─ portals/                            # Next.js 15 - gov + admin + fleet, role-routed
│  └─ src/app/(gov)/ (admin)/ (fleet)/
└─ auto/                              # Android Auto prototype

packages/
├─ ui/                                # design system, platform-adaptive
│  └─ primitives/ patterns/ motion/ icons/
├─ design-tokens/                    # source of truth, exports to RN + CSS + Figma
├─ api-client/                      # generated from OpenAPI + auth interceptor
├─ offline/                         # WatermelonDB models, sync engine, queue
├─ maps/                           # MapLibre wrappers, offline pack manager
├─ i18n/                            # 8 locales
└─ analytics/                      # typed event emitter matching D3's taxonomy

```

Screen inventory (48 screens)

Area	Screens
Onboarding (6)	Splash, Language select, Phone entry, OTP, Profile setup, Permissions primer
Garage (7)	Home/garage, Add vehicle, Vehicle detail, Vehicle health, Service history, Documents, Scan RC
Request help (9)	Problem picker, Symptom chat, Photo capture, OBD scan, Diagnosis result, Location confirm, Service selection, Quote review, Confirm
Live (6)	Matching, Mechanic assigned, Live tracking, Chat, On-site, Work in progress

Area	Screens
Payment (5)	Invoice, Payment method, UPI flow, Success, Receipt
Emergency (4)	SOS button (persistent), Countdown, Incident room, Emergency contacts
Account (6)	Profile, Sessions, Consent centre, Subscription, Language & accessibility, Storage & offline
Mechanic (8)	Job feed, Job detail + pre-brief, Navigation, Checklist, Parts, Invoice builder, Earnings, Availability
History (3)	Booking list, Booking detail, Reviews
Offline (2)	Sync status, Conflict resolution
Portals	Gov (P12), Admin (12 screens), Fleet (9 screens)

Component inventory (42 core components)

Category	Components
Primitives	Button (5 variants, 3 sizes, loading + disabled), Input , OTPInput , PhoneInput , Select , Checkbox , Radio , Switch , Slider , Text , Icon , Avatar , Badge , Divider , Spinner , Skeleton
Layout	Screen (safe area + scroll + refresh + offline banner), Card , Sheet , Modal , Tabs , Accordion , List , EmptyState , ErrorState , OfflineState
Domain	VehicleCard , MechanicCard , BookingStatusTimeline , PriceBreakdown , DiagnosisResult (with confidence), SOSButton , LiveMap , TrackingBottomSheet , RatingInput , PartsList , ChecklistItem
Feedback	Toast , Banner , ProgressBar , ConfidenceIndicator , SyncStatusPill

Design system foundations

Token group	Decision
Colour	Semantic tokens only (bg.primary , text.danger), never raw hex in components. Light + dark. Primary action contrast ≥ 7:1 (AAA) because of sunlight.
Type	System font stack (no custom font download — saves ~400 KB and renders 8 scripts correctly). Scale 12/14/16/18/22/28/34. Line height 1.5 minimum for Devanagari/Tamil ascenders.
Spacing	4 pt base. 4/8/12/16/24/32/48/64.
Radius	4/8/12/16/full.
Elevation	4 levels; on Android use elevation, on iOS use shadow — abstracted in the token layer.
Motion	Durations 120/200/320 ms. Easing: standard, decelerate, accelerate, spring for gestures. All animation respects prefers-reduced-motion and is disabled automatically on low-end devices detected at runtime.
Touch targets	Minimum 44×44 dp. Primary actions 56 dp.
Iconography	One 24 dp grid set, outline + filled. Every icon paired with text or an accessible label — never icon-only for a critical action.

Phase 7 — AI Development · S4–S5 · 2 days (support role)

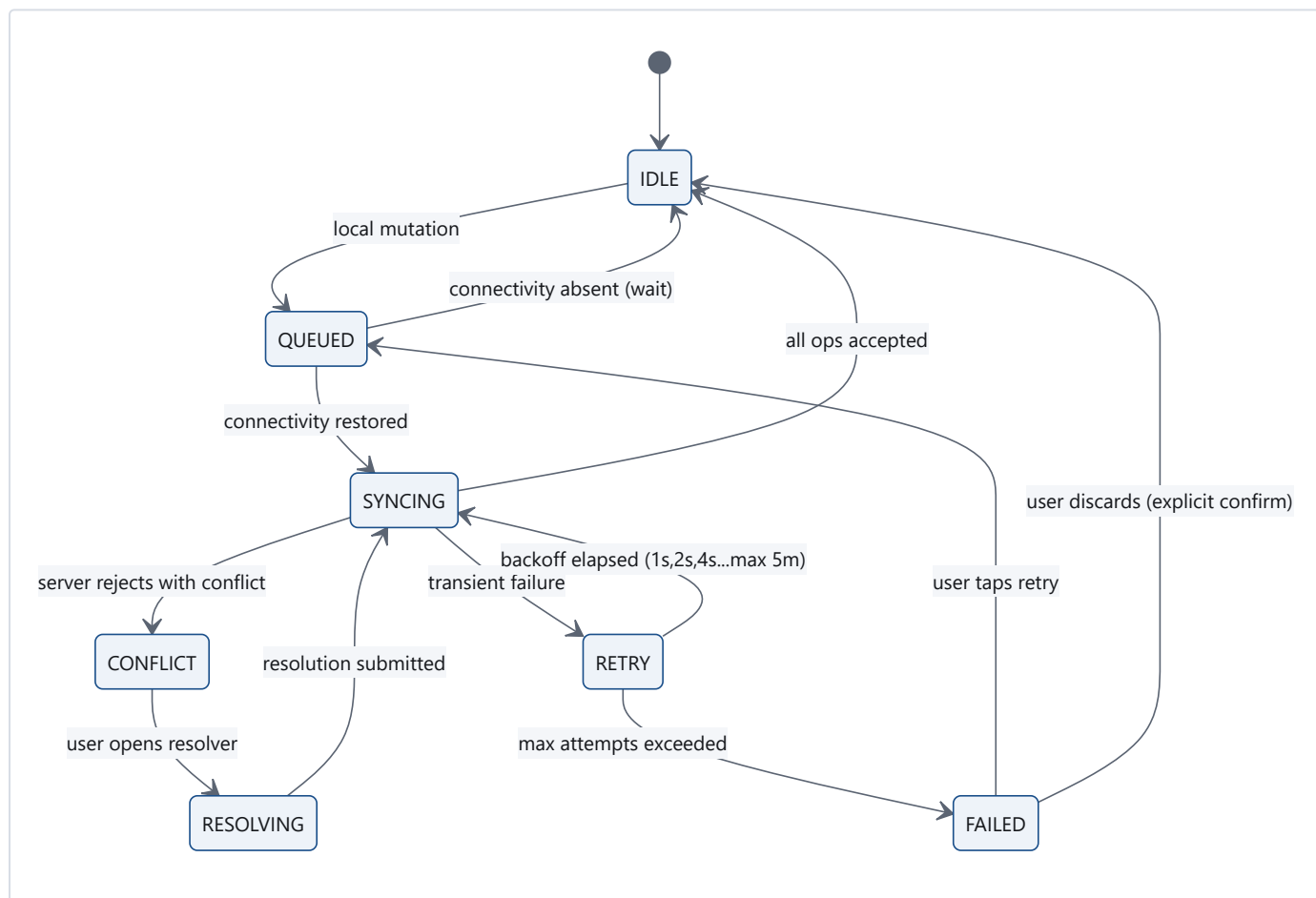
Goal	Present AI honestly in the UI, and build the feedback loop that improves it.
Deliverables	<code><ConfidenceIndicator></code> , AI result presentation patterns, user feedback capture flowing to D3, voice assistant UI shell.
Components	Confidence bands (high/medium/low) rendered as plain language ("we're fairly sure" / "this is a guess — a mechanic will confirm"), never as a raw percentage. Feedback: thumbs + free text + "what was it actually?" — which is the highest-value training signal we can collect.
AI models	Consumes diagnosis, matching, voice.
Risks	Users over-trusting a confident-looking wrong answer → design rule: any AI output that affects a safety decision is presented as <i>advisory</i> , with the conservative rule-based verdict shown alongside it and given visual primacy.
Security	Voice recordings require explicit per-session consent and are never retained beyond the session unless the user opts in.
Testing	Test that low-confidence outputs render the correct hedging language in all 8 locales.
Documentation	AI presentation guidelines.
Git branches	<code>feat/p07-d2-confidence-ui</code> , <code>feat/p07-d2-ai-feedback-loop</code>
Time	2 ideal days
Demo	Show the same diagnosis at 0.95 and 0.45 confidence. The UI says genuinely different things.

Phase 8 — Offline Engine (client) · S5 W11–12 · 10 days · You own this phase

Goal	Make the app fully usable with no connectivity. This is the feature that differentiates the product.
Deliverables	Offline client library, operation queue, sync engine, conflict resolution UI, offline map pack manager, background sync, sync status UX.
Expected output	A user in airplane mode completes a help request, sees honest offline state, and everything syncs correctly on reconnect.
Folder structure	<code>packages/offline/src/{db,queue,sync,conflict,network}/</code>
Database tables (local)	As designed in P3, now implemented with migrations.
APIs	Consumes D1's <code>/v1/sync/*</code> .
UI screens	Sync status screen, conflict resolution screen, offline banner (persistent, non-dismissible while offline), storage management screen, map pack downloader.
Components	<code>useOfflineQuery</code> / <code>useOfflineMutation</code> (the <i>only</i> data access pattern in the app — this is what makes offline uniform rather than per-screen). <code><SyncStatusPill></code> , <code><OfflineBanner></code> , <code><ConflictResolver></code> , <code><QueuedOperationList></code> .
AI models	Integrate D3's on-device models so diagnosis works fully offline.

Libraries	<code>@nozbe/watermelondb</code> , <code>react-native-mmkv</code> , <code>@react-native-community/netinfo</code> , <code>react-native-background-fetch</code> , <code>@msgpack/msgpack</code> .
Risks	(a) Optimistic UI showing success for something that later fails to sync → every optimistic write is visibly marked "pending" until confirmed; a failed sync surfaces as an actionable notification, never a silent revert. (b) Battery drain from background sync → sync only on unmetered + charging for non-urgent, immediate for urgent, with an explicit priority queue.
Security	Local DB encrypted with SQLCipher, key in Keychain. Queued operations signed with the device key. Nothing sensitive (medical, payment) is ever queued locally.
Testing	Offline test matrix: airplane mode, flaky 2G (10% packet loss), mid-sync app kill, device reboot with a full queue, clock skew ± 24 h, storage full, sync of 100 ops. Detox tests that toggle connectivity mid-journey.
Documentation	Offline architecture doc, sync status state chart, conflict UX guidelines.
Git branches	<code>feat/p08-d2-local-db</code> , <code>feat/p08-d2-operation-queue</code> , <code>feat/p08-d2-sync-engine</code> , <code>feat/p08-d2-conflict-ui</code> , <code>feat/p08-d2-offline-ux</code>
Time	10 ideal days
Demo	Put the phone in airplane mode. Add a vehicle, run a diagnosis (on-device model), create a help request, view an offline map, cancel and re-create. Turn connectivity back on. Everything syncs; one deliberate conflict is raised and resolved by the user in a screen that makes the choice obvious.

Sync state machine (client)



Phase 9 — Maps · S6 W13 · 7 days · You own this phase

Goal	Maps that work with the device fully offline, at a cost of zero per tile.
Deliverables	MapLibre component library, offline map pack downloader + delta updates, live tracking UI, geofence visualization, navigation handoff.
Folder structure	<code>packages/maps/src/{native,web,packs,tracking}/</code>
UI screens	Map pack manager (browse by state → district, size shown, download progress, delete), full-screen map, tracking view, location picker with landmark search.
Components	<code><Map></code> (platform-adaptive MapLibre wrapper), <code><MapMarker></code> , <code><RouteLine></code> , <code><TrackingSheet></code> , <code><LocationPicker></code> (with "I'm near a landmark" mode for addresses that don't exist), <code><MapPackCard></code> , <code><KmMarkerDisplay></code> .
Libraries	<code>@maplibre/maplibre-react-native</code> , <code>maplibre-gl</code> (web), <code>@turf/turf</code> , <code>react-native-blob-util</code> (pack download with resume).
Risks	(a) Map pack size making downloads impractical on limited data → target ≤60 MB/district, downloads resume, and only on unmetered by default. (b) Battery drain from continuous location → adaptive sampling driven by booking state, significant-change API when idle, and a visible "location sharing active" indicator so it's never covert.
Security	Location sharing is time-bounded and relationship-bounded; the UI always shows who can currently see your location and offers one-tap revocation.
Testing	Offline navigation test with the device in airplane mode; pack download resume-after-kill test; battery measurement over 1 hour of tracking.
Documentation	Maps integration guide, offline pack strategy, location privacy UX.
Git branches	<code>feat/p09-d2-maplibre-components</code> , <code>feat/p09-d2-offline-map-packs</code> , <code>feat/p09-d2-live-tracking-ui</code> , <code>feat/p09-d2-location-picker</code>
Time	7 ideal days
Demo	Download the Gurugram district pack over WiFi. Airplane mode. Search for a landmark, get directions, watch the route render — entirely offline. Then show battery usage after an hour of live tracking.

Phase 10 — Vehicle Diagnostics · S6 W14 · 5 days

Goal	Make diagnosis feel like talking to a knowledgeable friend, not filling in a form.
Deliverables	OBD-II BLE integration, diagnosis conversation UI, photo capture flow, diagnosis result presentation, mechanic pre-brief screen.
Folder structure	<code>apps/mobile/src/features/diagnose/</code> , <code>packages/ble/</code>
UI screens	Diagnosis entry (4 paths: describe / photo / listen / plug in), symptom conversation, OBD pairing + scan, photo capture with guidance overlay, diagnosis result, "can I drive it?" verdict, mechanic pre-brief.

Components	<code><SymptomChat></code> (chat UI with quick-reply chips — typing is hard on a highway), <code><OBDPairing></code> , <code><GuidedPhotoCapture></code> (overlay showing what to photograph, with real-time quality feedback), <code><DiagnosisCard></code> , <code><DrivabilityVerdict></code> (the most safety-critical component in the app), <code><PartsList></code> .
AI models	Consumes D3's photo diagnosis, symptom triage, and on-device fallback.
Libraries	<code>react-native-ble-plx</code> , <code>react-native-vision-camera</code> , <code>react-native-image-resizer</code> .
Risks	(a) BLE pairing UX is notoriously bad → a guided flow with an illustrated "find your OBD port" step per vehicle model, plus an explicit "skip, describe it instead" escape hatch that is always visible. (b) Poor-quality photos degrading model accuracy → real-time capture guidance (framing, focus, lighting) rather than rejecting after the fact.
Security	Photos may contain number plates and bystanders — the capture screen warns, and images are uploaded over TLS with pinning and deleted locally after successful upload.
Testing	BLE tests against 3 dongle models; photo capture quality tests; verdict rendering tests asserting the conservative verdict always wins visually.
Documentation	BLE integration guide, photo capture guidelines, drivability verdict design rationale.
Git branches	<code>feat/p10-d2-obd-ble</code> , <code>feat/p10-d2-symptom-chat</code> , <code>feat/p10-d2-guided-photo</code> , <code>feat/p10-d2-diagnosis-result</code>
Time	5 ideal days
Demo	Photograph a real damaged part; get a diagnosis in Hindi with a confidence band, a parts list, and a clear "do not drive this" verdict. Then unplug everything and do the symptom-chat path offline.

Phase 11 — Emergency Engine (clients) · S7 W15 · 4 days

Goal	An SOS that a panicking person can use, and that a person who is unconscious doesn't need to.
Deliverables	SOS UI across all clients, crash countdown, incident room, emergency contact management, degraded SMS UI.
UI screens	Persistent SOS affordance (reachable from every screen), crash countdown (full-screen, audio, haptic, unmissable), incident room (live location, responder ETA, contacts notified), emergency contacts setup, medical profile.
Components	<code><SOSButton></code> (long-press to prevent accidental triggering, with haptic progress), <code><CrashCountdown></code> (30 s, one enormous cancel button, screen-reader announcement, works over the lock screen), <code><IncidentRoom></code> , <code><ResponderCard></code> .
AI models	Consumes D3's on-device crash detection.
Risks	Accidental SOS triggering, which erodes trust and wastes responder capacity → long-press with haptic feedback, plus the 30 s countdown, plus a post-cancel "was that an accident?" prompt that feeds D3's false-positive dataset.
Security	The countdown screen must render over the lock screen and in Do Not Disturb. Medical data is displayed to responders only through the break-glass flow, and the user is told afterwards who accessed it.
Testing	Countdown accessibility test (screen reader, one-handed, gloves). Lock-screen render test. Accidental-trigger rate measured in dogfooding. Degraded-mode UI test with the backend down.
Documentation	Emergency UX rationale, accessibility notes for the countdown.

Git branches	<code>feat/p11-d2-sos-button</code> , <code>feat/p11-d2-crash-countdown</code> , <code>feat/p11-d2-incident-room</code>
Time	4 ideal days
Demo	Trigger a simulated crash with the phone locked and in a pocket. The countdown takes over the screen, sounds an alarm, and vibrates. Cancel it. Then let it complete and watch the incident room open with contacts already notified.

Phase 12 — Government Dashboard · S7 W16 · 6 days · You own this phase

Goal	A dashboard a District Transport Officer can actually use, on a government-issue desktop with an old browser.
Deliverables	Gov portal (Next.js), live incident map, blackspot analysis, response-time analytics, 6 statutory report generators, officer audit view.
Folder structure	<code>apps/portals/src/app/(gov)/</code>
UI screens	Login (mTLS + cert guidance), jurisdiction dashboard, live incident map, blackspot analysis, corridor health, response-time analytics, report builder, report archive, audit log, settings.
Components	<code><IncidentMap></code> (clustered, 10k markers, severity layers, no jank), <code><HeatmapLayer></code> , <code><CorridorSelector></code> , <code><ReportBuilder></code> (parameterized, previewable, exportable to PDF/XLSX/CSV), <code><SuppressionNotice></code> (shown wherever k-anonymity suppresses a cell — transparency about <i>why</i> data is missing builds trust), <code><AuditTable></code> .
AI models	Consumes D3's forecasting for predicted-demand and risk overlays.
Libraries	<code>next</code> , <code>maplibre-gl</code> , <code>deck.gl</code> (for 10k+ point rendering), <code>recharts</code> , <code>@tanstack/react-table</code> , <code>exceljs</code> , <code>@react-pdf/renderer</code> .
Risks	(a) Government desktops running old browsers → target baseline is Chrome 100 / Edge 100, no bleeding-edge APIs, and a graceful message rather than a blank page below baseline. (b) 10k markers killing the browser → deck.gl WebGL layer with clustering, tested at 50k.
Security	Client-side certificate flow documented for non-technical officers. Session timeout 30 min. No data cached in the browser beyond the session. Every export watermarked with the officer ID and timestamp.
Testing	Playwright E2E for report generation; performance test at 10k and 50k incidents; browser compatibility matrix; a11y audit (government accessibility standards apply).
Documentation	Officer user guide (written for a non-technical audience, in English + Hindi), report format specifications.
Git branches	<code>feat/p12-d2-gov-portal-shell</code> , <code>feat/p12-d2-incident-map</code> , <code>feat/p12-d2-blackspot-analysis</code> , <code>feat/p12-d2-report-builder</code>
Time	6 ideal days
Demo	Log in as a district officer. See live incidents in your jurisdiction only. Identify a blackspot. Generate the statutory monthly report as a PDF. Then try to see individual citizen data and fail.

Phase 13 — Analytics (dashboards) · S8 W17 · 4 days

Goal	Give fleets and mechanics numbers they'll act on, and give us numbers we'll act on.
Deliverables	Admin analytics dashboards, fleet analytics screens, mechanic earnings analytics, internal ops dashboard.
UI screens	Ops overview, demand/supply heatmap, funnel view, cohort retention, fleet cost dashboard, fleet vehicle health, driver behaviour, mechanic earnings, mechanic utilization.
Components	<code><MetricCard></code> (value + delta + sparkline + definition tooltip — every metric links to its dictionary definition), <code><TimeSeriesChart></code> , <code><FunnelChart></code> , <code><CohortGrid></code> , <code><HeatmapCalendar></code> , <code><DrilldownTable></code> .
Libraries	<code>recharts</code> , <code>@tanstack/react-table</code> , <code>date-fns</code> .
Risks	Dashboards nobody uses → each dashboard is built for a named person answering a named recurring question. If we can't name them, we don't build it.
Security	Fleet admins see only their fleet. Mechanics see only their own earnings. Row-level enforcement is server-side; the UI never filters for security.
Testing	Load test at 100M underlying events; chart rendering performance; permission boundary tests.
Documentation	Dashboard guide per audience.
Git branches	<code>feat/p13-d2-admin-analytics</code> , <code>feat/p13-d2-fleet-dashboard</code> , <code>feat/p13-d2-mechanic-earnings</code>
Time	4 ideal days
Demo	Fleet manager view: cost per km by vehicle, downtime ranking, and the one truck that's costing them the most. That's a dashboard someone will open on a Monday.

Phase 14 — Testing · S8 W18 · 6 days

Goal	Prove it works on real devices, in real languages, for real people.
Deliverables	Detox E2E for 12 journeys, Playwright E2E for portals, visual regression baseline, device farm results across 12 Android + 4 iOS, accessibility audit, localization QA across 8 languages, bug fixes to zero P0/P1.
Components	Fix what testing finds. Budget honestly — this phase is mostly debugging.
Risks	Localization breaking layouts (German-length problem, but for Tamil and Devanagari) → automated screenshot diffing across all 8 locales catches truncation and overflow before a human sees it.
Security	Verify no PII in analytics events, logs, or crash reports. Verify screenshot prevention actually works on both platforms.
Testing	Full matrix. Accessibility: TalkBack and VoiceOver walkthrough of every critical journey, not just an automated scan.
Documentation	Device compatibility matrix, accessibility audit report, localization QA report.
Git branches	<code>test/p14-d2-detox-journeys</code> , <code>test/p14-d2-visual-regression</code> , <code>fix/p14-d2-device-issues</code> , <code>fix/p14-d2-all-findings</code>
Time	6 ideal days

Demo	The full journey driven entirely by TalkBack with the screen off. If a blind user can request roadside assistance, the app is well built.
-------------	---

Phase 15 — Deployment · S9 W19 · 3 days

Goal	Ship to the stores and to the web, repeatably.
Deliverables	Play Store + App Store submissions with staged rollout, web deployment, CodePush/OTA update channel, crash reporting, store listing assets in 8 languages.
Components	Fastlane pipelines, signing key management (with D4), staged rollout (1% → 10% → 50% → 100% gated on crash-free rate), OTA for JS-only fixes.
Risks	Store review rejection delaying launch → submit a TestFlight/internal-track build in P14, not P15, so rejection reasons surface early.
Security	Signing keys in Vault, never on a laptop. Certificate pinning verified in the release build (it's easy to accidentally ship debug config).
Testing	Release-build smoke test on real devices — release builds behave differently from debug builds and this catches real bugs.
Documentation	Release process runbook, store listing copy.
Git branches	<code>chore/p15-d2-fastlane</code> , <code>chore/p15-d2-store-assets</code>
Time	3 ideal days
Demo	Push a JS-only fix via OTA and watch it land on a device in under 2 minutes.

Phase 16 — Optimization · S9 W20 · 5 days

Goal	28 MB, 2 seconds, 30 MB/month, 60 fps. Numbers, not adjectives.
Deliverables	APK size reduction, startup optimization, jank elimination, memory leak fixes, data usage reduction, web bundle optimization, performance budgets in CI.
Components	R8/Proguard + resource shrinking + Hermes bytecode; dynamic feature modules for maps and diagnostics; startup profiling with Perfetto; Flipper-driven leak hunting; FlashList audit; image transcoding to AVIF with WebP fallback; delta sync; protobuf on the tracking channel; route-level code splitting and critical CSS on web.
Risks	Optimization breaking correctness → every optimization PR must keep the full E2E suite green; no exceptions for "it's just a size change."
Testing	Performance budgets wired into CI: APK size, bundle size, startup time, and frame drop rate all fail the build when exceeded.
Documentation	Performance report with before/after, and a note on what we chose <i>not</i> to optimize and why.
Git branches	<code>perf/p16-d2-apk-size</code> , <code>perf/p16-d2-startup</code> , <code>perf/p16-d2-jank</code> , <code>perf/p16-d2-data-usage</code>
Time	5 ideal days

Demo	Two identical reference devices side by side, old build vs new. Cold start both simultaneously.
-------------	---

Phase 17 — Future Roadmap · S10 · 3 days

Goal	Say what the frontend needs next and what it owes.
Deliverables	Frontend tech debt register, design system v2 plan, platform expansion analysis (Android Auto → production, CarPlay, wearables, KaiOS for feature phones), frontend section of the 18-month roadmap.
Documentation	Debt register, design system evolution plan, platform expansion analysis with effort estimates.
Git branches	docs/p17-d2-frontend-roadmap
Time	3 ideal days
Demo	Present the Android Auto prototype and an honest assessment of what production readiness costs.

D2 Effort Summary

Phase	Days	Phase	Days
P0 Research	4	P9 Maps	7
P1 Planning	4	P10 Diagnostics	5
P2 Architecture	3	P11 Emergency	4
P3 Local DB	2	P12 Gov Dashboard	6
P4 Auth clients	4	P13 Analytics UI	4
P5 API support	2	P14 Testing	6
P6 Frontend	18	P15 Deployment	3
P7 AI UI	2	P16 Optimization	5
P8 Offline client	10	P17 Roadmap	3
		Total	92 ideal days

ROADASSIST BHARAT

D3 - AI & Data Science Lead

Individual Roadmap

| *One Platform. Every Vehicle. Every Phone. Every Road.*

Document docs/04-ai-lead-roadmap.md

Version 1.0 — planning baseline

Team 4 engineers · 22 weeks · 11 sprints · 18 phases

Status Baseline (frozen at Sprint 0 exit)

D3 — AI & Data Science Lead · Individual Roadmap

Owens: all 9 AI subsystems, model training and serving, on-device inference, feature store, voice/IVR NLU, analytics pipeline, government forecasting. **Backs up:** D1 on the analytics service. **Total allocation:** 22 weeks · ~176 ideal days.

Standing responsibilities: - **Define the baseline before you train.** A model that has no documented baseline cannot be said to work. - Every model ships with a **model card** and a **fallback**. No exceptions, including for models you're proud of. - No model is ever on the critical path for correctness or safety. Models advise; rules decide. - Latency and size budgets are hard constraints, not aspirations. A 94%-accurate model that takes 4 seconds on a 2 GB phone is a failed model.

The Nine AI Systems — Specification Summary

#	System	Inputs	Outputs	Approach	Where it runs	Baseline to beat
1	Vehicle Diagnosis AI	Symptom text/voice, vehicle model, DTC codes, telemetry, history	Ranked probable causes + severity + drivability + parts	Retrieval (BM25+dense) over a curated fault KB → gradient-boosted classifier → LLM for the natural-language explanation only	Server (LLM), on-device (classifier)	Rules engine on DTC lookup: top-3 acc. 0.62
2	Image Diagnosis AI	1–5 photos	Damage type, affected part, severity, repair estimate	Fine-tuned vision backbone (EfficientNetV2-S / MobileViT), multi-label + severity head	Server; quantized on-device variant	Human non-expert labelling: top-3 acc. 0.55
3	Voice Assistant	Speech, 8 languages + code-switching	Intent + slots + spoken response	Whisper-small fine-tuned on Indian-accented speech → intent classifier + slot filler → TTS	Server; on-device keyword spotting for "SOS"	Keyword matching: intent acc. 0.48
4	Predictive Maintenance	Telemetry time series, service history, model, usage pattern	Failure probability per component over 30/90 days + recommended service date	Survival analysis (Cox / DeepSurv) + LightGBM per component	Server, batch nightly	Fixed manufacturer service interval: recall@30d 0.31
5	Smart Mechanic Matching	Request context, mechanic features, geo, historical outcomes	Ranked mechanic list with acceptance probability + predicted ETA	Learning-to-rank (LambdaMART) + separate ETA regressor	Server, <300 ms	Distance + rating sort: acceptance@1 0.41
6	Crash Detection	Accelerometer, gyroscope, GPS speed, barometer (100 Hz windows)	Crash probability + severity estimate	1D CNN + GRU on sensor windows, INT8 quantized	On-device only	Threshold on peak G-force: FPR 1 per 12 driving hours

#	System	Inputs	Outputs	Approach	Where it runs	Baseline to beat
7	Fraud Detection	Booking graph, payment patterns, device fingerprints, GPS traces, review patterns	Risk score + reason codes	Graph features + isolation forest + supervised GBM on confirmed cases	Server, async	Manual review rules: precision 0.34
8	Government Analytics	Historical incidents, weather, traffic, road geometry, events	Spatiotemporal risk forecast, blackspot ranking, demand prediction	ST-GNN / temporal fusion transformer + spatial clustering	Server, batch	Historical-average-by-location: MAE baseline recorded in P13
9	Offline AI Bundle	Symptoms, DTC, sensors	Diagnosis + crash detection with zero connectivity	Quantized/distilled versions of #1, #3 (keyword), #6	On-device , ≤15 MB total	N/A — the alternative is nothing

Phase 0 — Research · S0 W1 · 4 days

Goal	Establish what data actually exists, what can be acquired, and which models are realistic in 22 weeks with one person.
Deliverables	Data availability audit, dataset acquisition plan, model feasibility assessment per system, 2 spike reports (crash detection from public sensor datasets; ASR quality on Indian-accented speech), labelling strategy and cost.
Expected output	An honest classification of each of the 9 systems: <i>trainable now</i> / <i>trainable with acquired data</i> / <i>rules-only in v1, ML in v2</i> .
Folder structure	<code>docs/research/ai/</code> , <code>ml/notebooks/00-feasibility/</code>
Database tables	None — but produce the feature requirements list that D1 designs the feature store around in P3.
APIs	None.
UI screens	None — but with D2, decide where AI is visible vs invisible.
Components	None.
AI models	None trained. Evaluate pre-trained options: Whisper variants, IndicWhisper, IndicBERT/MuRIL, MobileViT, EfficientNetV2, open DTC datasets, Kaggle vehicle damage datasets.
Libraries	Evaluation: PyTorch, scikit-learn, LightGBM, HuggingFace, ONNX Runtime, TFLite, MLflow, Feast.
Risks	The dominant risk of this entire vertical: no proprietary training data at project start. Mitigation: (a) every model has a rules-based v1 that ships regardless; (b) instrument for data collection from P5 onward so real data accumulates from the first booking; (c) use public/synthetic data plus transfer learning for v1 models; (d) be explicit in the pitch about which models are trained on real vs. public data — never claim otherwise.
Security	Establish the data governance position now: what can be used for training, under what consent, with what retention. This is a DPDP obligation, not a nice-to-have.
Testing	Define evaluation protocol per system: metric, held-out strategy, and — critically — the baseline each must beat.

Documentation	Data audit, feasibility assessment with the <i>trainable now / later / rules-only</i> classification, labelling plan, evaluation protocol.
Git branches	<code>docs/p00-d3-ai-feasibility</code> , <code>spike/p00-d3-crash-detection</code> , <code>spike/p00-d3-indic-asr</code>
Time	4 ideal days
Demo	Present the honest classification. Which of the 9 are real in v1, which are rules wearing an AI label, and what data would change that. This candour is what makes the rest credible.

Phase 1 — Planning · S0 W2 · 3 days

Goal	An AI backlog sequenced by value-per-effort, with the data pipeline that makes v2 possible built into v1.
Deliverables	AI epic breakdown (~35 stories), estimates, model development lifecycle definition, MLOps toolchain decision, data collection instrumentation plan.
Folder structure	<code>ml/ ├── data/{raw,processed,external,synthetic}/ ├── notebooks/ ├── src/{features,models,training,evaluation,serving}/ ├── configs/ ├── tests/ └── models/ (registry pointers, not weights)</code>
Database tables	Feature store schema handed to D1: <code>ml.feature_vehicle_daily</code> , <code>ml.feature_user_behaviour</code> , <code>ml.feature_mechanic_daily</code> , <code>ml.model_predictions</code> , <code>ml.training_snapshots</code> , <code>ml.labels</code> .
APIs	AI Gateway contract agreed with D1 (envelope, confidence, model version, fallback flag, timeout budget per capability).
AI models	None.
Libraries	Locked: PyTorch 2.x, scikit-learn, LightGBM, HuggingFace Transformers, ONNX Runtime, TFLite, MLflow, DVC, FastAPI, Feast (or a lightweight custom feature store — decide in P2).
Risks	MLOps overhead consuming the time meant for modelling → deliberately minimal toolchain: MLflow for tracking/registry, DVC for data versioning, plain Docker for serving. No Kubeflow, no Airflow, no feature-store platform. One person cannot operate that. Record as ADR-020.
Security	Data collection instrumentation designed with consent gating from the start — every training-eligible event carries the consent state at collection time.
Testing	Define the model test suite shape: data validation, training reproducibility, eval regression, inference contract.
Documentation	ML lifecycle doc, ADR-020 (minimal MLOps), data collection plan, model card template.
Git branches	<code>chore/p01-d3-ml-scaffold</code> , <code>docs/p01-d3-ml-lifecycle</code>
Time	3 ideal days
Demo	Run an end-to-end toy pipeline: data → train → MLflow registry → ONNX export → FastAPI serve → prediction. The plumbing works before any real model exists.

Phase 2 — System Architecture · S1 W3 · 3 days (support role)

Goal	Define the AI Gateway so models can be swapped, versioned, and rolled back without touching product code.
Deliverables	AI Gateway design, model serving architecture, A/B and shadow-mode design, fallback registry contract with D1.
Folder structure	<code>apps/ai-gateway/src/{routing, capabilities, models, guardrails, telemetry}/</code>
APIs	<code>POST /ai/v1/diagnose · /ai/v1/diagnose-image · /ai/v1/voice/transcribe · /ai/v1/voice/intent · /ai/v1/predict-maintenance · /ai/v1/match · /ai/v1/fraud-score · /ai/v1/forecast</code> — all sharing one envelope.
Components	Capability router (capability name → model version, resolved at request time from the registry, so a rollback is a config change not a deploy). Shadow mode (new model runs alongside the old, results logged and compared, never served). Guardrails layer: PII scrubbing on input, schema validation on output, confidence thresholding, timeout enforcement.
AI models	None yet.
Libraries	<code>fastapi</code> , <code>pydantic</code> , <code>onnxruntime</code> , <code>mlflow</code> , <code>prometheus-client</code> .
Risks	Model rollback being slow during an incident → the registry-driven router makes rollback a single config write, provable in the P15 drill.
Security	PII scrubbing before any model sees input (names, phone numbers, plate numbers redacted from free text). Model outputs validated against a schema — a model must never be able to inject arbitrary content downstream. Prompt-injection defence on any LLM path: user text is data, never instructions, and the LLM's role is limited to explaining a structured result it did not choose.
Testing	Gateway contract tests; guardrail tests (malformed output, PII leakage, timeout, injection attempt).
Documentation	AI Gateway architecture, ADR-021 (registry-driven routing), guardrail specification.
Git branches	<code>docs/p02-d3-ai-gateway-design</code>
Time	3 ideal days
Demo	Change a model version in the registry; the next request uses the new model with no deploy. Then roll it back in 5 seconds.

Phase 3 — Database Design · S1 W4 · 2 days (support role)

Goal	Make sure the OLTP schema will actually yield trainable data.
Deliverables	Feature store schema, label capture design, training snapshot strategy.
Database tables	<code>ml.feature_vehicle_daily</code> , <code>ml.feature_user_behaviour</code> , <code>ml.feature_mechanic_daily</code> , <code>ml.model_predictions</code> , <code>ml.training_snapshots</code> , <code>ml.labels</code> , <code>ml.feedback</code>
Components	Label capture is the highest-leverage design decision in this phase. Every booking outcome, every mechanic's actual diagnosis vs. our predicted one, every user "that wasn't right" — these are labels. If they aren't captured from day one, there is no v2 model. Design the capture now, even though it won't be used for months.
Risks	Training/serving skew — features computed differently in training and production → single feature definition module (<code>ml/src/features/</code>) imported by both the batch job and the serving path. Non-negotiable.

Security	Feature store contains no direct identifiers; entities are referenced by salted hash. Consent state stored alongside each row so training sets can be filtered by lawful basis.
Testing	Feature parity test: the same input through the batch path and the serving path must produce byte-identical features.
Documentation	Feature dictionary, label taxonomy, training snapshot policy.
Git branches	<code>feat/p03-d3-feature-store-schema</code> , <code>docs/p03-d3-label-taxonomy</code>
Time	2 ideal days
Demo	Show the feature parity test failing when someone changes a feature in only one place.

Phase 4 — Authentication · S2 W5 · 1 day (support role)

Goal	Ensure auth emits the signals fraud detection will need.
Deliverables	Auth event requirements handed to D1; device fingerprinting design.
Components	Signals: <code>auth.otp_failed</code> , <code>auth.device_new</code> , <code>auth.refresh_reuse</code> , <code>auth.velocity_anomaly</code> , <code>auth.msisdn_changed</code> , device fingerprint hash.
Security	Device fingerprinting must be privacy-respecting: a salted hash of stable, non-identifying attributes. No advertising ID, no cross-app tracking. Documented in the privacy policy.
Documentation	Fraud signal catalogue.
Git branches	<code>docs/p04-d3-fraud-signals</code>
Time	1 ideal day
Demo	N/A — reviewed in D1's auth demo.

Phase 5 — Backend APIs · S2–S3 · 4 days (support role)

Goal	Build the AI Gateway shell with rules-based capabilities so D1 and D2 can integrate immediately, months before the models exist.
Deliverables	AI Gateway service running, all 8 capability endpoints live returning rules-based results , prediction logging, shadow-mode infrastructure.
Expected output	Every AI endpoint the product needs exists and returns a correct, useful answer on day one — via rules. Models replace the rules behind the same contract, one at a time, with no product change.
Folder structure	<code>apps/ai-gateway/src/capabilities/{diagnose,image,voice,maintenance,match,fraud,forecast}/</code> each with <code>rules.py</code> and (later) <code>model.py</code> .
APIs	All 8 capabilities live.

Components	Rules implementations: DTC lookup + symptom keyword matching for diagnosis; distance+rating+availability scoring for matching; threshold rules for fraud; manufacturer intervals for maintenance. These are the permanent fallbacks, not throwaway stubs.
AI models	None yet — but this phase is what makes the later phases low-risk.
Libraries	<code>fastapi</code> , <code>uvicorn</code> , <code>pydantic</code> , <code>rapidfuzz</code> (symptom matching), <code>pandas</code> .
Risks	Rules being treated as temporary and left untested → they are the production fallback for the life of the product; they get the same test coverage as the models.
Security	Guardrails active from day one, before any model exists.
Testing	Capability contract tests; rules correctness tests reviewed by 2 domain-expert mechanics from P0.
Documentation	Rules documentation per capability — this doubles as the specification each model must beat.
Git branches	<code>feat/p05-d3-ai-gateway</code> , <code>feat/p05-d3-rules-capabilities</code> , <code>feat/p05-d3-prediction-logging</code>
Time	4 ideal days
Demo	Every AI feature in the app works end-to-end. None of it is a model yet. Nobody can tell from the product — and that's the point: the models are now a pure upgrade, not a dependency.

Phase 6 — Frontend · S3–S4 · 1 day (support role)

Goal	Make sure AI is presented honestly.
Deliverables	Confidence band definitions and copy, AI presentation guidance for D2, feedback capture schema.
Components	Confidence bands: ≥ 0.85 "high", $0.6\text{--}0.85$ "medium", < 0.6 "low → defer to mechanic". Each maps to specific plain-language copy in 8 languages, not a percentage.
Risks	Users over-trusting output → the copy for the low band explicitly says a mechanic will confirm, and the conservative rules verdict is always shown alongside on safety questions.
Documentation	Confidence band spec, AI copy guidelines.
Git branches	<code>docs/p06-d3-confidence-bands</code>
Time	1 ideal day
Demo	Reviewed in D2's demo.

Phase 7 — AI Development · S4 W9 – S5 W12 · 18 days · You own this phase

This is the core of your work. Broken into per-model sub-blocks.

7.1 Vehicle Diagnosis AI · 4 days

Goal	Beat the rules baseline (top-3 accuracy 0.62) on symptom → cause.
-------------	---

Inputs	Symptom text (8 languages), vehicle make/model/year/fuel, DTC codes if present, odometer, service history, recent telemetry.
Outputs	Ranked causes with probability, severity (1–5), drivability verdict, predicted parts, natural-language explanation.
Training data	Curated fault knowledge base (~4k fault patterns, built from service manuals, mechanic interviews from P0, and public DTC databases), plus synthetic symptom paraphrases generated across 8 languages and validated by native speakers.
Algorithm	Hybrid retrieval + classification: BM25 + dense retrieval (MuRIL embeddings) over the KB → LightGBM re-ranker on retrieved candidates + structured features → an LLM writes the explanation from the selected structured result only . The LLM never chooses the diagnosis.
Deployment	Server (retriever + ranker + LLM explanation); distilled classifier-only variant on-device.
Metrics	Top-1 acc, top-3 acc, severity MAE, false-"safe-to-drive" rate (must be 0) , latency p95.
Risks	Hallucinated explanations → the LLM is constrained to explain a structured result it did not pick, with the cause, severity, and parts injected as fixed facts; output is validated against those facts before it is returned.
Testing	Held-out set of 400 real cases labelled by 3 mechanics with inter-annotator agreement reported. Adversarial test: 50 prompt-injection attempts in the symptom field.
Branches	<code>model/p07-d3-diagnosis-retriever</code> , <code>model/p07-d3-diagnosis-ranker</code> , <code>model/p07-d3-diagnosis-explainer</code>

7.2 Image Diagnosis AI · 3 days

Goal	Top-3 accuracy ≥ 0.85 on damage/part identification.
Inputs	1–5 photos + vehicle context. Outputs
Training data	Public vehicle damage datasets + ~3k images collected and labelled during P0–P6 dogfooding + heavy augmentation (rain, night, motion blur, low light — the actual conditions).
Algorithm	MobileViT-S or EfficientNetV2-S backbone, multi-label part head + ordinal severity head, transfer learning from ImageNet then domain fine-tuning.
Deployment	Server FP16; INT8 quantized on-device variant (~6 MB).
Metrics	Top-1/top-3 accuracy, severity ordinal MAE, per-class recall (watch for the rare-but-serious classes), calibration (ECE).
Risks	Bias toward well-photographed cars in daylight → the augmentation strategy and a stratified eval set covering night, rain, and low-end camera quality. Report metrics per stratum, not just overall — an aggregate number hides the failure that matters.
Testing	Stratified eval by lighting, weather, camera quality, vehicle class. Calibration plot required in the model card.
Branches	<code>model/p07-d3-image-diagnosis</code> , <code>model/p07-d3-image-quantize</code>

7.3 Voice Assistant · 3 days

Goal	Intent accuracy ≥ 0.85 across 8 languages including code-switched speech.
Inputs	Speech audio. Outputs
Training data	IndicWhisper / Whisper-small fine-tuned on public Indian-accented corpora (Shrutilipi, IndicSUPERB) + ~500 in-domain utterances recorded across the team and pilot users. Intent classifier trained on templated + paraphrased in-domain utterances.

Algorithm	ASR (Whisper-small, fine-tuned) → intent classifier (MuRIL + linear head) → slot filler (token classification) → TTS (Indic TTS). On-device: keyword spotting for "help"/"SOS" in 8 languages only (~2 MB).
Deployment	Server for full pipeline; on-device keyword spotting always-on for emergency.
Metrics	WER per language, intent accuracy, slot F1, end-to-end latency, keyword-spotting FPR (critical — a false "SOS" wake is bad).
Risks	Code-switching (Hinglish) degrading both ASR and intent → include code-switched examples in training and evaluate on a dedicated code-switched set; report it separately.
Testing	Per-language WER; noisy-environment tests (highway noise at 70 dB, which is the actual use case); keyword FPR over 100 hours of ambient audio.
Branches	<code>model/p07-d3-asr-finetune</code> , <code>model/p07-d3-intent-nlu</code> , <code>model/p07-d3-keyword-spotting</code>

7.4 Predictive Maintenance · 3 days

Goal	recall@30-days $\geq$ 0.60 on component failure — roughly double the fixed-interval baseline.
Inputs	Telemetry time series (engine temp, RPM patterns, battery voltage, fuel trim, EV cell data), service history, odometer, vehicle model, usage pattern (city/highway/load).
Outputs	Per-component failure probability at 30/90 days, recommended service date, reasoning.
Training data	Synthetic + public degradation datasets (NASA C-MAPSS style adapted), plus real telemetry accumulating from P5. Honest position: v1 is largely synthetic-trained; this is stated in the model card and the pitch.
Algorithm	Survival analysis (Cox PH baseline, DeepSurv if data supports it) per component + LightGBM classifier on engineered rolling features.
Deployment	Server, nightly batch, results cached.
Metrics	Concordance index, recall@30d, precision@30d, calibration, lead time distribution.
Risks	Alarm fatigue from false positives → precision floor enforced; a prediction is surfaced only above a threshold tuned for precision, not recall, because a user who ignores three false warnings will ignore the real one.
Testing	Time-based split (never random — that leaks the future). Per-vehicle-class evaluation.
Branches	<code>model/p07-d3-predictive-maintenance</code>

7.5 Smart Mechanic Matching · 2 days

Goal	acceptance@1 $\geq$ 0.65 vs the 0.41 distance+rating baseline.
Inputs	Request (service type, vehicle, location, urgency, time), mechanic features (skills, ratings, inventory, historical acceptance by context, current load, distance, predicted travel time), plus contextual features (traffic, weather, time of day).
Outputs	Ranked mechanics with acceptance probability and predicted ETA.
Training data	Historical dispatch outcomes from P5 onward. Cold start: rules-based, transitioning to ML once ~5k offers are accumulated.
Algorithm	LambdaMART (LightGBM ranker) for ranking + separate gradient-boosted ETA regressor.
Deployment	Server, hard 300 ms budget, fallback to rules on timeout.

Metrics	acceptance@1, NDCG@5, ETA MAE, fairness: Gini coefficient of job distribution across mechanics.
Risks	The model concentrating all jobs on a few high-performing mechanics, starving new entrants and destroying supply → explicit exploration term (epsilon-greedy) plus a fairness constraint monitored as a first-class metric. A matching model that optimizes only for acceptance will kill the marketplace.
Testing	Offline replay on historical data; online A/B via shadow mode before serving; fairness metric tracked per week.
Branches	<code>model/p07-d3-mechanic-ranker</code> , <code>model/p07-d3-eta-regressor</code>

7.6 Crash Detection · 3 days

Goal	recall ≥ 0.95 , FPR ≤ 1 per 1,000 driving hours, on-device, <120 ms. The highest-stakes model in the project.
Inputs	100 Hz accelerometer, gyroscope, GPS speed, barometer — 5-second sliding windows.
Outputs	Crash probability + severity estimate.
Training data	Public crash sensor datasets, controlled drop/impact tests, simulated crash signatures, plus a large corpus of <i>negative</i> data — normal driving on Indian roads, which contains potholes, speed breakers, and sudden braking that all look somewhat like crashes. The negative data is harder to get right than the positive data and matters more.
Algorithm	1D CNN feature extractor + GRU temporal model, INT8 quantized, ~1.5 MB.
Deployment	On-device only. Never server-side — it must work with no connectivity, which is exactly when a highway crash happens.
Metrics	Recall, FPR per driving hour, severity MAE, inference latency, battery cost per hour.
Risks	This is R3, a critical project risk. A false positive dispatches real emergency services. Mitigations, layered: (a) the model raises a <i>signal</i> , never an incident; (b) 30 s user-cancellable countdown; (c) call-back verification before any 112 handoff; (d) the model cannot auto-dispatch — this is enforced in D1's code, not just policy; (e) speed-breaker and pothole signatures explicitly in the negative training set; (f) FPR measured over real driving hours during dogfooding, not just on a test set.
Security	Sensor data stays on-device unless a crash is confirmed. No continuous sensor upload — that would be both a privacy violation and a battery disaster.
Testing	Controlled tests: 50 speed breakers, 30 potholes, 20 hard-braking events, 10 phone drops, 5 phone-thrown-on-seat events — all must produce zero crash signals. Recall measured on labelled crash traces. 200 hours of real driving during dogfooding with FPR measured.
Branches	<code>model/p07-d3-crash-detection</code> , <code>model/p07-d3-crash-quantize</code>

7.7 Fraud Detection · 2 days

Goal	Precision ≥ 0.70 at recall 0.50, versus the 0.34-precision rules baseline.
Inputs	Booking graph (user↔mechanic↔device↔payment relationships), payment patterns, GPS trace plausibility, review patterns, auth signals from P4, timing patterns.
Outputs	Risk score 0–100 + reason codes.
Training data	Confirmed fraud cases (few, initially) + synthetic fraud injection + unsupervised anomaly detection to bootstrap labels for human review.
Algorithm	Graph feature extraction (community detection for collusion rings) + isolation forest for unsupervised anomaly + LightGBM supervised model as confirmed cases accumulate.

Deployment	Server, async — never blocking a booking.
Metrics	Precision@k, recall on confirmed cases, false-accusation rate (the metric that matters most).
Risks	Falsely flagging a legitimate mechanic, destroying their livelihood → the model never auto-suspends. It queues for human review with reason codes, and the mechanic is told the outcome. Reason codes are mandatory: an unexplainable fraud score is unusable.
Security	Fraud scores are internal only, never exposed to users or mechanics. Access to the fraud queue is role-restricted and audited.
Testing	Synthetic fraud injection (collusion rings, GPS spoofing, review farming); false-positive review with the ops team.
Branches	<code>model/p07-d3-fraud-graph-features</code> , <code>model/p07-d3-fraud-model</code>

7.8 Model serving & registry · 2 days

Wire all trained models into the AI Gateway behind the existing rules contracts, with shadow mode, A/B, and one-config rollback. Prometheus metrics per model: latency, throughput, confidence distribution, fallback rate, and drift indicators.

Phase 7 summary

Deliverables	7 trained models, model registry populated, 7 model cards, eval harness, shadow-mode results, on-device bundle started.
Libraries	<code>torch</code> , <code>transformers</code> , <code>lightgbm</code> , <code>scikit-learn</code> , <code>lifelines</code> , <code>onnxruntime</code> , <code>optimum</code> , <code>mlflow</code> , <code>dvc</code> , <code>librosa</code> , <code>alumentations</code> , <code>networkx</code> .
Security	No PII in any training set without a documented lawful basis. Model weights are artefacts in the registry, not in git. Every model card includes a bias evaluation across vehicle class, language, and region.
Testing	Every model must beat its documented baseline on a held-out set. Every model has a calibration report. Every model has an adversarial/robustness test appropriate to its input type.
Documentation	7 model cards, evaluation report, training reproducibility guide (<code>dvc repro</code> reproduces any model from its commit).
Time	18 ideal days
Demo	For each model: the baseline, the model, the delta, the failure modes, and a live inference. Then unplug the model and show the rules fallback keeping the product working.

Phase 8 — Offline Engine (on-device AI) · S5 W12 · 4 days

Goal	Fit useful intelligence into 15 MB and 120 ms on a 2 GB phone.
Deliverables	On-device model bundle (crash detection, diagnosis classifier, keyword spotting), TFLite/ONNX Runtime Mobile integration, on-device inference benchmarks, model update channel.
Folder structure	<code>ml/src/serving/ondevice/</code> , <code>packages/ml-ondevice/</code> (RN native module)
Components	Bundle budget: crash detection 1.5 MB · diagnosis classifier 8 MB · keyword spotting 2 MB · DTC lookup table 3 MB = 14.5 MB . Models delivered out-of-band (downloaded on first run, not in the APK) so the APK stays small and models can update independently of app releases.

AI models	Quantized/distilled variants of #1, #3 (keyword only), #6.
Libraries	<code>onnxruntime-mobile</code> , <code>tensorflow-lite</code> , <code>react-native-fast-tflite</code> , <code>optimum</code> (quantization), <code>torch.ao.quantization</code> .
Risks	(a) Quantization degrading accuracy below usefulness → measure post-quantization accuracy explicitly; if the drop exceeds 3 points, distil instead of quantizing further. (b) Model download failing on poor connectivity → the app is fully functional without on-device models (falls back to server, then to rules); the models are an enhancement, never a requirement.
Security	Models are signed and verified before loading — an unverified model file is a code execution vector. Model files stored in app-private storage.
Testing	Benchmark on 5 real low-end devices: latency, memory peak, battery cost, thermal behaviour under sustained inference. Accuracy comparison quantized vs full precision.
Documentation	On-device deployment guide, quantization report (accuracy before/after per model), device benchmark table.
Git branches	<code>feat/p08-d3-ondevice-bundle</code> , <code>feat/p08-d3-tflite-native-module</code> , <code>feat/p08-d3-model-update-channel</code>
Time	4 ideal days
Demo	Airplane mode on a ₹8,000 phone. Describe a symptom, get a diagnosis in under 200 ms. Simulate a crash, get detection on-device. Nothing touches the network.

Phase 9 — Maps · S6 W13 · 1 day (support role)

Goal	Improve ETA beyond what the routing engine gives.
Deliverables	ETA correction model — Valhalla's ETA adjusted by learned residuals from historical actuals.
Components	Features: route distance, time of day, day of week, weather, historical corridor speed, mechanic's vehicle type, monsoon flag. Target: actual – predicted travel time.
Metrics	ETA MAE improvement vs raw routing engine; target ≥25% reduction.
Risks	Insufficient historical data early → ship the raw routing ETA with an honest range ("15–25 min") rather than a false precision ("18 min") until the residual model has data.
Testing	Time-based split; per-corridor evaluation.
Branches	<code>model/p09-d3-eta-correction</code>
Time	1 ideal day

Phase 10 — Vehicle Diagnostics · S6 W14 · 5 days · You own this phase (with D2)

Goal	Extend diagnosis to the vehicles that don't have OBD — which in India is most of them.
Deliverables	Two-wheeler / auto / tractor rule packs, EV diagnostic module, sound-based diagnosis spike, DTC → plain language in 8 languages, integrated diagnosis pipeline.

Components	Per-vehicle-class rule packs built with the mechanics interviewed in P0 — this is knowledge engineering, not machine learning, and for two-wheelers it's the right tool. EV module: SoC anomaly, cell imbalance, thermal events, charging faults, range estimation. Sound diagnosis: timeboxed 1-day spike (mel-spectrogram + small CNN on engine/brake noise) — ships in v1.5 only if the spike succeeds.
AI models	Diagnosis model extended with vehicle-class conditioning; sound classifier if the spike passes.
Metrics	Per-vehicle-class top-3 accuracy; mechanic satisfaction score — 5 mechanics rate 50 outputs each, which is a more honest measure than accuracy on a curated set.
Risks	Two-wheeler diagnosis being weak because there's no telemetry → lean entirely on structured symptom questioning plus photo/sound, and be honest in the UI that it's a triage, not a diagnosis.
Security	Diagnostic photos scrubbed of EXIF location before storage or training use.
Testing	Per-class held-out sets; the mechanic rating exercise; the zero-false-"safe" assertion across all classes.
Documentation	Rule pack documentation per vehicle class, EV diagnostic spec, sound diagnosis spike report.
Git branches	<code>model/p10-d3-vehicle-class-rules</code> , <code>model/p10-d3-ev-diagnostics</code> , <code>spike/p10-d3-sound-diagnosis</code>
Time	5 ideal days
Demo	Diagnose a scooter that won't start, using only structured questions and a photo. Then have a real mechanic grade the answer live.

Phase 11 — Emergency Engine · S7 W15 · 3 days (support role)

Goal	Get crash detection to production quality and validate severity estimation.
Deliverables	Crash detection deployed on-device, severity estimation model, false-positive dataset from dogfooding, tuned thresholds.
Components	Threshold tuning against the dogfooding FPR data. Severity estimation feeding the escalation ladder (severity ≥ HIGH escalates faster). Two-signal corroboration logic: sensor signal + no user response + abnormal post-event motion pattern.
Risks	R3 again, now with real stakes. Final gate before production: FPR measured over ≥200 real driving hours across ≥5 devices and ≥3 road types must be ≤ 1 per 1,000 hours. If it isn't, crash detection ships disabled by default and manual SOS carries the feature. That decision is pre-committed here so it isn't a judgement call under launch pressure.
Testing	The full controlled test battery from 7.6, re-run on the production build. Dogfooding FPR report.
Documentation	Crash detection model card updated with production FPR; the ship/no-ship decision recorded.
Git branches	<code>model/p11-d3-crash-threshold-tuning</code> , <code>model/p11-d3-severity-estimation</code>
Time	3 ideal days
Demo	Present the real dogfooding FPR number. Drive over a speed breaker on camera. Nothing happens. That's the demo.

Phase 12 — Government Dashboard · S7 W16 · 3 days (support role)

Goal	Forecasting and blackspot analysis that a transport department would act on.
Deliverables	Blackspot detection model, spatiotemporal incident forecasting, corridor risk scoring, demand prediction.
Components	Blackspot detection: spatial clustering (DBSCAN on incident locations) weighted by severity, with statistical significance testing so a cluster isn't just traffic volume. Forecasting: temporal fusion transformer or a well-tuned gradient-boosted model on spatiotemporal features (historical incidents, weather, festival calendar, road geometry, traffic volume).
Metrics	Blackspot precision against known official blackspot lists (a genuinely strong validation signal — if we rediscover the blackspots the state already knows about, plus some it doesn't, that's credible). Forecast MAE vs a historical-average baseline.
Risks	Forecasts driving real infrastructure spending → outputs carry confidence intervals and an explicit methodology note; we present ranges, never point estimates, to a government user.
Security	All outputs are aggregate and k-anonymized before they reach the gov service. The model never sees, and never emits, individual identifiers.
Testing	Validation against published official blackspot data for one state. Time-based forecast backtesting.
Documentation	Blackspot methodology, forecasting model card, a limitations section written for a non-technical government reader.
Git branches	<code>model/p12-d3-blackspot-detection</code> , <code>model/p12-d3-incident-forecasting</code>
Time	3 ideal days
Demo	Run blackspot detection on a state's historical data; overlay the official blackspot list; show the overlap and the new candidates.

Phase 13 — Analytics · S8 W17 · 5 days · You own this phase

Goal	An analytics layer where every number has exactly one definition and nobody argues about which dashboard is right.
Deliverables	Event taxonomy, ClickHouse schema, 4 dashboard suites (with D2), metric dictionary, anomaly alerting, cohort/funnel analysis.
Folder structure	<code>ml/src/analytics/</code> , <code>infra/clickhouse/</code>
Database tables	ClickHouse: <code>events</code> , <code>bookings_fact</code> , <code>incidents_fact</code> , <code>mechanic_daily</code> , <code>vehicle_daily</code> , <code>funnel_steps</code> , <code>cohorts</code>
Components	Metric dictionary is the deliverable that matters most here. One definition per metric, versioned, referenced by every dashboard. Anomaly detection on business metrics (seasonal decomposition + threshold on residuals) so a 40% drop in bookings pages someone.
Metrics tracked	Demand (requests/hour by geo), supply (active mechanics), match rate, time-to-match, cancellation rate and reasons, completion rate, NPS, revenue, take rate, CAC, LTV, cohort retention, SLO attainment, incident response times, mechanic utilization and earnings distribution.

Risks	Metric definitions drifting between dashboards → the dictionary is enforced by a shared SQL macro layer; dashboards cannot define their own aggregations ad hoc.
Security	Zero PII in ClickHouse, enforced by a CI scan of the schema. Analytics access role-restricted.
Testing	Metric correctness tests (compute each metric two ways and assert agreement). Pipeline lag test. Anomaly detection tested against injected synthetic anomalies.
Documentation	Event taxonomy, metric dictionary, dashboard guide, anomaly alerting runbook.
Git branches	<code>feat/p13-d3-event-taxonomy</code> , <code>feat/p13-d3-clickhouse-schema</code> , <code>feat/p13-d3-metric-dictionary</code> , <code>feat/p13-d3-anomaly-alerting</code>
Time	5 ideal days
Demo	Inject a synthetic 40% booking drop. The anomaly alert fires within 10 minutes with the affected segment identified.

Phase 14 — Testing · S8 W18 · 4 days

Goal	Prove the models are correct, fair, robust, and safe to fail.
Deliverables	Model test suite in CI, drift detection, bias audit across all models, robustness testing, adversarial testing, model card finalization.
Components	Data validation (Great Expectations) on every training input. Eval regression tests — a model version cannot be promoted if it regresses any tracked metric. Drift detection (PSI/KL on feature distributions) alerting in production. Bias audit: performance by vehicle class, language, region, and device tier for every model; a model that works well only for car owners in metros is not shippable.
Risks	Silent model degradation in production → drift alerting plus weekly automated eval on a held-out set that refreshes with new labelled data.
Security	Adversarial testing: prompt injection on the LLM path, adversarial images on the vision model, sensor spoofing on crash detection, GPS spoofing on fraud detection.
Testing	Full suite in CI. Fairness metrics reported per model.
Documentation	All 9 model cards finalized with bias audit results; AI risk assessment.
Git branches	<code>test/p14-d3-model-test-suite</code> , <code>test/p14-d3-bias-audit</code> , <code>test/p14-d3-adversarial</code>
Time	4 ideal days
Demo	Present the bias audit. Show where each model is weakest and what we're doing about it. A team that knows its models' weaknesses is more credible than one claiming they have none.

Phase 15 — Deployment · S9 W19 · 3 days

Goal	Models deployable and rollback-able without drama.
-------------	--

Deliverables	Model serving in production (Triton or FastAPI+ONNX), GPU/CPU sizing decision, model CI/CD, monitoring, on-device model distribution.
Components	Serving on CPU with ONNX Runtime where possible (dramatically cheaper than GPU and sufficient for our latency budgets except the LLM path). Model promotion pipeline: registry → shadow → canary 5% → 100%, with automatic rollback on a metric regression.
Risks	GPU cost → measured decision: only the LLM explanation path needs GPU, and it's off the critical path with a template-based fallback. Everything else is CPU.
Security	Model artefacts signed; serving containers scanned; no model loaded from an unverified source.
Testing	Rollback drill: promote a deliberately bad model, watch the automatic rollback trigger.
Documentation	Model deployment runbook, serving architecture, cost model per 1M inferences.
Git branches	<code>feat/p15-d3-model-serving</code> , <code>feat/p15-d3-model-cicd</code>
Time	3 ideal days
Demo	Promote a bad model on purpose. Watch it get rolled back automatically in under 2 minutes.

Phase 16 — Optimization · S9 W20 · 3 days

Goal	Cheaper and faster inference without losing accuracy.
Deliverables	Inference optimization, batching, caching, distillation, cost-per-inference model.
Components	Dynamic batching on server models; result caching for deterministic inputs (same DTC + same vehicle model = same diagnosis); distillation of the LLM explanation path into a template + small model for the 80% of common cases; INT8 across all server models where accuracy permits.
Metrics	Latency p95 per capability, cost per 1M inferences, accuracy delta from each optimization (must be reported, never assumed to be zero).
Risks	Optimization silently degrading accuracy → every optimization PR includes a before/after eval on the held-out set. A PR that doesn't report it is rejected.
Documentation	Optimization report with accuracy/latency/cost tradeoffs made explicit.
Git branches	<code>perf/p16-d3-inference-optimization</code> , <code>perf/p16-d3-llm-distillation</code>
Time	3 ideal days
Demo	Cost per 1M inferences, before and after. And the accuracy table showing what it cost us.

Phase 17 — Future Roadmap · S10 · 3 days

Goal	Define the AI roadmap that the data we're now collecting makes possible.
Deliverables	AI roadmap (18 months), data strategy, model improvement plan, research agenda.

Components	The core argument: v1 models are largely bootstrapped from public and synthetic data. From launch, real labelled data accumulates from every booking (predicted vs actual diagnosis), every mechanic correction, and every user feedback tap. The roadmap is a data-flywheel plan: which model improves first, at what data volume, and what that improvement is worth. Research agenda: multimodal diagnosis (photo + sound + telemetry jointly), federated learning for on-device personalization without centralizing data, road condition mapping from aggregated accelerometer traces, V2X readiness.
Documentation	AI roadmap with data-volume thresholds per model, data strategy, research agenda, honest v1 capability statement for the pitch deck.
Git branches	<code>docs/p17-d3-ai-roadmap</code>
Time	3 ideal days
Demo	The flywheel: here is what each model does today, here is the data volume at which it gets meaningfully better, and here is when we expect to reach it.

D3 Effort Summary

Phase	Days	Phase	Days
P0 Research	4	P9 ETA model	1
P1 Planning	3	P10 Diagnostics	5
P2 AI Gateway design	3	P11 Emergency	3
P3 Feature store	2	P12 Gov forecasting	3
P4 Fraud signals	1	P13 Analytics	5
P5 Gateway + rules	4	P14 Testing	4
P6 AI UI support	1	P15 Deployment	3
P7 AI Development	18	P16 Optimization	3
P8 On-device AI	4	P17 Roadmap	3
		Total	70 ideal days

The lower total is deliberate: model work is research-shaped and the estimates are less reliable than engineering estimates. The extra slack absorbs the experiments that don't work — and some won't.

ROADASSIST BHARAT

D4 - DevOps, QA & Security Lead

Individual Roadmap

| *One Platform. Every Vehicle. Every Phone. Every Road.*

Document docs/05-devops-qa-lead-roadmap.md

Version 1.0 — planning baseline

Team 4 engineers · 22 weeks · 11 sprints · 18 phases

Status Baseline (frozen at Sprint 0 exit)

D4 — DevOps, QA & Security Lead · Individual Roadmap

Owens: Kubernetes, Terraform, CI/CD, observability, all testing disciplines, security, load and chaos engineering, SMS/IVR/USSD gateway operations, compliance evidence. **Backs up:** D2 on portal builds. **Total allocation:** 22 weeks · ~176 ideal days.

Standing responsibilities: - CI is never red on `main` for more than 30 minutes. You own that number. - You are the only person who says "this is not ready to ship." Use it. - Every alert you create has a runbook before it is enabled. An alert without a runbook is noise that trains people to ignore alerts. - Security findings are not suggestions. High and critical block the release.

Phase 0 — Research · S0 W1 · 4 days

Goal	Establish the operating environment: what we can host, where, under what regulations, and with which telecom partners.
Deliverables	Cloud provider evaluation (MeitY-empanelled), data localization analysis, telecom gateway evaluation (SMS/IVR/USSD), compliance obligation register, cost model v0, USSD feasibility spike.
Expected output	A decision on where this runs and what it costs, plus a started DLT registration — because that timeline is the longest lead item in the project.
Folder structure	<code>docs/research/infra/</code> , <code>docs/compliance/</code>
Database tables	None.
APIs	None — but evaluate telecom gateway APIs (Gupshup, Exotel, Kaleyra, Twilio-India) for SMS, voice/IVR, and USSD.
UI screens	None.
Components	None.
AI models	None.
Libraries	Evaluation: Terraform vs Pulumi, EKS vs AKS vs self-managed, ArgoCD vs Flux, Prometheus stack vs managed.
Risks	R2, and the single longest lead time in the project: TRAI DLT registration for SMS templates takes weeks. Start it in week 1. Register more templates than you think you need — adding one later is another multi-week wait. Also register with two gateway vendors, not one.
Security	Compliance register: DPDP Act 2023 (consent, localization, breach notification, data principal rights), TRAI UCC/DLT, MeitY cloud empanelment, RBI PA/PG (avoid scope by using a PSP), IRDAI if we touch insurance, AIS-140 for telematics. Each obligation mapped to the phase where it's satisfied and the evidence that proves it.
Testing	Define the test strategy skeleton and the tooling decisions.
Documentation	Cloud evaluation, compliance obligation register, telecom gateway comparison, cost model v0, USSD spike report.
Git branches	<code>docs/p00-d4-infra-research</code> , <code>docs/p00-d4-compliance-register</code> , <code>spike/p00-d4-ussd-feasibility</code>
Time	4 ideal days

Demo	Present the compliance register and the DLT timeline. Make it visible that this is a hard dependency nobody can compress.
-------------	---

Phase 1 — Planning · S0 W2 · 4 days

Goal	Stand up the engineering platform on day one so nobody is ever blocked on tooling.
Deliverables	CI/CD skeleton running, repo protections configured, environment strategy, test strategy doc, security policy, DLT registration submitted.
Folder structure	<code>.github/workflows/</code> , <code>infra/{terraform,k8s,scripts}/</code> , <code>tests/{load,security,e2e}/</code>
APIs	None.
Components	GitHub Actions: lint, typecheck, test, build, scan — running on every PR from the first commit. Branch protection with required checks. Dependabot/Renovate. Secret scanning. CODEOWNERS mapping each area to its lead.
Libraries	<code>actions/*</code> , <code>semgrep</code> , <code>trivy</code> , <code>osv-scanner</code> , <code>gitleaks</code> , <code>commitlint</code> , <code>k6</code> , <code>playwright</code> , <code>detox</code> .
Risks	Slow CI destroying the team's velocity → target: PR feedback in under 8 minutes. Aggressive caching, parallel jobs, affected-only testing via Nx. Measure CI duration as a tracked metric from week 2.
Security	Security policy: secret handling, dependency policy, disclosure process, incident response outline. Pre-commit hooks for secret scanning — catching a leaked key before the push is much better than rotating it after.
Testing	Test strategy document: what we test at which layer, coverage targets, tooling, environments, and who owns what.
Documentation	Test strategy, security policy, environment strategy, CI/CD guide, contribution guide.
Git branches	<code>chore/p01-d4-ci-skeleton</code> , <code>chore/p01-d4-repo-protections</code> , <code>docs/p01-d4-test-strategy</code>
Time	4 ideal days
Demo	Open a PR with a lint error, a failing test, and a hardcoded secret. Watch CI block all three in under 8 minutes.

Environment strategy

Env	Purpose	Data	Deploy trigger	Who can access
<code>local</code>	Development	Seeded synthetic	Manual	Everyone
<code>preview</code>	Per-PR ephemeral	Seeded synthetic	PR open	Everyone
<code>dev</code>	Integration	Seeded synthetic	Merge to <code>main</code>	Everyone
<code>staging</code>	Pre-production, prod-shaped	Anonymized prod-like	Merge to <code>main</code>	Everyone
<code>prod</code>	Production	Real	Manual approval on a tagged release	D4 + on-call, audited

Phase 2 — System Architecture · S1 W3 · 4 days (co-owner of the threat model)

Goal	Ensure the architecture is operable and defensible, not just elegant.
Deliverables	STRIDE threat model, infrastructure architecture, network topology, observability strategy, SLO definitions with error budgets.
Folder structure	<code>infra/terraform/modules/</code> , <code>docs/security/threat-model.md</code>
Components	Network: private subnets for everything, one ingress, no service directly internet-reachable, mTLS between process groups, egress allowlisting. Observability strategy: OpenTelemetry everywhere, trace context propagated across every service boundary including into the AI gateway and the mobile client.
Risks	Architecture that's elegant but unoperable by one person → veto power on anything that adds operational surface without proportionate value. This is the phase to say no to Kafka Streams, service meshes, and three databases.
Security	Full STRIDE pass across every trust boundary with D1. Output: a control list mapped to the phase where each control is implemented and the test that proves it. Trust boundaries: internet↔gateway, gateway↔services, service↔service, service↔DB, platform↔AI, platform↔telecom, platform↔government, platform↔PSP, device↔platform.
Testing	Define the security testing plan: SAST, DAST, dependency, container, secrets, pentest scope and timing.
Documentation	Threat model, infra architecture, network topology, SLO document with error budgets, observability strategy.
Git branches	<code>docs/p02-d4-threat-model</code> , <code>docs/p02-d4-infra-architecture</code> , <code>docs/p02-d4-slos</code>
Time	4 ideal days
Demo	Walk the threat model. For each of the top 10 threats: the control, the phase it lands in, and the test that will prove it works.

SLOs and error budgets

Service	Availability SLO	Latency SLO	Error budget/28d
API Gateway	99.9%	p95 < 300 ms	40 min
Auth	99.95%	p95 < 200 ms	20 min
Booking	99.9%	p95 < 300 ms	40 min
Dispatch	99.9%	p95 < 500 ms	40 min
Emergency	99.99%	p95 < 2 s end-to-end	4 min
Maps	99.5%	p95 < 400 ms	3.4 h
AI Gateway	99.5%	p95 < 800 ms	3.4 h
Sync	99.9%	p95 < 8 s for 100 ops	40 min

Error budget policy: when a service burns 50% of its budget, feature work on that service stops and reliability work starts. This is automatic, not a discussion.

Phase 3 — Database Design · S1 W4 · 2 days (support role)

Goal	Make the database operable: backed up, restorable, monitored, and secured.
Deliverables	Database infrastructure (Terraform), backup and PITR configuration, connection pooling, monitoring, encryption at rest.
Components	Managed Postgres with PITR, automated daily snapshots, cross-region encrypted copies, PgBouncer in transaction mode, <code>pg_stat_statements</code> enabled, replication lag and slot monitoring.
Risks	A backup that has never been restored is not a backup → restore drill scheduled in P15, and a scripted restore-to-scratch runs monthly from S2 onward, not just once at the end.
Security	Encryption at rest with a customer-managed key. Least-privilege DB roles per module (D1 defines them, you enforce them in Terraform). No human has standing production DB write access — break-glass only, time-limited, audited.
Testing	Automated restore verification: restore the latest backup into a scratch instance nightly and assert row counts. A backup nobody verifies is a backup nobody has.
Documentation	Database operations runbook, backup/restore procedure, access policy.
Git branches	<code>feat/p03-d4-database-infra</code> , <code>feat/p03-d4-backup-verification</code>
Time	2 ideal days
Demo	Delete a table in a scratch environment. Restore it from PITR to 5 minutes before the deletion.

Phase 4 — Authentication · S2 W5 · 3 days (support role)

Goal	Secrets, keys, and the SMS path that OTP depends on.
Deliverables	Secrets management (Vault/KMS), JWT signing key management with rotation, SMS gateway integration live, rate limiting at the edge, WAF rules.
Components	Vault with dynamic DB credentials and automatic rotation. JWT signing keys with a rotation schedule and an overlap window so rotation never invalidates live tokens. Edge rate limiting at the gateway (before the application, so a flood never reaches it). WAF with OWASP core rules plus custom rules for our endpoints.
AI models	None.
Libraries	<code>vault</code> , <code>external-secrets-operator</code> , <code>kong</code> / <code>envoy</code> , <code>coraza</code> or a managed WAF.
Risks	(a) SMS gateway being a single point of failure for all authentication → two vendors integrated with automatic failover, and the failover tested, not just configured. (b) DLT template rejection blocking OTP → templates submitted in P0, with variants pre-approved.
Security	Keys never leave the KMS/HSM boundary. Rotation automated with an overlap window. Rate limits: per-IP, per-MSISDN, per-endpoint, with distinct budgets. Bot detection on OTP request. Geo-anomaly detection feeding D3's fraud signals.
Testing	Rate limit tests; SMS failover test (kill vendor 1, assert delivery via vendor 2); key rotation test with live tokens; WAF rule tests against an attack corpus.
Documentation	Secrets management runbook, key rotation procedure, SMS gateway runbook, WAF rule documentation.

Git branches	<code>feat/p04-d4-vault-secrets</code> , <code>feat/p04-d4-sms-gateway</code> , <code>feat/p04-d4-edge-rate-limiting</code>
Time	3 ideal days
Demo	Disable the primary SMS vendor mid-login. The OTP still arrives, via the secondary, within the same latency budget.

Phase 5 — Backend APIs · S2–S3 · 4 days (support role)

Goal	Every service is observable and testable from its first commit, not retrofitted at the end.
Deliverables	Observability stack running, per-service dashboards, structured logging standard, distributed tracing, integration test infrastructure, preview environments.
Components	Prometheus + Grafana + Loki + Tempo + Sentry. RED dashboards auto-generated per service from a template — a new service gets a dashboard by existing, not by someone remembering. Testcontainers-based integration test infrastructure in CI. Ephemeral preview environments per PR.
Risks	Observability added at the end, which never works because the instrumentation has to be in the code → the shared <code>libs/observability</code> from P2 makes it automatic, and a service without metrics fails a CI fitness check.
Security	Log scrubbing: PII, tokens, and payment data redacted at the logging library level, not by convention. Verified by a CI test that logs a fake token and asserts it's redacted.
Testing	Integration test infra; contract test running in CI both directions; load smoke test on every merge to <code>main</code> (small, fast — a canary for performance regressions, not a full load test).
Documentation	Observability guide, logging standard, dashboard catalogue.
Git branches	<code>feat/p05-d4-observability-stack</code> , <code>feat/p05-d4-preview-environments</code> , <code>feat/p05-d4-integration-test-infra</code>
Time	4 ideal days
Demo	Follow one request from the mobile app through the gateway, three services, the database, and the AI gateway — as a single distributed trace. Then show the log scrubbing test catching a leaked token.

Phase 6 — Frontend · S3–S4 · 3 days (support role)

Goal	Mobile and web build pipelines that don't consume D2's time.
Deliverables	Mobile CI (Android + iOS builds, signing, distribution), web build/deploy pipeline, visual regression infrastructure, device farm access.
Components	Fastlane in CI with signing keys from Vault. Internal-track distribution on every merge to <code>main</code> so the team dogfoods continuously from S4 — this is how real bugs get found. Chromatic for visual regression. Device farm (BrowserStack or a physical rack of the 6 most common Indian Android models).
Risks	iOS build infrastructure being painful and expensive → macOS runners are budgeted explicitly; if cost is prohibitive, Android leads and iOS follows by one sprint, decided now rather than discovered in P15.
Security	Signing keys in Vault, never on a developer machine. Release builds verified to have certificate pinning enabled and debug flags off — this is a checklist item that catches real mistakes.

Testing	Visual regression baseline; build reproducibility check.
Documentation	Mobile release runbook, signing key procedure.
Git branches	<code>feat/p06-d4-mobile-ci</code> , <code>feat/p06-d4-visual-regression</code>
Time	3 ideal days
Demo	Merge a PR; a signed build lands on everyone's phone via the internal track within 15 minutes.

Phase 7 — AI Development · S4–S5 · 3 days (support role)

Goal	ML infrastructure that one person can operate.
Deliverables	MLflow deployment, model artefact storage, training compute provisioning, model serving infrastructure, GPU cost controls.
Components	MLflow with S3-backed artefact store. Training on spot instances with checkpointing — cheap, and the checkpointing makes preemption a non-event. Model serving on CPU nodes with ONNX Runtime; a small GPU pool only for the LLM path, scale-to-zero when idle.
Risks	GPU cost running away → hard budget alerts, scale-to-zero, and spot-only for training. A forgotten running GPU instance is a genuinely common and expensive mistake.
Security	Model artefacts signed and verified at load. Training data access audited. Model serving containers scanned like any other.
Testing	Model serving load test; artefact integrity verification.
Documentation	ML infrastructure runbook, GPU cost policy.
Git branches	<code>feat/p07-d4-ml-infrastructure</code> , <code>feat/p07-d4-model-serving-infra</code>
Time	3 ideal days
Demo	Show the GPU pool scaling to zero and the resulting cost graph.

Phase 8 — Offline Engine · S5 W11–12 · 6 days · You own the telecom half

Goal	Make feature phones genuine first-class clients — SMS, IVR, and USSD, in production.
Deliverables	Telecom gateway service, SMS command protocol live, IVR menu tree in 8 languages, USSD session flow, network chaos testing infrastructure.
Folder structure	<code>apps/telecom-gateway/src/{sms,ivr,ussd,session,templates}/</code>
Database tables	<code>telecom_sessions</code> , <code>sms_inbound</code> , <code>sms_outbound</code> , <code>ivr_calls</code> , <code>dlt_templates</code>
APIs	Webhook receivers per vendor; internal <code>POST /internal/telecom/send</code> ; session state API.

UI screens	The IVR menu tree is a UI. Designed with the same rigour: max 4 options per level, max 3 levels deep, "press 0 for a human" always available, and every prompt under 8 seconds.
Components	SMS command grammar (compact, error-tolerant, works with typos — <code>HELP</code> <code>MADAD</code> <code>HELP CAR</code> all work). IVR with DTMF and speech input (using D3's ASR). USSD session state machine with a 180-second timeout. Vendor abstraction so a gateway swap is a config change.
AI models	D3's ASR and intent model on the IVR speech path, with DTMF always available as a fallback because ASR on a 2G voice call is genuinely hard.
Libraries	Vendor SDKs, <code>xstate</code> (USSD session state), <code>libphonenumber-js</code> .
Risks	(a) USSD requiring a telecom operator agreement per circle, which is slow and may not land → SMS and IVR are the guaranteed paths; USSD is a bonus, and the product does not depend on it. (b) DLT template rejection → templates submitted in P0 with variants.
Security	MSISDN-based auth is weaker than app auth — the SMS path is limited to a reduced capability set (request help, check status, cancel, SOS) and never exposes payment or profile changes. Inbound SMS is validated against the vendor's signature to prevent spoofing.
Testing	End-to-end tests against vendor sandboxes; full SMS command grammar test including malformed input; IVR flow test in all 8 languages; USSD session timeout and resume tests. Network chaos harness (<code>toxiproxy</code>) for D2's offline testing.
Documentation	SMS command reference (published for users), IVR flow diagram, telecom runbook, DLT template register.
Git branches	<code>feat/p08-d4-telecom-gateway</code> , <code>feat/p08-d4-sms-protocol</code> , <code>feat/p08-d4-ivr-flow</code> , <code>feat/p08-d4-ussd-session</code> , <code>feat/p08-d4-network-chaos</code>
Time	6 ideal days
Demo	On an actual ₹1,200 feature phone: send <code>MADAD</code> by SMS, receive a reply, confirm the location, get the mechanic's name and number, and cancel. Then call the IVR number and do the same in Tamil. No smartphone anywhere in the demo.

SMS command grammar

Command	Aliases	Effect
<code>HELP</code>	<code>MADAD</code> , <code>SAHAYA</code> , <code>H</code>	Start an assistance request; replies asking for location confirmation
<code>HELP <type></code>	<code>HELP CAR</code> , <code>HELP BIKE</code>	Start with the vehicle type pre-filled
<code>SOS</code>	<code>EMERGENCY</code> , <code>112</code>	Emergency — highest priority, immediate escalation
<code>STATUS</code>	<code>S</code>	Current booking status and mechanic ETA
<code>CANCEL</code>	<code>C</code>	Cancel the active booking (confirms first)
<code>LANG <code></code>	<code>LANG HI</code>	Change the language for all future messages
<code>STOP</code>		Opt out (TRAI requirement)

Unrecognized input replies with the command list rather than an error — a user who mistypes should not be stuck.

Phase 9 — Maps · S6 W13 · 3 days

Goal	Host our own map infrastructure so tiles cost compute, not licence fees.
Deliverables	Self-hosted tile server, Valhalla routing service, map pack build pipeline, CDN configuration.
Components	OSM India extract → tile generation (Tilemaker/Planetiler) → object storage → CDN. Valhalla for routing/ETA with India tiles. Nightly pack build pipeline producing per-district offline packs with delta updates so a refresh downloads kilobytes, not 60 MB.
Risks	Tile generation for all of India being computationally heavy → build once, update weekly, serve from a CDN. This is a batch problem, not a serving problem, and it's cheap when treated as one.
Security	Tile CDN with referrer/token restrictions to prevent our tiles being hotlinked and becoming someone else's free map service.
Testing	Tile serving load test; routing accuracy spot-check against ground truth on 20 routes (with D2); pack integrity verification.
Documentation	Map infrastructure runbook, tile update procedure.
Git branches	<code>feat/p09-d4-tile-infrastructure</code> , <code>feat/p09-d4-valhalla-routing</code> , <code>feat/p09-d4-map-pack-pipeline</code>
Time	3 ideal days
Demo	The cost comparison: our self-hosted tile serving cost per million requests versus the commercial alternative at national scale. It's a large number.

Phase 10 — Vehicle Diagnostics · S6 W14 · 2 days (support role)

Goal	Secure handling of diagnostic media and infrastructure for photo processing.
Deliverables	Media storage pipeline (upload, virus scan, EXIF strip, transcode, lifecycle), BLE testing infrastructure.
Components	Direct-to-S3 presigned uploads (never proxying image bytes through the API). Async pipeline: virus scan → EXIF strip → transcode to AVIF/WebP → thumbnail → notify. Lifecycle: auto-delete at 180 days.
Risks	Malicious file upload → content-type validation, magic-byte verification, virus scanning, size limits, and rendering only from a separate origin so a malicious file can't execute in our context.
Security	EXIF stripped on ingest (location data in a photo is a privacy leak nobody thinks about). Presigned URLs are short-lived and scoped. Bucket is private with no public access, verified by a Terraform policy check.
Testing	Malicious upload test corpus (polyglot files, zip bombs, oversized images, wrong content types).
Documentation	Media pipeline runbook, retention policy.
Git branches	<code>feat/p10-d4-media-pipeline</code>
Time	2 ideal days
Demo	Upload a photo with GPS EXIF data and an embedded payload. Show both stripped, and the payload quarantined.

Phase 11 — Emergency Engine · S7 W15 · 4 days

Goal	Give the emergency service a reliability class the rest of the platform doesn't have.
Deliverables	Isolated emergency infrastructure, dedicated alerting, degraded-mode path verification, chaos testing of the emergency path, 112 integration security.
Components	Emergency service on its own node pool, its own database connection pool, its own SMS vendor quota, spread across 3 AZs, with a PodDisruptionBudget that keeps replicas available during any deploy. Dedicated alert channel with a 5-minute ack SLA. The degraded path deliberately shares zero runtime dependencies with the main platform — separate ingress, separate direct SMS vendor connection, separate minimal database.
Risks	R9 — the emergency service failing during a real emergency. This phase's entire purpose is to make that as close to impossible as one team can achieve. Chaos testing is the proof, not the architecture diagram.
Security	112 handoff over mTLS with signed, replay-protected payloads. Break-glass medical access logged to an append-only store that even an admin cannot alter. Emergency endpoints rate-limited generously but not unlimited (a flood of fake SOS calls is a real attack).
Testing	The most important chaos test in the project: scale every non-emergency service to zero, sever the main database, and verify an SOS still reaches a responder. Latency test asserting p95 < 10 s under load. Monthly drill with a written report.
Documentation	Emergency runbook (the highest-priority document in the repo — printed and physically available, because "the runbook is in Confluence" is not a plan during an outage), drill procedure, 112 integration spec.
Git branches	<code>feat/p11-d4-emergency-isolation</code> , <code>feat/p11-d4-emergency-chaos-tests</code> , <code>docs/p11-d4-emergency-runbook</code>
Time	4 ideal days
Demo	Scale the entire platform to zero replicas, live, in front of everyone. Then trigger an SOS from a phone. It works.

Phase 12 — Government Dashboard · S7 W16 · 3 days (support role)

Goal	Government-grade access control and auditability.
Deliverables	mTLS infrastructure for gov clients, certificate lifecycle management, IP allowlisting, gov-specific WAF rules, audit log infrastructure.
Components	Internal CA for government client certificates, with issuance, renewal, and revocation procedures. Per-jurisdiction IP allowlists. Append-only audit log with hash chaining so tampering is detectable.
Risks	Certificate expiry locking out a government user with no easy recovery → expiry monitoring with 60/30/7-day alerts, and a documented emergency re-issuance procedure.
Security	Certificates are per-officer, not per-department, so audit trails identify individuals. Revocation is immediate and tested. Audit log is write-only from the application's perspective; no application role can delete or modify it.
Testing	Certificate lifecycle test (issue, use, revoke, verify rejection). Audit log tamper detection test.
Documentation	Certificate management runbook, government onboarding procedure.
Git branches	<code>feat/p12-d4-mtls-infrastructure</code> , <code>feat/p12-d4-audit-log-infra</code>
Time	3 ideal days

Demo	Revoke an officer's certificate; their next request is rejected within seconds. Then attempt to alter the audit log and show the hash chain detecting it.
-------------	---

Phase 13 — Analytics · S8 W17 · 3 days (support role)

Goal	Analytics infrastructure that cannot affect production.
Deliverables	ClickHouse cluster, CDC infrastructure, analytics network isolation, dashboard hosting.
Components	ClickHouse on separate nodes, reading from a read replica and the CDC stream — never from the primary. Debezium with replication slot lag monitoring and a documented kill switch, because an unmonitored replication slot can fill a disk and take down the primary database.
Risks	Replication slot growth taking down Postgres — a genuinely common production incident. Alert on slot lag as a P1, with an automated kill switch above a threshold.
Security	PII scan on the ClickHouse schema in CI. Analytics access role-restricted and audited.
Testing	Pipeline lag test; failure-recovery test (kill the CDC connector mid-stream and verify no data loss on restart).
Documentation	Analytics infrastructure runbook, CDC operations guide.
Git branches	<code>feat/p13-d4-clickhouse-cluster</code> , <code>feat/p13-d4-cdc-infrastructure</code>
Time	3 ideal days
Demo	Kill the CDC connector for 10 minutes. Restart it. Show zero data loss and the backlog clearing.

Phase 14 — Testing · S8 W18 · 10 days · You own this phase

Goal	Find the problems now, not in production, and prove the quality bar is met with evidence.
Deliverables	Load test report, security test report, chaos experiment log, device compatibility matrix, offline test matrix results, pentest findings, bug burn-down to zero P0/P1.
Folder structure	<code>tests/{load,security,chaos,e2e,offline}/</code>
Components	See the test plan below.
AI models	Test D3's models for robustness and adversarial inputs alongside D3.
Libraries	<code>k6</code> , <code>playwright</code> , <code>detox</code> , <code>owasp-zap</code> , <code>semgrep</code> , <code>trivy</code> , <code>toxiproxy</code> , <code>chaos-mesh</code> , <code>artillery</code> (WebSocket load).
Risks	Finding an architectural problem this late → mitigated by the load smoke test running since S3 and the chaos tests starting in P11 rather than all landing here. This phase should confirm, not discover. If it discovers something fundamental, that's a signal we got the earlier phases wrong.
Security	Full ASVS L2 checklist. External pentest if budget allows; otherwise a structured internal red-team pass with a written scope.

Testing	The full matrix below.
Documentation	Test reports (load, security, chaos, compatibility), known-limitations register, go/no-go recommendation for production.
Git branches	<code>test/p14-d4-load-tests</code> , <code>test/p14-d4-security-tests</code> , <code>test/p14-d4-chaos-experiments</code> , <code>test/p14-d4-offline-matrix</code>
Time	10 ideal days
Demo	Present the go/no-go with evidence. Every claim backed by a number and a test run. Including the things that failed.

Load test plan (k6)

Scenario	Profile	Pass criteria
Steady state	10k RPS, 30 min	p95 < 300 ms, error rate < 0.1%
Peak spike	10k → 50k in 60 s	Graceful degradation, no cascading failure, recovery < 3 min
Booking surge	5k bookings/min, 10 min	All bookings persisted, no double-assignment
WebSocket tracking	100k concurrent connections	Memory stable, message latency p95 < 500 ms
Sync storm	50k devices reconnecting simultaneously (a real scenario after a network outage)	Queue drains, no data loss, backpressure holds
Emergency burst	500 concurrent SOS	All notified p95 < 10 s
Soak	2k RPS, 24 h	No memory growth, no connection leak, no disk fill

Chaos experiments

Experiment	Expected behaviour
Kill 1 of 3 API pods	No user-visible impact
Kill all AI gateway pods	Rules fallback serves everything; no errors surfaced to users
Kill the primary database	Failover < 60 s; in-flight requests fail cleanly with retry guidance
Network partition between gateway and dispatch	Bookings queue; users see honest "finding a mechanic" state, not an error
Saturate the DB connection pool	Requests queue then shed with 503 + Retry-After; no crash
Expire a TLS certificate	Alert fires 30 days before; simulated expiry is caught in staging
Fill the disk on a node	Pod evicted and rescheduled; alert fires
Scale everything except Emergency to zero	SOS still works via the degraded path
SMS vendor 1 total outage	Automatic failover to vendor 2, no user impact
Clock skew of 5 minutes on one node	Token validation still works within the skew tolerance

Offline test matrix

Condition	Expected
Full airplane mode	All local features work; queued ops visible; honest offline UI
2G with 10% packet loss	Requests succeed with retry; no duplicate bookings
Connectivity lost mid-booking	Booking queued; syncs on reconnect; no duplicate on retry
App killed mid-sync	Queue intact on restart; sync resumes from the cursor
Device rebooted with 100 queued ops	All 100 replay correctly, in order
Clock skew ±24 h	Conflict resolution still correct (HLC, not wall clock)
Storage full	Graceful degradation; user prompted; no crash, no corruption
Two devices, same account, both offline, conflicting edits	Conflicts resolved per the matrix; ambiguous cases surfaced to the user
Sync interrupted at op 47 of 100	99 applied, op 47 reported, no partial-state corruption

Security test checklist (ASVS L2 extract)

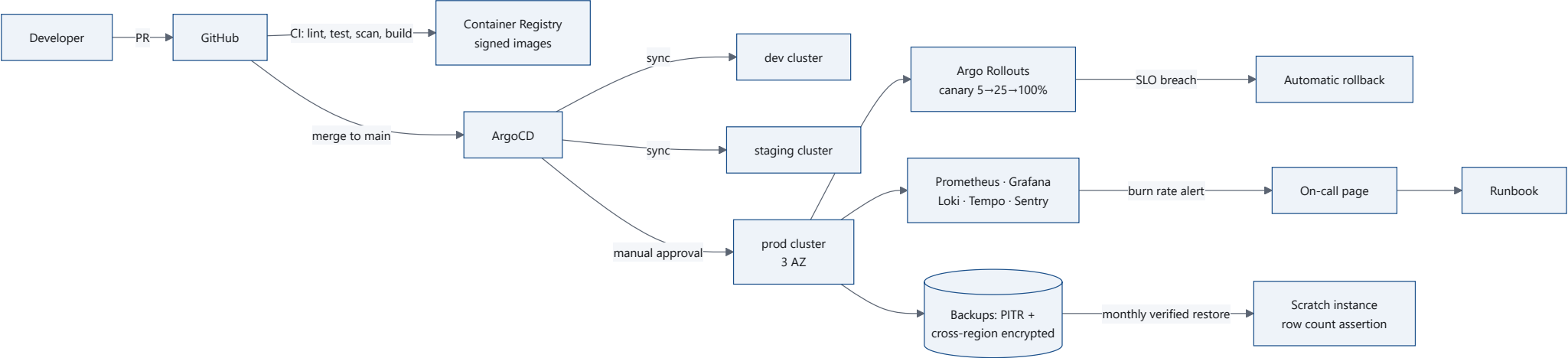
Area	Tests
Authentication	Brute force, credential stuffing, session fixation, token replay, refresh reuse, OTP enumeration, SIM-swap scenario
Authorization	IDOR across every resource type, privilege escalation, tenant isolation (fleet + government), mass assignment
Input	SQLi, NoSQLi, command injection, path traversal, XXE, SSRF, prompt injection on AI paths
Output	XSS (stored, reflected, DOM), CSRF, clickjacking, open redirect
Crypto	TLS configuration, cert pinning verification, key storage, weak-algorithm scan
Data	PII in logs, PII in analytics, PII in error responses, backup encryption, EXIF leakage
Mobile	Root/jailbreak handling, reverse engineering resistance, local storage inspection, deep-link abuse, screenshot leakage
API	Rate limit bypass, mass enumeration, verbose errors, missing auth on any route (automated route-coverage scan)
Infra	Exposed ports, misconfigured buckets, container escape surface, secrets in images, over-permissive IAM

Phase 15 — Deployment · S9 W19 · 8 days · You own this phase

Goal	Production infrastructure a four-person team can operate at 3 a.m.
Deliverables	Full Terraform for production, Kubernetes manifests, ArgoCD GitOps, progressive delivery, DR plan with a passed drill, 25+ runbooks, on-call rotation, cost monitoring.
Folder structure	<code>infra/terraform/{modules,envs}/</code> , <code>infra/k8s/{base,overlays}/</code> , <code>docs/runbooks/</code>
Components	Everything below.

Libraries	<code>terraform</code> , <code>helm</code> , <code>kustomize</code> , <code>argocd</code> , <code>argo-rollouts</code> , <code>cert-manager</code> , <code>external-secrets</code> , <code>prometheus-operator</code> .
Risks	(a) Infrastructure complexity exceeding what one person can operate → deliberate simplicity: managed services wherever available, no service mesh, one cluster per environment, boring choices everywhere. (b) A DR plan that has never been executed → the drill is a deliverable of this phase, with a written report, not a document that says we would be fine.
Security	Everything encrypted in transit and at rest. No standing production access — break-glass with time-limited, audited elevation. Network policies default-deny. Pod security standards enforced. Image signing and admission control.
Testing	Rollback drill per service. DR drill: restore into a clean region, measure RTO and RPO against targets. On-call readiness test: give a runbook to someone who didn't write it and have them resolve a simulated incident.
Documentation	25+ runbooks (one per alert), DR plan, on-call handbook, production access policy, cost model.
Git branches	<code>feat/p15-d4-terraform-production</code> , <code>feat/p15-d4-argocd-gitops</code> , <code>feat/p15-d4-progressive-delivery</code> , <code>feat/p15-d4-dr-plan</code> , <code>docs/p15-d4-runbooks</code>
Time	8 ideal days
Demo	Deploy a change through the full pipeline to production with one approval click. Then deliberately break the canary and watch it roll back automatically. Then run the DR drill and restore the platform into a clean region, timed, live.

Deployment architecture



Runbook catalogue (25 minimum)

Category	Runbooks
Availability	Service down · High error rate · Elevated latency · Pod crashloop · Node not ready
Database	Connection pool exhausted · Replication lag · Slot growth · Disk pressure · Slow query storm · Failover
Emergency	SOS not delivering · 112 handoff failing · Crash detection false-positive spike · Degraded mode activation
Telecom	SMS not delivering · Vendor failover · DLT template rejection · IVR call failures
Security	Suspected breach · Credential leak · Certificate expiry · WAF blocking legitimate traffic · Fraud spike
Data	Backup failure · Restore procedure · CDC lag · Data corruption suspected
Deployment	Rollback procedure · Failed migration · Stuck ArgoCD sync · Cost anomaly

Phase 16 — Optimization · S9 W20 · 5 days

Goal	Right-size everything and make cost per user a tracked number.
Deliverables	Infrastructure right-sizing, autoscaling tuning, spot instance adoption, CDN optimization, cost model per 100k MAU, performance budgets in CI.
Components	Node right-sizing from actual usage data (almost always over-provisioned after an initial build). HPA tuning on the right metric — for dispatch that's queue depth, not CPU. Spot instances for batch and non-critical workloads with a documented interruption strategy. CDN caching rules. Cost allocation tags so we know what each service costs.
Risks	Over-optimizing for cost and hurting reliability → the emergency service is explicitly exempt from spot instances and aggressive downscaling. Some things should cost more.
Testing	Load test after right-sizing — confirm the SLOs still hold at the new capacity. Spot interruption simulation.
Documentation	Cost model per 100k MAU with a breakdown by service, optimization report, capacity planning guide.
Git branches	<code>perf/p16-d4-infra-rightsizing</code> , <code>perf/p16-d4-autoscaling-tuning</code> , <code>feat/p16-d4-cost-monitoring</code>
Time	5 ideal days
Demo	The cost curve before and after, and the projected cost per user at 100k, 1M, and 10M MAU. This is the slide investors ask about.

Phase 17 — Future Roadmap · S10 · 3 days

Goal	Define what operating this at national scale requires.
Deliverables	Infrastructure scaling plan, reliability roadmap, security roadmap, compliance certification plan, operations maturity assessment.

Components	Scaling plan: multi-region active-active, database sharding trigger points, CDN expansion, the specific point at which one cluster stops being enough. Compliance roadmap: ISO 27001 and SOC 2 timeline and cost, DPDP audit readiness, government empanelment requirements. Operations maturity: what a 4-person team can sustain versus what needs the 8th and 12th hires, and in which order.
Documentation	Scaling plan, reliability roadmap, security roadmap, compliance certification plan with cost and timeline, hiring recommendation for operations.
Git branches	<code>docs/p17-d4-infra-roadmap</code> , <code>docs/p17-d4-compliance-roadmap</code>
Time	3 ideal days
Demo	Present the honest operational picture: what we can run today, where it breaks, and what the next hire should be.

D4 Effort Summary

Phase	Days	Phase	Days
P0 Research	4	P9 Map infra	3
P1 Planning	4	P10 Media pipeline	2
P2 Threat model	4	P11 Emergency infra	4
P3 DB infra	2	P12 Gov mTLS	3
P4 Secrets + SMS	3	P13 Analytics infra	3
P5 Observability	4	P14 Testing	10
P6 Mobile CI	3	P15 Deployment	8
P7 ML infra	3	P16 Optimization	5
P8 Telecom gateway	6	P17 Roadmap	3
		Total	74 ideal days

The remainder covers on-call, CI maintenance (a continuous cost that plans routinely forget), incident response, security reviews on every PR touching auth or PII, and the compliance evidence-gathering that runs throughout rather than in a single phase.